

FAULT-TOLERANT AND COST-EFFICIENT SERVERLESS WORKFLOW EXECUTION ON SPOT INSTANCES

by

BANDHAN PATEL

Under the Direction of In Kee Kim

ABSTRACT

Cloud spot instances cost 3 to 10 times less than on-demand infrastructure, but a preemption anywhere in a long-running workflow can cascade through its dependency graph, stalling every parallel branch and forcing a full restart. Without fault tolerance, complex DAG workflows complete only 39% of spot runs on average. Existing fault-tolerant systems recover reliability by mixing in on-demand capacity—a strategy that directly erodes the cost advantage that motivated spot use.

This thesis presents *FaaSpot*, a serverless workflow execution system that runs entirely on spot instances with no on-demand fallback. The key insight is that preemption impact is determined by *topology*: fan-out nodes and synchronization barriers carry orders-of-magnitude higher cascade potential than sequential tasks, and these positions are statically identifiable before execution begins. *FaaSpot* classifies each task by its structural cascade impact in a single offline DAG analysis and uses that single classification to drive three online mechanisms in coordination: selective checkpointing concentrates I/O at high-impact positions; lineage-based retry resumes execution from the nearest structural checkpoint rather than restarting from the workflow origin; and survival-aware scaling provisions additional capacity before preemption occurs, guided by a Kaplan-Meier instance lifetime estimator and a Thompson Sampling adaptive controller.

Evaluation across eight DAG workflows on a trace-driven spot emulator shows that *FaaSpot* achieves 89.03% average success at \$0.018 per successful run and 126 seconds mean latency—outperforming four state-of-the-art baselines, all of which use on-demand fallback, on every metric simultaneously. Checkpoint efficiency analysis reveals that *FaaSpot* achieves this reliability with 10.6

checkpoints per successful run, $3.8\times$ fewer than the most checkpoint-heavy baseline, confirming that structural placement outperforms uniform frequency.

INDEX WORDS: Cloud Computing, Public Cloud Infrastructure,
Workflow execution, Spot instances, Cost efficiency,
Fault Tolerance

FAULT-TOLERANT AND COST-EFFICIENT SERVERLESS WORKFLOW
EXECUTION ON SPOT INSTANCES

by

BANDHAN PATEL

B.S in Computer Science, Georgia Institute of Technology, 2021

A Thesis Submitted to the Graduate Faculty of the
University of Georgia in Partial Fulfillment of the Requirements for the
Degree.

MASTERS OF SCIENCE

ATHENS, GEORGIA

2026

FAULT-TOLERANT AND COST-EFFICIENT SERVERLESS WORKFLOW
EXECUTION ON SPOT INSTANCES

by

BANDHAN PATEL

Major Professor: In Kee Kim

Committee: Kyu Hyung Lee
Wei Niu

Electronic Version Approved:

Ron Walcott
Dean of the Graduate School
The University of Georgia
August 2026

DEDICATION

I dedicate this thesis to my family, friends, and mentors, whose unwavering support and encouragement have been the foundation of this journey. Their belief in me provided strength during moments of doubt and helped me push forward through challenges.

I am grateful for their guidance and the values they have instilled in me, which continue to shape who I am today.

I also dedicate this work to myself, as a reflection of the persistence and effort invested in reaching this milestone.

ACKNOWLEDGMENTS

I would like to sincerely thank my advisor, Dr. In Kee Kim, for his continuous guidance, support, and valuable feedback throughout this project. His insights played a crucial role in shaping both the direction and quality of this work.

I am also grateful to Dr. Kwangsung Oh, Dr. Kyu Hyung Lee, and Dr. Wei Niu for their time and support in serving on my committee.

I would like to thank my project team members, Sushruth Harsha and Preetam Jain, for their collaboration, thoughtful discussions, and contributions throughout this project.

CONTENTS

Acknowledgments	v
List of Figures	viii
1 Introduction	1
1.1 The Structural Insight	2
1.2 Thesis Hypothesis	2
1.3 FaaSpt: A Structural Approach to Spot Reliability	3
1.4 Key Results	4
1.5 Contributions	5
2 Background and Related Work	7
2.1 Cloud Computing	7
2.2 Serverless Computing	8
2.3 Spot Instances	9
2.4 DAG Workflow Fundamentals	11
2.5 Checkpointing Fundamentals	12
2.6 Survival Analysis and Kaplan-Meier Estimation	15
2.7 Related Work	16
3 Motivation and Failure Characterization	18
3.1 Naive Serverless Deployment on Spot: Measurement	18
3.2 Structural Root Cause: Why Complex Workflows Fail More	20
3.3 The Design Opportunity	20
4 FaaSpt System Design	25
4.1 Architecture Overview	25
4.2 Task Ranker	28
4.3 Checkpoint System: Planner and Manager	29
4.4 Adaptive Scaler	31
4.5 Retry Manager	36

5	Implementation	39
5.1	Platform Stack	39
5.2	Offline Planning Components	40
5.3	Runtime Control Components	40
5.4	Spot Preemption Emulator	41
5.5	Experiment Orchestration	44
6	Evaluation	46
6.1	Experimental Setup Summary	46
6.2	Success Rate	48
6.3	Cost Efficiency	51
6.4	Cost-Reliability Pareto Frontier	55
6.5	End-to-End Latency	56
6.6	Checkpoint Efficiency	63
6.7	Ablation Study	65
6.8	Parameter Sensitivity Analysis	69
6.9	Summary of Results	70
6.10	Discussion	71
7	Limitations and Future Work	74
7.1	System Scope and Core Assumptions	74
7.2	Threats to Validity	76
7.3	Practical Implications for Deployment	79
7.4	Future Work	80
8	Related Work	82
8.1	Survey of Related Work	82
8.2	Comparison and Positioning	85
9	Conclusion	87
9.1	Final Perspective	87
A	Supplementary Evaluation and Reproducibility Details	89
A.1	Reproducibility Checklist	89
A.2	Workflow and Baseline Inventory	90
A.3	Key Aggregate Results Snapshot	92
A.4	Per-Workflow Result Tables	93
A.5	Metric Definitions Used in Analysis	93
A.6	Bandit and Survival Hyperparameters	94
A.7	Notes for Future Extensions	95

LIST OF FIGURES

2.1	A simple six-task DAG workflow. Task <i>B</i> is a <i>fan-out</i> node: it spawns tasks <i>C</i> and <i>D</i> as independent parallel branches. Task <i>E</i> is a <i>fan-in</i> node (synchronization barrier): it cannot begin until <i>both</i> <i>C</i> and <i>D</i> complete. A spot preemption of <i>C</i> or <i>D</i> therefore stalls <i>E</i> and every task downstream of it, even though only one task was actually preempted.	11
2.2	Cascade failures in three representative workflows. A spot preemption at a fan-out node or at a task feeding a synchronization barrier (red) forces every parallel branch to restart (orange) and leaves downstream tasks stalled (gray). The structural position of a failure—not its duration—determines its blast radius.	13
2.3	Kaplan-Meier survival curve for p3.2xlarge spot instances in us-west-2a estimated from real production traces (Wu et al., 2024). Survival probability falls steeply within the first few hours, then tails off. FaaS uses this curve to classify each running instance into a risk tier and trigger proactive scale-out before preemption occurs.	16
3.1	Success rate, cost, and latency of naive serverless deployment on spot vs. on-demand across eight workflows (wf-1 through wf-8). Each workflow is run 500 times on the trace-driven emulator using real p3.2xlarge spot lifetime traces. Spot runs have no fault-tolerance mechanism; a preemption anywhere fails the entire run. (a) Success rate falls well below 50% on complex workflows, dropping to $\sim 13\%$ on the most complex. (b) Cost per successful run remains below on-demand on every workflow, but the margin is modest (3%–43%), far below the nominal $2\times$ – $10\times$ spot discount. (c) End-to-end latency on spot exceeds on-demand by up to $2.7\times$ because every failure forces a full restart.	22

4.1	FaaS _{Spot} system architecture. The offline phase derives $\kappa(v)$ and $\ell(v)$ from the workflow DAG before execution begins. Three online fault-tolerance controllers consume these signals at runtime. All runtime state is persisted to a shared storage layer; the execution plane runs task pods on a shared spot VM pool.	26
4.2	Three-tier checkpoint classification. Task B is the fan-out node: its single-replica output spawns parallel branches, so the replica count ρ changes across each outgoing edge— B is CRITICAL . The parallel branch tasks (C, D) run with $\rho > 1$ and no outgoing replica change—they are MEDIUM . Tasks A, E, F are single-replica sequential tasks with no replica transition—they are LOW . A CRITICAL checkpoint at B means any branch failure recovers from B 's output rather than restarting from A	29
6.1	Success rate by workflow and baseline. FaaS _{Spot} (FS) achieves the highest or tied-highest completion rate on six of eight workflows. The gap widens on complex workflows with wide fan-out stages, where baseline methods suffer cascading failures.	49
6.2	Total experiment cost by workflow and baseline. FaaS _{Spot} (FS) achieves high reliability without proportionate cost increases because checkpoint-based retry avoids re-executing already-completed work.	52
6.3	Cost per successful run by workflow and baseline (lower is better). FaaS _{Spot} achieves the lowest cost per success on 7 of 8 workflows.	53
6.4	Cost-reliability Pareto plots for the four representative workflows. Each point is one method at its measured (cost-per-success, success-rate) operating point; FaaS _{Spot} is drawn as a star. The upper-left corner is the dominant region (higher reliability at lower cost). FaaS _{Spot} occupies the dominant region on (a), (c), and (d). On (b) wf-4, FaaS _{Spot} and MScheduler are non-dominating with respect to each other on success rate alone, but FaaS _{Spot} dominates on cost per success.	58
6.5	Average end-to-end latency by workflow and baseline (lower is better). FaaS _{Spot} (FS) is consistently the fastest method, with the largest margins on complex workflows where recovery overhead dominates.	59

6.6	End-to-end latency distribution (violin plot) by workflow and baseline. FaaS _{Spot} (FS) shows not only a lower median but also a tighter distribution, indicating fewer extreme-latency outliers caused by repeated full-workflow restarts.	61
6.7	Checkpoints per successful run by workflow and baseline (lower is better). FaaS _{Spot} (FS) achieves higher success rates with substantially fewer checkpoints than Hourglass, indicating that selective structural placement uses checkpoint I/O more productively than high-frequency uniform placement.	64
6.8	Ablation study: effect of degrading each mechanism. Variant A (uniform checkpointing) degrades success and inflates cost relative to FaaS _{Spot} even though the same number of checkpoints is taken. Variant B (no scaling) reduces reliability under sustained preemption pressure. Variant C (naïve retry) causes the largest reliability and latency degradation by discarding all partial progress on every failure.	67

CHAPTER I

INTRODUCTION

Serverless platforms are increasingly adopted to run complex DAG workflows, from data analytics and machine learning to scientific computing (Carver et al., 2020; Malawski et al., 2020; Roy, Patel, and Tiwari, 2022). To run them cheaply, practitioners increasingly turn to cloud spot instances, which are 3 to 10 times cheaper than on-demand instances. But, these spot instances are unreliable – a provider can reclaim a spot instance at any moment, typically with just two minutes’ warning (Wu et al., 2024, Amazon Web Services, 2026). For individual tasks or short batch jobs, this is an inconvenience. For serverless workflows—pipelines of dozens to hundreds of interdependent tasks (functions) that can run for hours—the consequences are far more severe. A single preemption at the wrong position in a workflow’s dependency graph can stall every parallel branch, forcing the entire multi-hour pipeline to restart from scratch. One failure becomes many.

In our baseline measurement, this cascade effect is stark. Deploying representative DAG workflows on spot instances without any fault-tolerance mechanism yields only a 39% average completion rate across eight workflows—while identical workflows run on on-demand instances complete every time (Chapter 3). Worse, the repeated restarts caused by preemptions erode spot’s price advantage: the cost per successful run on naive spot deployment is often within 3–25% of on-demand, far below the nominal 3–10 $\times$ discount. Cheap infrastructure is only cheap if it finishes the work.

Existing fault-tolerance approaches restore reliability by spending on non-spot resources. Hybrid systems (Joaquim et al., 2019, F. Yang et al., 2023, Portella et al., 2023) maintain an on-demand fleet as a safety net, migrating preempted tasks to stable capacity when spot fails. DAG-aware systems (Yan et al., 2016, Y. Yang et al., 2017) identify which positions in a workflow are most vulnerable and protect them—by placing those tasks on reserved nodes. Every approach in

the literature anchors reliability to the presence of non-spot capacity, treating it as a necessary condition for completing complex workflows under preemption.

This thesis challenges that assumption directly. The central question is: *can pure-spot workflow execution—with no on-demand fallback whatsoever—match or exceed the reliability of hybrid approaches, while still delivering spot-level economics?*

The answer, as this thesis demonstrates, is yes—but only when the three pillars of fault tolerance (selective checkpointing, lineage-based retry, and survival-aware scaling) are jointly coordinated around a single, offline structural analysis of the workflow graph.

1.1 The Structural Insight

The key observation underlying this work is that preemption consequences are determined by *where* a failure occurs in the workflow graph, not just *whether* it occurs. A preemption on a sequential task forces only that task to recompute. A preemption at a fan-out node—a task that spawns many parallel branches—blocks every branch it spawned. A preemption at any task feeding a synchronization barrier—a point where all parallel branches must complete before downstream execution continues—stalls every branch waiting at that barrier, even branches that have already completed their work (Poolal et al., 2016, Monge et al., 2020).

These structurally critical positions are deterministic: they follow from the workflow graph’s topology, not from runtime state. They can be identified *before execution begins*, from the directed acyclic graph (DAG) submitted at workflow entry time. If a system concentrates its fault-tolerance effort at these positions—and *only* these positions—it can absorb preemptions with a fraction of the overhead of a system that treats all tasks uniformly.

This observation motivates a design principle that runs through every component of this thesis: *offline structure-aware planning, tightly coupled with runtime adaptive control, is sufficient for reliable pure-spot execution.*

1.2 Thesis Hypothesis

Hypothesis: *Spot-only workflow execution can remain reliable and cost-efficient when offline structure-aware planning is tightly coupled with runtime adaptive control.*

This hypothesis has two falsifiable predictions. First, a pure-spot system that concentrates fault-tolerance effort at structurally critical DAG positions

should achieve reliability competitive with—or exceeding—hybrid systems that use on-demand fallback. Second, this reliability improvement should not come at the cost of economic efficiency: checkpoint-based recovery from structural positions, rather than full-workflow restart, should reduce the wasted compute that drives cost per successful run upward.

Evaluation across eight DAG workflows on a trace-driven spot emulator (Chapter 6) confirms both predictions. FaaS_{Spot} achieves an aggregate 89.03% success rate at \$0.018 per successful run and a mean latency of 126 seconds—outperforming all four hybrid baselines on aggregate success rate, cost, and latency, despite operating under an even tighter constraint: no on-demand capacity whatsoever.

1.3 FaaS_{Spot}: A Structural Approach to Spot Reliability

To evaluate the thesis hypothesis, this thesis introduces **FaaS_{Spot}**, a serverless workflow execution system built for pure-spot infrastructure. FaaS_{Spot} analyzes the workflow DAG once at submission time and classifies each task by the potential cascade impact of its preemption. This single offline classification then drives three online fault-tolerance mechanisms in a coordinated closed loop—with no runtime coordination overhead between them.

- **Selective checkpointing:** FaaS_{Spot} assigns each task to one of three tiers—Critical, Medium, or Low—based on its structural position relative to fan-out nodes and synchronization barriers. Critical tasks are always checkpointed upon completion; Low tasks are never checkpointed. Checkpoint I/O is therefore concentrated at exactly the positions where a missed checkpoint would trigger the largest recomputation cascade (Aupy et al., 2016, Young, 1974).
- **Lineage-based retry:** When a spot preemption occurs, the retry manager traverses the DAG backward from the failed task to find the nearest completed Critical or Medium checkpoint. Execution resumes from that point, replaying only the downstream sub-DAG rather than restarting from the beginning of the workflow. This converts a potential hours-long restart into a bounded partial re-execution (Poolal et al., 2016, Kail et al., 2016).
- **Survival-aware adaptive scaling:** A Kaplan-Meier survival estimator (Kaplan and Meier, 1958) tracks per-instance preemption risk from ob-

served uptime. When an instance’s conditional survival probability falls below a tier threshold, a Thompson Sampling bandit (Auer et al., 2002; Agrawal and Goyal, 2013) selects scale-out actions—prioritized by HEFT task rank (Topcuoglu et al., 2002)—to provision capacity on safer instances before the preemption occurs. This proactive intervention prevents retry-queue saturation on the workflows where cascade failures are most likely.

Because all three mechanisms are guided by the same offline classification, their decisions are inherently consistent without any runtime coordination. The checkpoint manager knows which positions are structurally critical; the retry manager knows which checkpoints are worth resuming from; the adaptive scaler knows which tasks should receive protective replicas first. The result is a closed loop where planning, recovery, and resource control reinforce each other rather than operating in isolation.

To enable fair and reproducible evaluation, FaaS_{Spot} includes a trace-driven spot emulator that replays empirical instance lifetime distributions from the SkyPilot dataset (Wu et al., 2024) under controlled time compression. The emulator exposes all evaluated systems to identical preemption sequences within each trial, ensuring that reported differences reflect algorithmic behavior rather than variation in underlying spot conditions.

1.4 Key Results

Table 1.1 summarizes FaaS_{Spot}’s performance against four state-of-the-art baselines across eight DAG workflows. Every baseline invokes on-demand capacity as a reliability backstop; FaaS_{Spot} does not.

Table 1.1: Summary of FaaS_{Spot} vs. baselines on three primary metrics (Chapter 6). FaaS_{Spot} is the only spot-only system; all baselines use on-demand fallback.

System	Success (%)	Latency (s)	Cost/run (\$)	On-demand fallback?
FaaS_{Spot} (this work)	89.03	126	0.018	No (pure spot)
MScheduler (Portella et al., 2023)	81.78	334	0.040	Yes
Hourglass (Joaquim et al., 2019)	76.22	506	0.047	Yes
Snape (F. Yang et al., 2023)	58.56	297	0.050	Yes (mixed)
Burst-HADS (Teylo et al., 2023)	53.43	290	0.259	Yes

Three results stand out. First, FaaS_{Spot} outperforms the best hybrid baseline (MScheduler) by 7.25 percentage points in success rate—while operating without the on-demand safety net that MScheduler relies on. The advantage widens on structurally complex workflows: on wf-5 (LIGO, with a dual-sub-pipeline synchronization barrier), FaaS_{Spot} achieves 86.2% completion while all four baselines fall below 41%. Second, FaaS_{Spot} delivers the best cost per successful run on all eight workflows—more than $2.2\times$ cheaper than the strongest baseline (MScheduler)—because checkpoint-based recovery dramatically reduces the wasted compute from failed and restarted runs. Third, FaaS_{Spot}’s mean latency (126 s) is $2.3\times$ lower than the nearest competitor and $4.0\times$ lower than Hourglass, with the tightest tail behavior: p99 latency is 346 s versus 2,199 s for Hourglass.

The ablation study further confirms that all three mechanisms are necessary. Removing lineage-based retry—while keeping checkpointing and scaling—drops aggregate success from 89.03% to 50.05% and more than quadruples cost per success. No single mechanism achieves the performance of the full integrated system.

1.5 Contributions

This thesis makes the following contributions to the field of cloud workflow systems:

1. **A pure-spot serverless workflow execution system.** FaaS_{Spot} demonstrates that complex, dependency-rich DAG workflows can be executed

reliably without on-demand fallback, closing a gap that the prior literature treated as fundamental. It outperforms every evaluated hybrid baseline on all three primary metrics.

2. **A unified offline DAG classification for multi-mechanism fault tolerance.** A single HEFT-based criticality ranking and three-tier checkpoint classifier drive selective checkpointing, lineage-based retry, and survival-aware scaling from the same shared signal—eliminating runtime coordination overhead while ensuring consistent protection of structurally critical tasks.
3. **A two-layer survival-gated contextual bandit for adaptive scaling.** The Kaplan–Meier estimator and the Thompson Sampling bandit are individually well-known; the contribution is their *coupling*: the KM layer constrains the bandit’s action set to a survival-appropriate range, and the bandit optimizes within that range against a cost-normalized reward. Sensitivity analysis shows this coupled policy outperforms the best fixed parameter setting by 11–17 percentage points on success rate for the most preemption-intensive workflows.
4. **A trace-driven spot emulator for reproducible evaluation.** The emulator replays real-world spot lifetime distributions under controlled time compression and exposes all methods to identical interruption sequences. It is decoupled from FaaSpt and can serve as an evaluation harness for future spot-aware workflow systems.
5. **A characterization of cascade failure structure and its implications for checkpoint design.** Chapter 3 documents how DAG topology determines the blast radius of each preemption, providing a structural model of why uniform checkpointing wastes I/O budget and why structure-aware placement achieves higher reliability with fewer checkpoints (10.6 per success for FaaSpt vs. 39.9 for the most checkpoint-heavy baseline).

CHAPTER 2

BACKGROUND AND RELATED WORK

This chapter provides the technical foundation for understanding FaaS_{Spot} and the problem it addresses. We begin with cloud computing—its delivery models and pricing tiers—then narrow the focus to the specific execution model and infrastructure class this thesis targets: serverless computing on spot instances. With that context in place, we formalize the DAG workflow model and explain why its dependency structure makes spot preemptions far more damaging than they are for independent batch jobs. We then cover checkpointing, the primary recovery primitive, and survival analysis, the statistical tool FaaS_{Spot} uses to anticipate preemption risk at runtime. The chapter closes with a survey of related systems and a precise statement of the design gap that motivates this work.

2.1 Cloud Computing

Cloud computing provides on-demand access to computational resources—virtual machines, storage, databases, and networking—without requiring users to own or manage physical hardware. Instead of provisioning a fixed cluster, users can dynamically allocate and release resources based on current workload requirements, paying only for what they consume. This elasticity makes cloud platforms particularly well-suited for large-scale DAG workflows, which often exhibit bursty, data-dependent execution patterns: a large compute campaign may require hundreds of cores for a few hours, then nothing for days.

2.1.1 Pricing Tiers

Within public clouds, providers offer multiple pricing models that trade cost against reliability:

- **On-demand instances** provide reliable, predictable access to compute with no long-term commitment. They are the most expensive option per unit time but carry strong availability guarantees.
- **Reserved instances** offer significant discounts (commonly 30–60%) in exchange for one- or three-year commitments. They are cost-efficient for steady, predictable workloads but are too inflexible for the bursty demand patterns common in scientific computing.
- **Spot (preemptible) instances** expose spare provider capacity at steep discounts—often $2\times$ – $10\times$ cheaper than on-demand—but can be reclaimed by the provider at any time with short notice (Wu et al., 2024, Poola et al., 2014).

For DAG workflows that run for minutes to hours, the cost gap between spot and on-demand is compelling. However, spot preemptions introduce failures that propagate through dependent tasks in ways that simple batch jobs never experience. Understanding and tolerating these failures is the central challenge this thesis addresses. To understand why the execution model matters as much as the pricing model, we next examine serverless computing—the platform layer that sits between the workflow scheduler and the underlying spot VMs.

2.2 Serverless Computing

Serverless computing is an execution model in which a developer deploys individual *functions* rather than managing virtual machines or servers directly. The cloud provider (or a self-managed platform) handles all infrastructure allocation: when a function is invoked, a short-lived container is spun up, the function runs to completion, and the container is released. From the developer’s perspective, there are no VMs to provision, patch, or scale manually. This model eliminates operational overhead and charges users only for the compute time actually consumed (Jonas et al., 2019).

Commercially, serverless functions are offered by every major cloud provider (AWS Lambda, Google Cloud Functions, Azure Functions). The same model is also available through open-source platforms designed to run on a user-controlled

pool of virtual machines—the regime directly relevant to this thesis. Two widely used open-source platforms are *Fission* (Fission Project, 2026), which runs serverless functions as Kubernetes pods, and *Apache Open Whisk* (Apache Software Foundation, 2024), which dispatches invocations across a container pool via a message-driven pipeline. FaaS is built on Fission running atop Kubernetes (The Kubernetes Authors, 2026).

Serverless platforms are increasingly used to execute complex *DAG workflows*—structured pipelines in which functions depend on the outputs of earlier functions. Scientific computing (genomics, astronomy, seismic analysis), data analytics, and machine learning training are common examples (Carver et al., 2020, Roy, Patel, Gadepally, and Tiwari, 2022, Malawski et al., 2020). These workflows are typically expressed and orchestrated through workflow-management platforms such as Apache Airflow (Apache Software Foundation, 2026) and Temporal (Temporal Technologies, 2026), which coordinate task dependencies but assume a non-preemptible substrate. As workflow makespans have grown from minutes to hours—Google Cloud now allows a single function to run for up to 60 minutes, and Azure has removed execution time limits entirely—deploying the underlying VM pool on cheap spot instances becomes increasingly attractive.

This combination—self-managed serverless on spot VMs—eliminates the per-function cost premium of managed services while using spot pricing, but it also means that the platform must tolerate VM preemptions entirely on its own, without a managed fallback to on-demand capacity. To understand how severe this exposure is, we must first examine how spot instances behave and what makes their preemptions so damaging for workflow execution.

2.3 Spot Instances

Spot instances give users access to spare cloud capacity at a steep discount—typically $2\times$ – $10\times$ cheaper than on-demand pricing (Wu et al., 2024). The trade-off is that the cloud provider can reclaim a spot instance with short notice—commonly a two-minute warning window on AWS (Amazon Web Services, 2026)—whenever demand for that capacity rises. From a scheduling perspective, spot resources behave as a *stochastic resource class*: the same instance type in the same availability zone can run undisturbed for hours on one day and be preempted within minutes on another. Lifetime characteristics vary significantly across instance types, regions, and time-of-day demand patterns (Wu et al., 2024, Son et al., 2024). Moreover, availability is highly correlated within a zone: Kim et al. (Kim et al., 2026) show that co-interruptions within the same

availability zone occur within three minutes in over 92% of observed events, highlighting that independent lifetime models understate zone-level failure exposure.

2.3.1 Pricing and Allocation Semantics

Modern spot provisioning (post-2017 AWS model) is driven by provider-side capacity pressure rather than user bids (Amazon Web Services, 2017). This simplified model makes spot instances easier to acquire, but it does not imply lifetime guarantees: a low spot price does not mean a long uninterrupted execution window—price and revocation risk are correlated but distinct. Prior work on spot price prediction using random forests (Khandelwal et al., 2020) and time-series forecasting (Cai et al., 2018) confirms that although price trajectories are learnable, they are only a weak predictor of imminent preemption. Schedulers must therefore treat spot capacity as inherently uncertain and design policies that remain effective across a *distribution* of interruption outcomes, not a single expected lifetime.

2.3.2 Implications for Workflow Execution

For workflows that complete in seconds, spot preemptions are uncommon enough to ignore. For workflows that run for tens of minutes to hours—the regime this thesis targets—a preemption is likely before the workflow finishes. Naive deployment of representative DAG workflows on spot instances yields a *39% average success rate* (Chapter 3), while the same workflows complete every run on on-demand. The cost-per-successful-run gap between spot and on-demand is far narrower than the nominal $2\times$ – $10\times$ price difference because repeated restarts consume additional compute.

From a cost-optimization standpoint, the scheduler faces the following objective: minimize the expected cost per successful completion,

$$J = \frac{E[C_{\text{run}}]}{P_{\text{succ}}},$$

where C_{run} is total execution cost and P_{succ} is completion probability. Naively minimizing unit-time cost by using spot without fault tolerance can *increase* J because P_{succ} falls faster than C_{run} (Poolal et al., 2014, Martinez et al., 2019). Theoretical analysis confirms that optimal spot scheduling under delay constraints requires balancing preemption risk against queueing delay in ways that simple greedy policies cannot (Bhuyan et al., 2026). A fault-tolerant mechanism that recovers efficiently from preemptions can reduce both the failure rate and

the cost of each recovery, recovering spot’s full price advantage. The severity of a preemption, however, is not uniform across workflow tasks—it depends critically on the DAG structure. We examine that structure next.

2.4 DAG Workflow Fundamentals

A *directed acyclic graph* (DAG) workflow $G = (V, E)$ represents a computation as a set of tasks V linked by precedence edges E : an edge (u, v) means task v cannot begin until task u has produced its output. Figure 2.1 shows a simple example. This model captures a wide range of scientific applications—from genomics pipelines with hundreds of parallel branches to multi-stage data analytics—and is the native representation used by serverless orchestration platforms such as Fission and OpenWhisk.

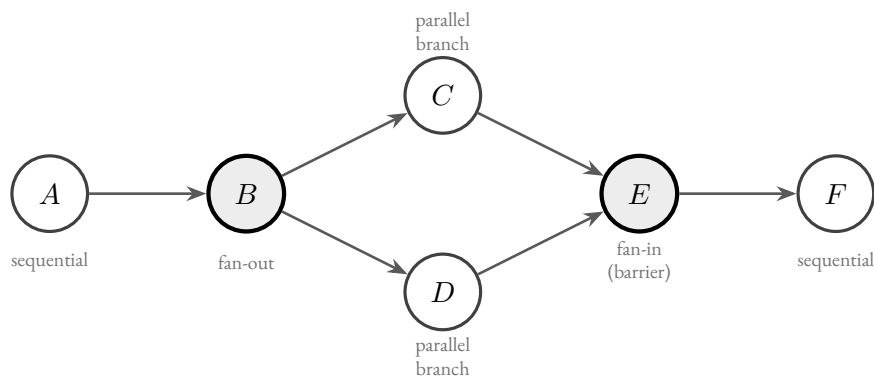


Figure 2.1: A simple six-task DAG workflow. Task B is a *fan-out* node: it spawns tasks C and D as independent parallel branches. Task E is a *fan-in* node (synchronization barrier): it cannot begin until *both* C and D complete. A spot preemption of C or D therefore stalls E and every task downstream of it, even though only one task was actually preempted.

2.4.1 Structural Features That Amplify Failures

Three structural features of a DAG strongly determine how much damage a single spot preemption causes:

- **Fan-out node:** A task whose completion triggers multiple independent parallel branches. A preemption at a fan-out node blocks every downstream branch it spawns—a single failure cascades to an arbitrarily wide set of tasks.

- **Synchronization barrier (fan-in node):** A point where all parallel branches must complete before execution continues. A preemption of *any single* task feeding the barrier stalls every other branch waiting at it, turning one failure into many.
- **Critical path:** The longest source-to-sink dependency chain, with makespan lower-bounded by $L_{cp} = \max_{P \in \mathcal{P}(G)} \sum_{v \in P} d(v)$ where $d(v)$ is task runtime. Disruptions on or near this path have disproportionate impact on end-to-end latency (Topcuoglu et al., 2002).

A preemption on a sequential path forces only that one task to recompute. A preemption at a fan-out node or at a task feeding a fan-in barrier can force every parallel branch to restart from scratch—a *cascade failure*. Figure 2.2 shows this pattern in three representative DAG workflows used in our evaluation: Epigenomics, LIGO, and Montage. In each case, a single preemption at a structurally critical position forces a restart of all parallel branches, not just the preempted one.

2.4.2 Why Structure Enables Selective Protection

Critically, the positions that trigger cascades—fan-out nodes and tasks feeding fan-in barriers—are *topological* properties of the DAG. They do not depend on runtime state; they can be identified once before execution begins and reused across all fault-tolerance decisions. This observation is the structural foundation of FaaSpt: rather than protecting every task uniformly, the system classifies tasks by cascade potential offline and concentrates fault-tolerance effort where it matters most. The primary mechanism for that protection is checkpointing, which we examine next.

2.5 Checkpointing Fundamentals

Checkpointing is the practice of saving a task’s intermediate state to durable storage at designated points during execution. If the task is subsequently interrupted, execution can *resume* from the most recent checkpoint rather than restarting from the beginning of the workflow. This eliminates the need to re-execute all predecessor tasks and is therefore the most cost-effective recovery primitive available when tasks are long-running or sit downstream of expensive parallel stages.

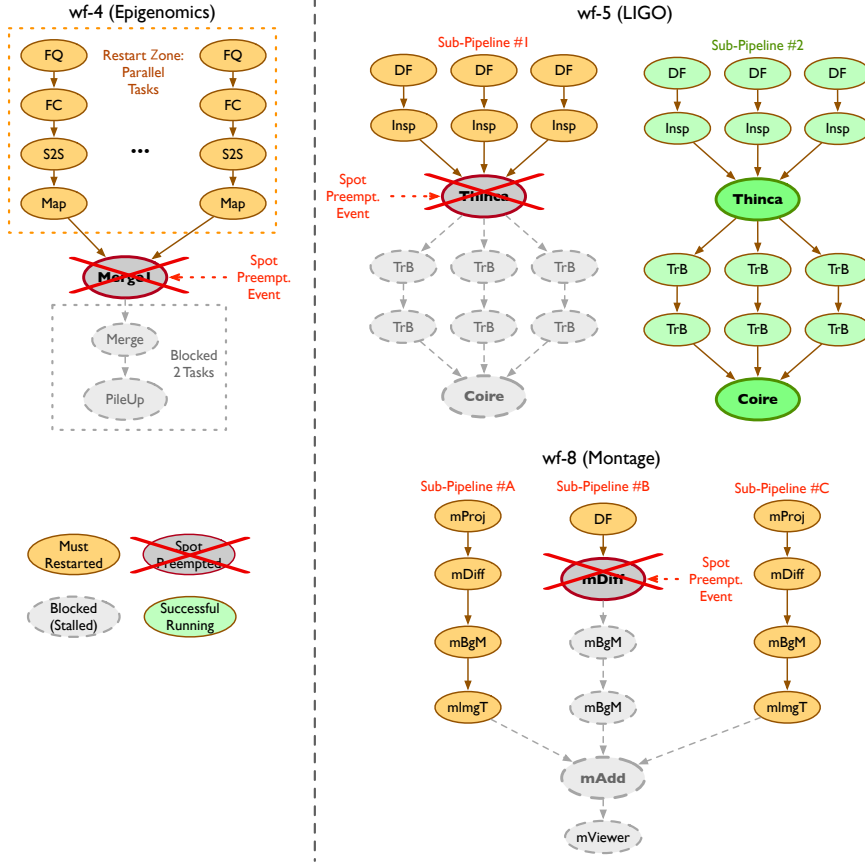


Figure 2.2: Cascade failures in three representative workflows. A spot preemption at a fan-out node or at a task feeding a synchronization barrier (red) forces every parallel branch to restart (orange) and leaves downstream tasks stalled (gray). The structural position of a failure—not its duration—determines its blast radius.

2.5.1 The Overhead-Recomputation Trade-off

The core trade-off in checkpointing is between *checkpoint overhead* (time and I/O spent writing state to persistent storage) and *recomputation cost* (work lost when a preemption occurs between checkpoints). Checkpointing too frequently wastes I/O on tasks that are unlikely to be interrupted; checkpointing too rarely forces expensive re-execution after a failure. Classical theory captures this balance: let C be the cost of writing one checkpoint and M the mean time between failures. Young’s first-order approximation gives the optimal check-

point interval as (Young, 1974):

$$\tau^* \approx \sqrt{2CM}.$$

Daly’s refinement reduces the bias of this estimate under practical conditions (Daly, 2006). Both results convey the same intuition: the higher the checkpoint cost or the lower the failure rate, the less frequently one should checkpoint.

2.5.2 Selective Checkpointing in Workflows

A single global checkpoint interval is suboptimal for DAG workflows because tasks differ in duration, position, and downstream impact. A checkpoint placed at a fan-out node recovers many downstream tasks on failure; a checkpoint at a sequential leaf task recovers only that one task. Uniform checkpointing therefore wastes I/O on low-impact positions while providing no extra benefit over selective placement at high-impact ones (Aupy et al., 2016, Kail et al., 2016). Multi-level hierarchical checkpointing, such as Cho et al. (Cho et al., 2024) for distributed ML workloads on spot clusters, reduces checkpoint overhead by stratifying writes across fast in-memory and slow persistent storage; FaaSpt takes a different approach, stratifying checkpoint decisions by structural DAG position rather than storage tier.

Formally, the checkpoint planner selects a binary vector $x_v \in \{0, 1\}$ to maximize:

$$\max_{x_v} \sum_{v \in V} x_v B(v) - \lambda \sum_{v \in V} x_v c(v),$$

where $B(v)$ estimates the expected recomputation saved by checkpointing task v , $c(v)$ is the checkpoint cost, and λ controls the cost-versus-recovery trade-off. Fan-out nodes and tasks feeding fan-in barriers receive the highest protection; sequential and interior tasks receive progressively less (illustrated in Figure 4.2 in Chapter 4).

FaaSpt operationalizes this idea through a three-tier criticality assignment described in Chapter 4: tasks at fan-out nodes or feeding fan-in barriers are always checkpointed, parallel interior tasks are checkpointed conditionally, and sequential tasks are skipped entirely. Before FaaSpt can make these decisions, however, it needs a way to estimate when a spot instance is likely to be pre-empted. That is where survival analysis enters.

2.6 Survival Analysis and Kaplan-Meier Estimation

Survival analysis is a statistical framework originally developed for clinical trials to estimate the probability that a subject survives beyond a given time. The core quantity is the *survival function* $S(t) = \Pr(T > t)$, where T is the random lifetime of a subject. The same mathematics applies directly to spot instance lifetimes: the “subject” is a running VM, and the “event” is preemption rather than death. Given a set of observed instance lifetimes—some ending in preemption, others still running when the observation window closes (right-censored observations)—survival analysis estimates $S(t)$ from data, without assuming any particular lifetime distribution (Kaplan and Meier, 1958).

2.6.1 The Kaplan-Meier Estimator

The *Kaplan-Meier* (KM) estimator is the standard non-parametric method for constructing $S(t)$ from partially observed data:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right),$$

where d_i is the number of preemptions observed at time t_i and n_i is the number of instances still at risk just before t_i . The product form updates the survival estimate each time a preemption is observed, discounting the population that has already been preempted. The key advantage of KM over parametric alternatives (exponential, Weibull) is that it makes no distributional assumptions—it fits whatever shape the empirical data exhibits. This matters for spot instances because preemption distributions differ across instance types, regions, and time-of-day demand patterns; no single parametric family fits them all (Wu et al., 2024, Son et al., 2024).

2.6.2 Application to Spot Instance Risk

Figure 2.3 shows the KM survival curve estimated from real spot lifetime traces for p3.2xlarge instances in us-west-2a. The curve falls steeply in the first few hours—reflecting elevated early preemption risk—then tails off as surviving instances tend to live longer. This shape motivates a key design choice in FaaSpt: rather than using a static failure rate, the adaptive scaler queries the KM curve to compute the *conditional* probability that an instance survives an

additional window Δt given that it has already been running for time u :

$$P(T > u + \Delta t \mid T > u) = \frac{S(u + \Delta t)}{S(u)}.$$

When this conditional probability drops below a threshold, the scaler proactively provisions additional (safer) instances for the highest-impact tasks—before a preemption occurs, not after. The full decision logic is described in Chapter 4; here, the essential point is that KM gives FaaS_{Spot} a principled, data-driven way to translate observed VM uptime into an actionable risk score.

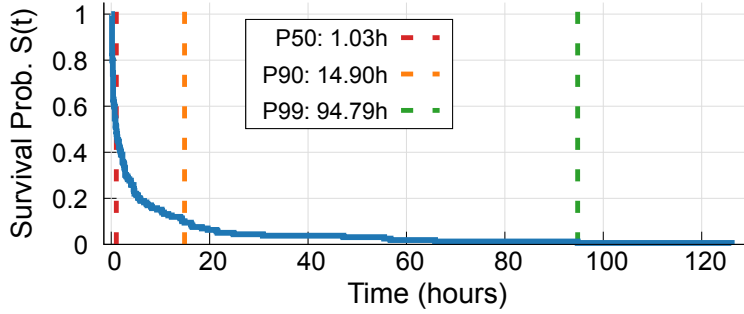


Figure 2.3: Kaplan-Meier survival curve for p3.2xlarge spot instances in us-west-2a estimated from real production traces (Wu et al., 2024). Survival probability falls steeply within the first few hours, then tails off. FaaS_{Spot} uses this curve to classify each running instance into a risk tier and trigger proactive scale-out before preemption occurs.

With the five foundational concepts established, we now turn to the prior literature to understand what existing systems do well and where the gap lies.

2.7 Related Work

FaaS_{Spot} addresses a gap left by two literatures. *Spot fault-tolerance systems* (Hourglass, Snape, MScheduler, Burst-HADS, TR-Spark, Pado) either require on-demand fallback or depend on non-spot resources for critical tasks; none jointly coordinates checkpointing, scaling, and retry in a spot-only setting. *Serverless workflow systems* (Wukong, Speedo, Mashup, DayDream, HyperFlow, Pronghorn) optimize dispatch and resource sizing on stable infrastructure but do not address spot preemption. Chapter 8 provides a full narrative survey. Table 2.1 maps all surveyed systems along six design dimensions; no prior system simultaneously occupies the spot-only + DAG-aware cell that FaaS_{Spot} occupies.

Table 2.1: Comparison of FaaS_{Spot} with related systems. “Spot-only” indicates no on-demand fallback. “DAG-aware” indicates topology-driven checkpoint/retry decisions. Recovery types: FW = full-workflow restart, PR = partial sub-DAG replay, MIG = migration to safe node.

System	Spot-only	DAG-aware	Checkpoint strategy	Recovery	Scaling	Target workload
FaaS _{Spot} (this work)	✓	✓	Structural tier-based	PR	Survival-aware	Serverless DAG
Hourglass	×	×	Uniform timed	FW	Reactive	Graph processing
Snape	×	×	RL-adjusted	FW	Reactive	Mixed workloads
MScheduler	×	×	App-native	FW	Reactive	HPC batch
Burst-HADS	×	×	None	FW	Reactive	Bag-of-tasks
TR-Spark	×	✓	Stage-level lineage	PR	Static	Spark analytics
Pado	×	✓	Task-level	MIG	Static	Data processing
SpotVerse	×	~	None	FW	Regional	Bioinformatics
Flint	✓	×	Task-level	FW	Reactive	Batch analytics
Portfolio	✓	×	None	FW	Portfolio	Batch
Tributary	×	×	None	FW	SLO-driven	Mixed
Bamboo	✓	×	Epoch (ML)	PR	Reactive	ML training
Parcae	×	×	Epoch (ML)	PR	Proactive	ML training
Wukong	×	×	None	N/A	Static	Serverless parallel
Mashup	×	×	None	N/A	Static	HPC serverless
DayDream	×	×	Hot-start	FW	Static	Dynamic serverless
Pronghorn	×	×	Hot-start	N/A	Static	Serverless FaaS

CHAPTER 3

MOTIVATION AND FAILURE CHARACTERIZATION

Chapter 2 established the foundational concepts—serverless computing, spot instances, DAG workflows, checkpointing, and survival analysis—needed to understand why spot-based workflow execution is difficult. This chapter makes the case for FaaS_{Spot} by quantifying exactly how bad the problem is and why existing approaches fall short. We begin by measuring the failure behavior of naive spot deployments across a suite of eight DAG workflows (Section 3.1). We then analyze the structural root cause—why complex workflows fail far more often than simple ones (Section 3.2). Finally, we state the design opportunity and requirements that FaaS_{Spot} must satisfy (Section 3.3).

3.1 Naive Serverless Deployment on Spot: Measurement

To establish a baseline, we deployed eight DAG workflows directly on spot VMs using the serverless platform’s default configuration—no checkpointing, no retry coordination, and no migration to on-demand. We call this *naive serverless deployment*. A preemption anywhere in the workflow fails the entire run and forces a restart from the first task. The measurements use a trace-driven spot emulator (described in Chapter 5) that replays real SkyPilot lifetime traces (Wu et al., 2024) for p3.2xlarge spot instances. Cost is computed using SpotLake (Lee et al., 2022) m3.2xlarge spot pricing. Each workflow is run 500 times under matched trace conditions, allowing direct paired comparison against on-demand execution.

Figure 3.1 reports the three key measurements across the eight-workflow suite.

3.1.1 Success Rate

The naive deployment completes only 39% of runs on average, while on-demand completes every run. On the more complex workflows (wf-4 through wf-8), success rates fall well below 50%, reaching as low as $\sim 13\%$ on wf-8 (Montage). Simple workflows with fewer parallel stages are more tolerant—wf-1 and wf-2 achieve 61%–62%—but even these fall far short of on-demand reliability.

3.1.2 Cost Efficiency

Despite the failures, spot remains cheaper per successful run on every workflow: the reduced per-VM price more than compensates for the cost of occasional restarts. However, the margin is far smaller than the nominal $2\times$ – $10\times$ spot discount. On the simplest workflow (wf-1), the gap is 43%; on the most complex (wf-8), it shrinks to just 3%. Repeated restarts consume compute time and erode spot’s price advantage. A fault-tolerant system that reduces restarts can recover this lost margin.

3.1.3 End-to-End Latency

Latency on spot exceeds on-demand by up to $2.7\times$ because every preemption forces a restart from the workflow’s first task, discarding all intermediate progress. Even workflows with moderate failure rates show substantially inflated latency because partial progress is unrecoverable. This restart-from-origin behavior is the direct consequence of the absence of checkpointing and lineage tracking.

3.1.4 Summary of the Measurement

The three measurements reveal a consistent pattern: spot’s price advantage is real but fragile. It survives even under high preemption rates only because the per-VM discount is large. But repeated full-workflow restarts erode the cost margin, inflate latency, and deliver unreliable completion—making naive spot unsuitable for serious workloads. The question is not whether to add fault tolerance, but *how* to add it without introducing so much overhead that it consumes the savings.

3.2 Structural Root Cause: Why Complex Workflows Fail More

The stark difference between simple and complex workflows in Figure 3.1(a) cannot be explained by task count or execution time alone. The primary cause is *dependency structure*: fan-out nodes and synchronization barriers cause a single preemption to propagate across many tasks. Under naive deployment, wf-4, wf-5, and wf-8 all start near 100% success at stage 1 and degrade monotonically—wf-5 falls to 37% within three stages, wf-8 to 42%—because each additional parallel stage multiplies preemption exposure. A preemption at a fan-out node blocks every branch it spawned; a preemption feeding a synchronization barrier stalls all branches waiting at the barrier, even branches that have already completed. wf-5’s dual sub-pipeline barrier and wf-8’s three nested fan-in stages produce the widest blast radii in the suite. The pattern is not application-specific: all three workflows come from different scientific domains yet exhibit the same structural failure amplification.

The key design consequence is that cascade-prone positions are determined *entirely* by the DAG topology—they do not change at runtime. They can be identified *once*, before execution begins, and reused across all fault-tolerance decisions. Rather than reacting uniformly to all preemptions, a system that knows which positions trigger cascades beforehand can concentrate protection precisely where it matters.

3.3 The Design Opportunity

The measurements establish a clear gap and a clear opportunity. Spot instances remain cost-effective even under preemption, but naive deployment does not come close to extracting spot’s full price advantage. A fault-tolerant system that eliminates unnecessary full-workflow restarts could:

- **Recover the cost margin.** By resuming from checkpoints rather than restarting from origin, each preemption wastes far less compute time, reducing cost-per-successful-run toward the on-demand baseline.
- **Reduce latency.** Checkpoint-based recovery replays only the sub-DAG downstream of the recovery point. For a workflow with expensive early parallel stages, this can reduce latency by orders of magnitude compared to a full restart.

- **Improve success rates.** Proactive scaling onto safer instances before a preemption occurs prevents failures at high-impact positions before they cascade.

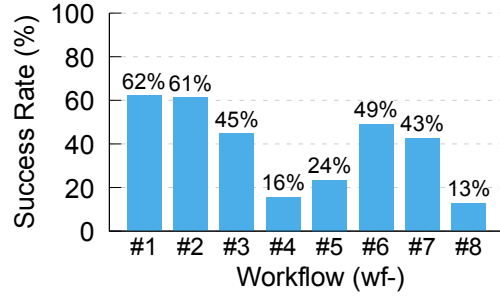
3.3.1 Requirements for a Pure-Spot Solution

Achieving these goals entirely on spot—without any on-demand fallback—imposes specific requirements:

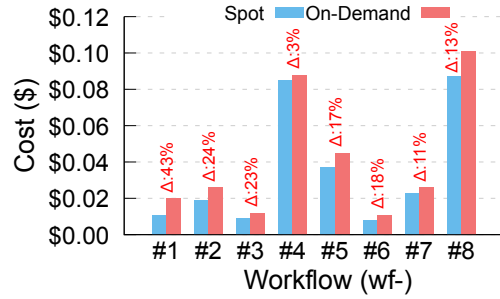
1. **Offline topology analysis.** The system must identify cascade-prone positions before execution begins, from the DAG structure alone, so that fault-tolerance decisions are consistent and require no runtime coordination overhead.
2. **Selective checkpointing.** Checkpoint I/O should be concentrated at high-impact structural positions (fan-out nodes, tasks feeding fan-in barriers) and skipped at low-impact ones (sequential tasks). Uniform checkpointing wastes I/O budget and still leaves cascade positions unprotected relative to their blast radius.
3. **Survival-aware scaling.** When a running instance’s preemption risk rises—as indicated by its KM-estimated survival probability—the system should proactively move the highest-impact tasks to safer instances before the preemption occurs.
4. **Lineage-based retry.** When a preemption does occur, recovery should resume from the nearest upstream checkpoint, replaying only the downstream sub-DAG, rather than restarting the entire workflow.

These four requirements map directly to FaaS_{Spot}’s four components: offline DAG analysis, the checkpoint tier planner, the adaptive scaler, and the retry manager. Each requirement is grounded in the failure characterization above: requirement 1 is justified by the structural analysis (Section 3.2), requirements 2 and 4 by the cost and latency measurements (Section 3.1), and requirement 3 by the survival analysis in Chapter 2.

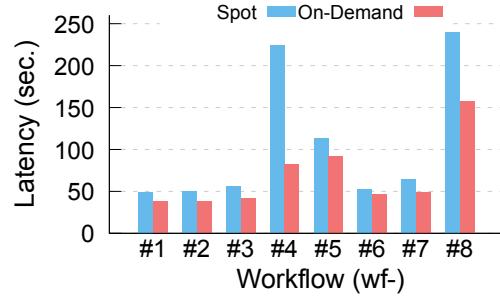
Table 3.1 maps representative prior systems against the four requirements. No existing system satisfies all four; FaaS_{Spot} is the first to do so.



(a) Success rate



(b) Cost per successful run



(c) Average end-to-end latency

Figure 3.1: Success rate, cost, and latency of naive serverless deployment on spot vs. on-demand across eight workflows (wf-1 through wf-8). Each workflow is run 500 times on the trace-driven emulator using real p3.2xlarge spot lifetime traces. Spot runs have no fault-tolerance mechanism; a preemption anywhere fails the entire run. (a) Success rate falls well below 50% on complex workflows, dropping to $\sim 13\%$ on the most complex. (b) Cost per successful run remains below on-demand on every workflow, but the margin is modest (3%–43%), far below the nominal $2\times$ – $10\times$ spot discount. (c) End-to-end latency on spot exceeds on-demand by up to $2.7\times$ because every failure forces a full restart.

Table 3.1: Coverage of the four pure-spot fault-tolerance requirements by representative prior systems and FaaSpot. A ✓ indicates the requirement is substantially addressed; a dash (—) indicates it is not. Hourglass is assigned — for requirement (3) because it uses deadline-slack pacing rather than KM-based statistical survival estimation; see Section 2.7 for discussion.

System	(1) Offline Topology	(2) Selective Ckpt.	(3) Survival Scaling	(4) Lineage Retry
Aupy et al. (Aupy et al., 2016)	—	✓	—	—
TR-Spark (Yan et al., 2016)	✓	✓	—	—
Pado (Y. Yang et al., 2017)	✓	—	—	—
Snape (F. Yang et al., 2023)	—	—	—	—
Hourglass (Joaquim et al., 2019)	—	—	—	—
MScheduler (Portella et al., 2023)	—	—	—	✓
Burst-HADS (Teylo et al., 2023)	—	—	—	—
SpotVerse (Son et al., 2024)	—	—	—	—
FaaSpot	✓	✓	✓	✓

The gap identified here—a serverless workflow system that satisfies all four requirements entirely on spot—is precisely the contribution of this thesis. Chapter 4 describes how FaaS_{Spot} addresses each requirement through its offline DAG analysis, checkpoint tier planner, adaptive scaler, and retry manager.

CHAPTER 4

FAASpot SYSTEM DESIGN

This chapter presents the FaaS_{Spot} architecture and its five components. Notation is consistent throughout: $G = (V, E)$ is the workflow DAG; $w(v)$ is the estimated execution time of task v ; $\rho(v)$ is the number of parallel task replicas required by v (1 for a sequential task, N for a task that runs as N parallel branches); $\kappa(v) \in [0, 1]$ is the per-task criticality score; $\ell(v)$ is the checkpoint tier; σ_c and σ_t are the current and target success rates.

4.1 Architecture Overview

FaaS_{Spot} executes a serverless DAG workflow entirely on spot instances through two tightly coupled phases (Figure 4.1).

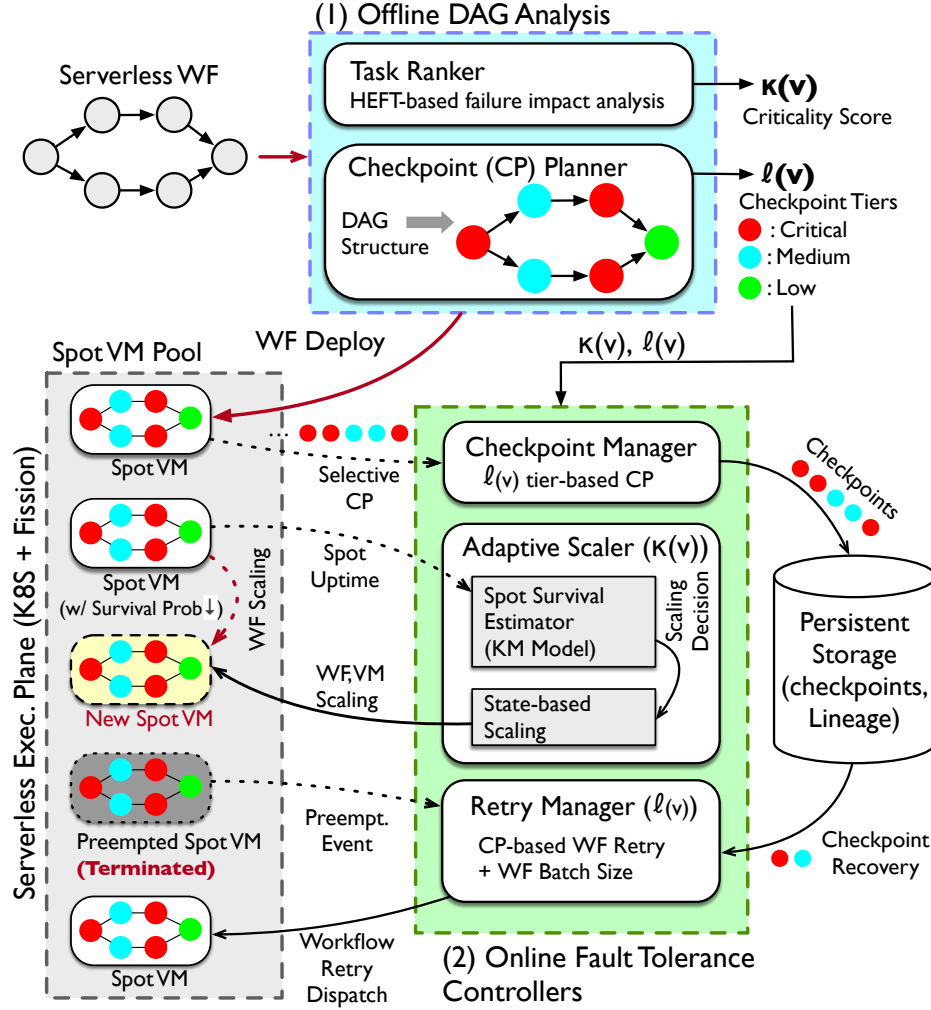


Figure 4.1: FaaSSpot system architecture. The offline phase derives $\kappa(v)$ and $\ell(v)$ from the workflow DAG before execution begins. Three online fault-tolerance controllers consume these signals at runtime. All runtime state is persisted to a shared storage layer; the execution plane runs task pods on a shared spot VM pool.

The **offline phase** runs once at workflow submission and produces two per-task annotations: the criticality score $\kappa(v)$ from the *Task Ranker*, and the checkpoint tier $\ell(v) \in \{\text{CRITICAL}, \text{MEDIUM}, \text{LOW}\}$ from the *Checkpoint Planner*. Both are distributed to all online controllers before any task executes.

The **online phase** runs three independent controllers. The *Checkpoint Manager* writes state snapshots at tier-selected positions. The *Adaptive Scaler* proactively provisions additional replicas when survival probability drops. The *Retry Manager* resumes preempted runs from their nearest checkpoint. All

three read the same static $(\kappa(v), \ell(v))$ signals—no runtime coordination is required.

Control-plane assumptions. Throughout this thesis, “pure-spot” refers to the *task execution plane*—all task pods run on spot virtual machines with no on-demand fallback. The *control plane* (MySQL state store, Fission router, scheduler pods, and the three runtime controllers themselves) is assumed to run on a small, non-preemptible footprint, consistent with the convention adopted by Bamboo (Thorpe et al., 2023), Parcae (Duan et al., 2024), and SpotVerse (Son et al., 2024). This footprint is small (single-digit pods regardless of workflow scale), so the cost figures in Chapter 6 are essentially unchanged whether the control plane runs on on-demand or reserved capacity. Chapter 7 §7.1 discusses the implications of running the control plane itself on spot, which is left to future work.

4.1.1 Three Design Principles

Offline topology analysis. Fan-out nodes and pre-fan-in tasks are topological properties of the DAG. Identifying them once before execution produces a stable classification that all three online controllers reuse without overhead.

Criticality-annotated DAG. Every fault-tolerance decision is anchored to the same $(\kappa(v), \ell(v))$ pair computed offline. No controller maintains an independent classification, so the three controllers are consistent by construction.

Tiered fault tolerance. Uniform checkpoint policies waste I/O at sequential tasks whose re-execution cost is bounded by a single task runtime. FaaS_{Spot} concentrates checkpoint I/O at cascade-prone positions and skips it everywhere else.

4.2 Task Ranker

4.2.1 Algorithm

Algorithm 1 Task Ranker: HEFT Rank and Criticality Scoring

Require: DAG $G = (V, E)$, execution times $w(v)$, parallel replicas $\rho(v)$, budget B

Ensure: Criticality scores $\kappa(v)$, per-task scaling factors $f(v)$

```

1: for each  $v \in V$  in reverse topological order do  $\triangleright R_2 - \text{HEFT rank}$ 
2:    $\text{rank}(v) \leftarrow w(v) + \max_{u \in \text{succ}(v)} \text{rank}(u)$ 
3: end for
4: for each  $v \in V$  do  $\triangleright R_3 - \text{fan-degree augmentation}$ 
5:    $\text{score}(v) \leftarrow \text{rank}(v) + \text{fanOutDegree}(v) + \text{fanInPressure}(v)$ 
6: end for
7:  $\kappa(v) \leftarrow (\text{score}(v) - \min_V) / (\max_V - \min_V)$  for all  $v$   $\triangleright R_4$ 
8:  $f_{\max} \leftarrow \min(B / \sum_v \text{cost}(v), f_{\text{cap}})$ 
9: for each task  $v$  do
10:   $f(v) \leftarrow 1 + (f_{\max} - 1) \cdot \kappa(v) / \sum_{v'} \kappa(v')$ 
11: end for
12: return  $\kappa, f$ 

```

4.2.2 Extended Discussion

Why HEFT? HEFT’s upward rank captures the longest remaining execution path, directly quantifying wall-clock time at risk if a task is preempted. A purely depth-based metric misses long sequential suffixes; a purely time-based metric misses structurally critical positions with short individual runtimes. The fan-degree augmentation inflates $\kappa(v)$ for nodes whose preemption stalls multiple parallel branches simultaneously—the cascade-prone positions from Chapter 3. Budget-proportional scaling ensures that within budget B , the highest- κ positions are protected first; $f_{\text{cap}} = 2.5 \times$ caps per-task offline amplification to preserve coverage across all high- κ positions.

Score combination is a heuristic, not a derivation. The expression $\text{score}(v) = \text{rank}(v) + \text{fanOutDegree}(v) + \text{fanInPressure}(v)$ adds quantities with different units ($\text{rank}(v)$ is in seconds; the fan-degree terms are unitless counts). The subsequent min–max normalization on line 4 of Algorithm 1 maps the combined score onto $[0, 1]$ and renders the result scale-invariant, but it does not give the additive combination a first-principles justification: the relative weighting of “time-at-risk” against “structural blast radius” is empirical.

The justification is operational. First, the downstream consumers of $\kappa(v)$ —the Checkpoint Planner (tier assignment) and the Adaptive Scaler (κ -ordered dispatch)—use only the *rank order* that $\kappa(v)$ induces, not its absolute value, so the additive weighting only needs to produce a sensible ordering. Second, the ablation study in §6.7 weakens each mechanism while keeping κ fixed, isolating its contribution from the rank formulation. Third, structural-only and time-only variants of κ were piloted on wf-1 and wf-2 during development and produced inferior or equivalent orderings on the structurally diverse evaluation suite. We therefore present the score as a deliberately simple heuristic that performs well in practice; a principled multi-objective formulation that explicitly trades cascade radius against critical-path time is left to future work.

4.3 Checkpoint System: Planner and Manager

The checkpoint system spans both phases of FaaS_{Spot}. The *Checkpoint Planner* runs offline and decides which tasks should be checkpointed; the *Checkpoint Manager* runs online and executes that policy at task completion. The Planner produces a static tier assignment $\ell(v)$ before execution begins, and the Manager acts on it with no further communication to the Planner.

4.3.1 Checkpoint Tiers

The three-tier classification assigns each task a protection level based on its structural position in the DAG. Figure 4.2 illustrates the classification on a concrete example.

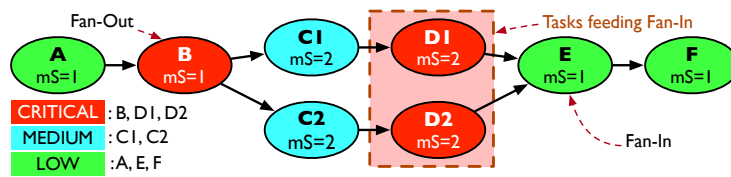


Figure 4.2: Three-tier checkpoint classification. Task B is the fan-out node: its single-replica output spawns parallel branches, so the replica count ρ changes across each outgoing edge— B is **CRITICAL**. The parallel branch tasks (C , D) run with $\rho > 1$ and no outgoing replica change—they are **MEDIUM**. Tasks A , E , F are single-replica sequential tasks with no replica transition—they are **LOW**. A **CRITICAL** checkpoint at B means any branch failure recovers from B ’s output rather than restarting from A .

- **CRITICAL:** sits at a parallelism boundary (ρ differs from at least one successor). Preemption triggers a cascade across all downstream parallel tasks. **Always checkpointed; highest retry priority.**
- **MEDIUM:** runs as multiple parallel replicas but is not at a boundary. Preemption forces all $\rho(v)$ replicas to re-execute. **Always checkpointed; retried when the success gap warrants it.**
- **Low:** single-replica sequential task. Preemption forces only that task to re-execute. **Not checkpointed; no checkpoint-based retry.**

4.3.2 Algorithm

Algorithm 2 Checkpoint Planner: Tier Classification

Require: Tasks V with $\text{succ}(v)$ and $\rho(v)$ for each $v \in V$

Ensure: Tiers $\ell(v) \in \{\text{CRITICAL}, \text{MEDIUM}, \text{LOW}\}$

```

1: for all  $v \in V$  do
2:    $\ell(v) \leftarrow \perp$ 
3:   for each  $s \in \text{succ}(v)$  do                                      $\triangleright P_2 - \text{boundary check}$ 
4:     if  $\rho(s) \neq \rho(v)$  then
5:        $\ell(v) \leftarrow \text{CRITICAL}$ 
6:       break
7:     end if
8:   end for
9:   if  $\ell(v) = \perp$  then                                            $\triangleright P_3 - \text{interior or sequential}$ 
10:     $\ell(v) \leftarrow \text{MEDIUM}$  if  $\rho(v) > 1$  else LOW
11:  end if
12: end for
13: Record  $\ell(v)$  for each  $v$                                         $\triangleright P_4$ 
14: return  $\ell$ 

```

4.3.3 Extended Discussion

Why replica transitions are the right signal. A parallelism boundary is the exact structural position where a single preemption stalls every downstream parallel task. Detecting it via replica-count transitions requires no execution-time profiling and runs in $O(|V| + |E|)$. Skipping **Low**-tier tasks focuses I/O budget on positions where a checkpoint actually saves meaningful downstream work—FaaSSpot writes 10.6 checkpoints per successful run versus 39.9

for the uniform-checkpoint baseline (Chapter 6), confirming that placement specificity dominates frequency.

4.4 Adaptive Scaler

4.4.1 Role

The Adaptive Scaler monitors spot instance survival probability and proactively provisions additional parallel replicas before an instance is preempted. It operates as a *two-layer* system: a **KM survival layer** that detects threat level and sets permissible scale-factor bounds, and a **contextual bandit layer** that learns, from experience, which action within those bounds best improves success rate per unit of cost. The KM layer ensures timely response to survival risk; the bandit layer ensures each response is efficient and context-appropriate.

4.4.2 Sub-Components

The KM Survival Estimator computes the conditional survival probability $S(\Delta t | u_i) = \hat{S}(u_i + \Delta t) / \hat{S}(u_i)$ for each running instance and maps it to one of four tiers (Table 4.1); the SAFE tier suppresses scaling, the others activate the bandit layer.

Contextual Bandit Layer

S4 Context Builder Assembles a 6-dimensional context vector $\mathbf{x}$ from real-time execution signals:

- σ_c — current workflow success rate,
- $\Delta\sigma$ — change in success rate over a recent observation window,
- retry exhaustion ratio — fraction of runs that have exhausted their retry limits,
- normalized cost — total cost spent relative to the spot budget,
- elapsed time fraction — how far along the workflow experiment is,
- normalized replica count — current total replicas relative to the expected maximum.

Together these features capture both the current execution health and the cost state at the moment of the scaling decision.

S5 Thompson Sampling Selector Maintains a separate Bayesian linear regression model for each of four discrete scaling actions $\mathcal{A}$:

$$\mathcal{A} = \{ \text{idle}(1.0\times), \text{conservative}(1.1\times), \text{moderate}(1.3\times), \text{aggressive}(1.5\times) \}$$

At each decision tick, it selects an action from the subset $\mathcal{A}(u_i) \subseteq \mathcal{A}$ permitted by the KM tier bounds and the remaining budget via Thompson Sampling:

1. For each action $a \in \mathcal{A}(u_i)$, sample a weight vector from the posterior: $\tilde{\mathbf{w}}_a \sim \mathcal{N}(\boldsymbol{\mu}_a, \sigma^2 \mathbf{B}_a^{-1})$.
2. Compute the sampled expected reward $\hat{r}_a = \mathbf{x}^\top \tilde{\mathbf{w}}_a$.
3. Select $a^* = \arg \max_{a \in \mathcal{A}(u_i)} \hat{r}_a$.

After an observation window, the posterior is updated with the observed reward:

$$\mathbf{B}_{a^*} \leftarrow \mathbf{B}_{a^*} + \mathbf{x}\mathbf{x}^\top, \quad \mathbf{b}_{a^*} \leftarrow \mathbf{b}_{a^*} + r \mathbf{x}, \quad \boldsymbol{\mu}_{a^*} = \mathbf{B}_{a^*}^{-1} \mathbf{b}_{a^*},$$

where the reward $r = \Delta\sigma/\text{cost}$ is the cost-normalized success gain, penalized if the budget is exhausted. Actions with consistently higher cost-normalized success gain accumulate higher posterior means and are selected more often over time.

S6 κ -Ordered Dispatcher Sorts the tasks on the at-risk instance by $\kappa(v)$ descending and dispatches $\lceil f_{a^*} \cdot \rho(v) \rceil$ additional replicas per task to safer instances. Sorting by $\kappa(v)$ ensures that a partial scale-out under capacity limits still covers the most cascade-prone positions first.

4.4.3 KM–Bandit Integration

The two layers are complementary by design. The KM layer answers: *is this instance likely to be preempted soon?* It constrains the bandit to a safe range $[f_{\min}, f_{\max}]$ appropriate to the measured threat. The bandit layer then answers: *given the current workflow execution context, which specific factor within that range maximizes success rate per dollar?*

This division avoids two failure modes: a pure KM system would scale aggressively whenever survival probability drops, even when the workflow is already recovering through retries (wasting cost); a pure bandit system without survival constraints might under-react to imminent preemption before accumulating enough data (missing the failure). KM enforces a safety floor; the bandit optimizes within it.

During a *cold-start phase*, before sufficient data is collected to trust the posterior, the bandit uses ε -greedy exploration ($\varepsilon = 0.5$) restricted to the $\{\textit{idle}, \textit{conservative}\}$ action pair to prevent aggressive early spending. The cold-start phase persists for the first $T_{\text{cold}} = 10$ decision ticks or until at least three reward observations have been recorded per action—whichever is later. Each workflow experiment initializes the bandit from a flat prior ($\mathbf{B}_a = \lambda_{\text{reg}} \mathbf{I}$, $\mathbf{b}_a = \mathbf{0}$, regularization $\lambda_{\text{reg}} = 1.0$); no cross-workflow posterior transfer is performed, so the posterior reflects only within-experiment observations. Budget gates further restrict the allowed action set: below 10% remaining budget only idle is permitted; below 20% only idle or conservative; below 35% the aggressive action is blocked entirely.

Table 4.1: Survival states and scale-factor bounds passed to the bandit. The bandit selects within the range; the SAFE state suppresses the bandit entirely.

State	Survival $S(t)$	Bandit Action Range	Effect
SAFE	$S(t) > 0.55$	1.0 $\times$ only	No action
PROACTIVE	$0.50 < S(t) \leq 0.55$	$[1.1, 1.5] \times$	Light scale-out
WARNING	$0.40 < S(t) \leq 0.50$	$[1.3, 1.8] \times$	Moderate scale-out
CRITICAL	$S(t) \leq 0.40$	$[1.5, 2.0] \times$	Aggressive scale-out

4.4.4 Algorithm

Algorithm 3 Adaptive Scaler: Two-Layer Decision Loop

Require: Instances I , uptimes u_i , task metadata $\kappa(v)$, $\rho(v)$, KM curve $\hat{S}$

Ensure: Scale-out replicas dispatched; bandit posterior updated

- 1: Initialise bandit with prior $\mathbf{B}_a \leftarrow \lambda \mathbf{I}$, $\mathbf{b}_a \leftarrow \mathbf{0}$ for each action a
- 2: **repeat**
- 3: **for** each instance $i \in I$ **do**
- 4: $S \leftarrow \hat{S}(u_i + \Delta t) / \hat{S}(u_i)$ $\triangleright S_2$ – conditional survival
- 5: $\text{tier} \leftarrow \text{classify}(S)$ $\triangleright S_3$ – Table 4.1
- 6: **if** $\text{tier} = \text{SAFE}$ **then**
- 7: **continue**
- 8: **end if**
- 9: $[f_{\min}, f_{\max}] \leftarrow \text{tierBounds}(\text{tier})$
- 10: $\mathcal{A}(u_i) \leftarrow \{a \in \mathcal{A} : f_a \in [f_{\min}, f_{\max}]\} \cap \mathcal{A}_{\text{budget}}$
- 11: $\mathbf{x} \leftarrow \text{buildContext}(\sigma_c, \Delta\sigma, \dots)$ $\triangleright S_4$
- 12: $\tilde{\mathbf{w}}_a \sim \mathcal{N}(\boldsymbol{\mu}_a, \sigma^2 \mathbf{B}_a^{-1})$ for each $a \in \mathcal{A}(u_i)$ $\triangleright S_5$
- 13: $a^* \leftarrow \arg \max_{a \in \mathcal{A}(u_i)} \mathbf{x}^\top \tilde{\mathbf{w}}_a$
- 14: **for** task v on i , sorted by $\kappa(v)$ descending **do** $\triangleright S_6$
- 15: Deploy $\lceil f_{a^*} \cdot \rho(v) \rceil$ replicas on safer instances
- 16: **end for**
- 17: **end for**
- 18: Wait Δ_{obs} seconds; observe $(\sigma_{\text{after}}, \text{cost})$
- 19: $r \leftarrow (\sigma_{\text{after}} - \sigma_{\text{before}}) / \text{cost}$ $\triangleright$ cost-weighted reward
- 20: $\mathbf{B}_{a^*} \leftarrow \mathbf{B}_{a^*} + \mathbf{x}\mathbf{x}^\top$; $\mathbf{b}_{a^*} \leftarrow \mathbf{b}_{a^*} + r \mathbf{x}$ $\triangleright$ posterior update
- 21: **until** workflow complete

4.4.5 Extended Discussion

Why conditional survival. Using $S(\Delta t \mid u_i)$ rather than raw lifetime captures the aging effect: long-running instances face lower conditional hazard than freshly launched ones. The four thresholds in Table 4.1 were set by examining the empirical KM curve; the sensitivity sweep in Chapter 6 confirms stability across the 0.60–0.80 range.

Why Thompson Sampling rather than alternatives. Three properties of the operating regime favor TS over its closest competitors. (i) *Sub-linear regret with closed-form Bayesian updates.* The Bayesian update—adding $\mathbf{x}\mathbf{x}^\top$ to the precision matrix—runs in $O(d^2)$ per step ($d = 6$). The same property holds for LinUCB, but TS’s randomized posterior sampling provides smoother

exploration–exploitation interpolation than LinUCB’s deterministic upper-confidence-bound rule, which is sensitive to the confidence-radius hyperparameter α when the prior is weak. *(ii) Automatic exploration annealing.* TS naturally anneals exploration as the posterior tightens, avoiding the fixed over- or under-exploration of ε -greedy with a constant ε . We confirmed this on wf-1 and wf-2 during development: ε -greedy at $\varepsilon = 0.5$ over-explored aggressive actions late in the run, while $\varepsilon = 0.1$ under-explored early; LinUCB at $\alpha = 1$ behaved comparably to TS but required tuning α per workflow to match. *(iii) Per-action models are independent.* Because each action has its own precision matrix, posterior updates are decoupled and can be persisted independently across runs without conflicting cross-action information sharing. The trade-off is slower convergence per action; we accept this because the KM tier layer constrains the action set to 2–3 actions in the regime where the bandit is active.

Hyperparameter provenance. The cost-weighted reward $\Delta\sigma/\text{cost}$ aligns the bandit’s objective with the system goal: maximize success-rate gain per dollar. The cold-start parameters ($\varepsilon = 0.5$ on the safe action subset, $T_{\text{cold}} = 10$ ticks for complex DAGs, 30-real-minute passive period for simple DAGs, 120-second aggressive-action cooldown, regularization $\lambda_{\text{reg}} = 1.0$, reward clip $[-1, 1]$) are not derived from first principles. They were tuned during development on wf-1 and wf-2 only and then held fixed for all reported evaluations. The KM-tier survival thresholds (0.55 / 0.50 / 0.40) were chosen by inspection of the SkyPilot CDF at points where the empirical hazard ratio changed sharply; Table 6.9 in Chapter 6 sweeps the dominant threshold to show robustness across a 0.60–0.80 range. The complete hyperparameter list is reproduced in Appendix A Table A.7 for reproducibility.

Reward delay and credit assignment. The bandit treats $r_t = \Delta\sigma_t/\text{cost}_t$ as the reward for the action selected at the start of the observation window. In practice, the success-rate change $\Delta\sigma_t$ reflects the cumulative effect of all actions taken within the window plus correlated preemption events that may begin before the window opens, so the bandit is in effect a non-stationary delayed-reward contextual bandit treated as a stationary instantaneous one. We accept this approximation because (a) clipping at $[-1, 1]$ limits the influence of any single mis-attributed reward, (b) the KM-tier safety floor prevents the bandit from over-spending even when the posterior is mis-attributed, and (c) the inner-loop period (Algorithm 3 line “Wait Δ_{obs} ”) is tuned so that most observation windows contain at most one preemption event. The principled treatment—an off-policy delayed-reward bandit such as the one in (Duan et al., 2024)—is left to future work, and Chapter 7 §7.2 discusses the threat this approximation poses to internal validity. Independent of the action selected, the κ -ordered

dispatcher (S6) ensures that any partial scale-out under capacity limits covers the highest-impact tasks first.

4.5 Retry Manager

4.5.1 Role

The Retry Manager is the primary contributor to FaaS_{Spot}'s success rate improvement. When a preemption occurs, it detects the failure, traverses the DAG backward to find the nearest checkpointed task upstream of the failed one, and resumes execution from there—replaying only the downstream sub-DAG. It also manages how many failed runs to retry per cycle, adapting batch size based on how far σ_c falls below σ_t .

4.5.2 Algorithm

Algorithm 4 Retry Manager: Lineage-based Retry

Require: Workflow runs U , retry history $R[\cdot]$, success rates (σ_t, σ_c) , tiers ℓ

Ensure: Batch of runs resumed from nearest checkpoints

```

1:  $\delta \leftarrow \max(0, (\sigma_t - \sigma_c) / \sigma_t)$ 
2:  $C \leftarrow \{u \in U : \text{incomplete}, R[u] < L_{\max}[u], \text{ not abandoned}\} \triangleright RT_2$ 
3: for each  $u \in C$  do
4:    $p(u) \leftarrow \pi_w + \lfloor \delta \cdot \lambda_d \rfloor + n_c \cdot w_c - r \cdot w_r \triangleright RT_4$ 
5: end for
6: Sort  $C$  descending by  $p(u)$ 
7: if  $\delta > \delta_{\text{high}}$  then
8:    $B \leftarrow B_{\text{high}}$ 
9: else if  $\delta > 0$  then
10:   $B \leftarrow B_{\text{mid}}$ 
11: else
12:   $B \leftarrow B_{\text{low}}$ 
13: end if  $\triangleright RT_5$ 
14: if scaler has not improved  $\sigma_c$  for  $\geq 2$  cycles then
15:   $B \leftarrow \lfloor B \times \alpha \rfloor$ 
16: end if
17: for top  $B$  runs from sorted  $C$  do  $\triangleright RT_3 + RT_6$ 
18:    $start \leftarrow \text{backoffChain}(u, \ell)$ 
19:   Resume at  $start$ ; replay sub-DAG downstream
20:    $R[u] \leftarrow R[u] + 1$ ; update state machine
21: end for

```

Table 4.2: Retry batch-sizing policy. α is the multiplier applied when scaling stalls.

Condition	Batch Size	Effect
$\delta > \delta_{\text{high}}$	B_{high}	Aggressive recovery
$0 < \delta \leq \delta_{\text{high}}$	B_{mid}	Moderate recovery
$\delta = 0$	B_{low}	Maintenance only
Scaler ineffective ≥ 2 cycles	$B \times \alpha$	Shift to retrying

4.5.3 Extended Discussion

Why lineage-based recovery is dominant. Removing it (Variant C, Chapter 6) drops success rate to 50% and raises cost per run by $4.6\times$ —the largest single-mechanism degradation. The backoff chain walks backward in reverse topological order until it finds a completed predecessor with a recorded checkpoint; if none exists, the workflow restarts from its source task. The priority score’s completion term $n_c \cdot w_c$ biases the queue toward near-complete runs; the penalty term $-r \cdot w_r$ redirects resources from repeatedly-failing candidates.

CHAPTER 5

IMPLEMENTATION

This chapter describes how the five FaaS_{Spot} components and the spot preemption emulator are realized in software. The system is implemented in Python on a Kubernetes/Fission stack. Chapter 4 defined the algorithms and their contracts; this chapter traces each algorithm to its concrete realization, covering key data representations, inter-component interfaces, and deployment decisions that are not obvious from the design alone.

5.1 Platform Stack

FaaS_{Spot} is built on three infrastructure layers.

Kubernetes provides the container scheduling substrate. Every workflow task function runs as a Kubernetes *Deployment*; each replica is one pod. Per-task configuration—HEFT rank, scaling factor, and scalability flag—is stored in Kubernetes ConfigMaps and written by the Task Ranker before experiment start. Replica counts are updated at runtime without function redeployment via rolling-restart semantics. Kubernetes Horizontal Pod Autoscaling is disabled throughout all experiments: FaaS_{Spot}'s replica-count decisions are made by the Task Ranker (offline) and the Adaptive Scaler (online), and an active HPA controller responding to CPU utilization would conflict with these decisions.

Fission runs atop Kubernetes as the serverless execution layer. Workflow tasks are registered as Fission functions and invoked via HTTP through the Fission router. Fission manages function lifecycle—pod creation, warm-pool maintenance, and request routing—so that task invocations arrive at an already-running pod rather than incurring a cold-start penalty on every call.

MySQL serves as the shared persistent state layer. All five runtime components read and write the same database, which eliminates the need for inter-process message passing and makes state observable from outside the system.

The principal tables are: workflow invocation log (one row per UID, tracks task completion and terminal status), checkpoint plan (per-task tier assignments read by the Retry Manager at recovery time), cost log (per-pod billing intervals for cost accounting), and retry log (attempt history per UID for retry-limit enforcement).

Database connections are pooled to bound per-query wait time to under 50 ms under 500 concurrent workflows. The polling-based design reflects a deliberate trade-off of latency for observability: failure detection carries a floor equal to the poll interval (5 s for most workflows, 2 s for the two longest). All end-to-end latency figures in Chapter 6 include this overhead.

5.2 Offline Planning Components

5.2.1 Task Ranker

The Task Ranker runs once before an experiment. It computes upward HEFT ranks in reverse topological order (Algorithm 1), groups tasks into virtual machines of 3 pods each, and allocates scale-out proportionally to machine rank within budget B . The per-task scaling factor is capped at $f_{\text{cap}} = 2.5\times$. Scaling factors are written to persistent metadata and patched into Kubernetes ConfigMaps via rolling restart.

5.2.2 Checkpoint Planner

The Checkpoint Planner implements Algorithm 2 directly: for each task v , it assigns CRITICAL if any successor has a different minimum replica count, MEDIUM if $\text{minscale}(v) > 1$, and LOW otherwise. The tier assignment is written to the checkpoint plan table keyed by workflow name, baseline label, and task identifier. String tier labels are mapped to integer priority values (CRITICAL $\rightarrow 3$, MEDIUM $\rightarrow 2$, LOW $\rightarrow 1$) for use by the Retry Manager at recovery time.

5.3 Runtime Control Components

5.3.1 Adaptive Scaler

The Adaptive Scaler implements Algorithm 3 as three cooperating modules: a KM survival control loop (polls every 30 s), a contextual Thompson Sampling bandit (Bayesian Linear Regression per action with closed-form posterior updates), and a budget-gating layer.

The 6-dimensional context vector is: current success rate σ_c , success-rate delta $\Delta\sigma$, retry-exhausted ratio, normalized cost, elapsed time fraction, and normalized pod count. The bandit maintains a precision matrix $\mathbf{B}_a$ and sufficient-statistic vector $\mathbf{b}_a$ per action; after observing reward $r = \Delta\sigma/\text{cost}$ (clipped to $[-1, 1]$), it updates $\mathbf{B}_a \leftarrow \mathbf{B}_a + \mathbf{x}\mathbf{x}^\top$ and $\mathbf{b}_a \leftarrow \mathbf{b}_a + r\mathbf{x}$. Bandit state is persisted to disk after each update.

Cold start: simple workflows (shallow DAGs) use a 30-minute real-time cold start with only *idle* permitted. Under the $60\times$ time compression factor, 30 real minutes corresponds to 30 emulated hours—longer than the p_{90} instance lifetime—so the Scaler is intentionally passive for simple workflows. Complex workflows use a 2-minute cold start with *{idle, conservative, moderate}* permitted. An aggressive-action cooldown of 120 seconds prevents repeated high-cost decisions before sufficient reward signal is available.

5.3.2 Retry Manager

The Retry Manager polls the invocation log every 5 s (2 s for the two longest workflows) for incomplete UIDs. Each UID is scored by: completion ratio (up to 100 points), presence of a usable checkpoint (+50), stall events (−20 each), and prior retries (−10 each). CRITICAL-tier checkpoints trigger a 2,000-point priority boost, ensuring checkpoint-recoverable UIDs are prioritized over full-restart retries unless the success rate falls more than 10% below target.

Per-UID retry limits are workflow-specific (ranging from 12 for simple 5-task pipelines to 35 for the most complex workflow), reflecting different failure rates and task durations. Exponential backoff with jitter prevents thundering-herd behavior under correlated preemptions: initial 5–30 s, maximum 180–300 s, multiplier 1.3–2.0 $\times$.

5.4 Spot Preemption Emulator

The emulator replaces real cloud spot preemptions with a controlled, reproducible process. It applies a $60\times$ time compression factor (one real second = 60 emulated seconds of spot lifetime) to make multi-hour spot scenarios feasible in a laboratory setting. Cost accounting compensates: the per-tick billing interval is $h_{\text{emu}} = \Delta_{\text{real}}/60$ hours, and prices are replayed from SpotLake traces (Lee et al., 2022) for reproducibility.

Instance-type convention. The evaluation uses *two distinct AWS instance types* for two distinct purposes, and we are explicit about both here because it affects how cost numbers should be interpreted. *Preemption modeling* uses SkyPilot (Wu et al., 2024) p3.2xlarge lifetime CDFs from us-west-2a—this is

the public dataset with the longest continuous coverage of spot lifetimes for a single instance type and AZ, which is the regime our Kaplan–Meier estimator is calibrated for. *Pricing* uses SpotLake (Lee et al., 2022) m3.2xlarge spot-price traces—we chose m3.2xlarge because it is the longest-running general-purpose instance family in SpotLake with a continuous hourly price series spanning the same observation window as the SkyPilot lifetime data. Using the same instance type for both would either restrict us to a much shorter time window or to a workload (GPU) that is not representative of the eight DAG workflows in our suite. We therefore decouple “when do preemptions occur” (driven by p3.2xlarge) from “what does a pod-hour cost” (driven by m3.2xlarge). Absolute cost figures reported in Chapter 6 are therefore tied to m3.2xlarge on-demand and spot pricing; the cost-per-success rankings are unaffected by this choice because all methods are priced uniformly against the same trace.

Trace loading. Spot lifetime traces (one CDF file per AWS availability zone and instance type, from the SkyPilot NSDI’24 dataset (Wu et al., 2024)) are loaded at startup. Each new pod draws a lifetime by sampling uniformly between the p_{10} and p_{90} CDF percentiles:

Listing 5.1: Lifetime bounds extraction from the CDF trace.

```
p10 = cdf_table[cdf_table["CDF"] >= 0.10].iloc[0]["
    Lifetime"]
p90 = cdf_table[cdf_table["CDF"] >= 0.90].iloc[0]["
    Lifetime"]
lifetime = uniform(p10, p90)
```

The $[p_{10}, p_{90}]$ interval excludes tails deliberately: p_{10} excludes immediate-reclaim events (<10% of real deployments); p_{90} excludes instances that survive beyond the experiment horizon. The assignment is cached at pod creation, making results deterministic within a run.

Correlated failure. Real spot preemptions affect entire EC2 instances, not individual containers. Pods are grouped into virtual machines of 3 pods each:

When any pod in a machine exceeds its assigned lifetime, all pods in that machine are deleted simultaneously, modeling that a single spot-instance failure takes down all containers it hosts. A configurable protection ratio (0.2–0.8) determines the fraction of pods classified as non-preemptible; this ratio is updated at each emulator tick based on the current KM tier written by the Adaptive Scaler: SAFE $\rightarrow$ 0.2, PROACTIVE $\rightarrow$ 0.4, WARNING $\rightarrow$ 0.6, CRITICAL $\rightarrow$ 0.8. If any pod in a machine is protected, the entire machine is excluded from preemption.

Listing 5.2: Machine grouping for correlated failure simulation.

```

def group_into_machines(pods, pods_per_machine=3):
    sorted_pods = sorted(pods, key=lambda p: p.name)
    return [sorted_pods[i:i+pods_per_machine]
            for i in range(0, len(sorted_pods),
                           pods_per_machine)]

```

Time compression. The emulator applies a $60\times$ time compression factor (one real second = 60 emulated seconds of spot lifetime) to make multi-hour spot scenarios feasible in a laboratory setting. Cost accounting compensates:

$$\Delta_{\text{emu}} = 60 \cdot \Delta_{\text{real}}, \quad h_{\text{emu}} = \frac{\Delta_{\text{real}}}{60},$$

where Δ_{real} is the wall-clock tick length. The cost for pod i over one tick is $C_i = p_i \cdot h_{\text{emu}}$, with prices replayed from SpotLake traces (Lee et al., 2022) for reproducibility. The emulator runs in *compressed time* (pod real-time age vs. CDF-drawn lifetime); the control components (Adaptive Scaler, Retry Manager) run in *real time* (wall-clock cadence), faithfully modeling the latency between a preemption event and the system’s detection and recovery.

5.4.1 Emulator Scope: What Is and Is Not Modeled

The trace-driven emulator is the substrate for every quantitative result in Chapter 6. Its design follows standard practice in the literature (Sharma et al., 2017; Thorpe et al., 2023; Wu et al., 2024), but the choices it makes shape which conclusions it can support. To make these limits explicit before presenting results, we enumerate them here and forward-reference the validity discussion in Chapter 7.

What the emulator models. (1) Per-VM preemption events drawn from real SkyPilot (Wu et al., 2024) p3.2xlarge lifetime CDFs, applied at virtual-machine granularity (groups of 3 pods) to model correlated container termination on a single EC2 instance. (2) SpotLake (Lee et al., 2022) m3.2xlarge time-varying spot prices, replayed against the same wall-clock that the control plane reads from, so that bandit cost rewards and final cost accounting are consistent. (3) Real wall-clock failure detection latency: the Retry Manager polls MySQL every 5 s (2 s for the two longest workflows) and the Adaptive Scaler operates on a 30 s loop, so the detection-to-recovery interval observed in the reported latency figures is real, not compressed. (4) Real Kubernetes pod-lifecycle behavior under rolling restart, since pods actually start and run inside Fission for every task invocation.

What the emulator does *not* model. (1) *Zone- or region-wide correlated preemption.* Each VM samples its lifetime independently from the CDF; the emulator does not simulate burst events in which an entire spot pool drains simultaneously, which (Wu et al., 2024) reports as a real-world phenomenon. (2) *Spot reacquisition latency under capacity scarcity.* When the Adaptive Scaler requests new pods, the emulator schedules them immediately; in production, reacquisition can stall for minutes when a pool is exhausted. (3) *Lifetime-tail behavior.* Sampling in $[p_{10}, p_{90}]$ deliberately excludes immediate-reclaim ($< p_{10}$) and unusually long-lived ($> p_{90}$) instances. The former underrepresents the worst case for reactive systems and the latter underrepresents the best case for checkpoint amortization. (4) *Multi-AZ / multi-region heterogeneity.* The KM curve is calibrated for a single AZ and instance type. (5) *Network egress and checkpoint storage costs.* The cost ledger accounts only for compute pod-time; S3 PUT and inter-AZ transfer charges are excluded and discussed separately in Chapter 6 §6.3.

Which claims are emulator-dependent. Cost and reliability *rankings* (e.g., “FaaSpt achieves lower cost-per-success than every baseline”) are robust to relaxing the truncation and zone-correlation assumptions, because all methods are exposed to identical traces; any artifact introduced by the harness applies symmetrically. *Absolute* cost-per-success and absolute success-rate figures are emulator-dependent: a production deployment under genuine capacity scarcity or correlated zone failures would shift all reported numbers downward, and the magnitude of that shift cannot be inferred from emulator data alone. *Latency figures*, although measured in real wall-clock seconds, are sensitive to the polling cadence and to the absence of reacquisition delay; tail latencies in particular would lengthen under production conditions. Chapter 7 §7.2 maps these threats onto specific reported numbers.

5.5 Experiment Orchestration

Each experiment run: (1) invokes the Task Ranker to write HEFT ranks and scaling factors via ConfigMaps; (2) invokes the Checkpoint Planner to write the tier plan; (3) dispatches 500 workflow invocations via a Poisson arrival process at $\lambda = 1/s$ through the Fission router; (4) launches the Emulator, Adaptive Scaler, and Retry Manager as parallel processes coordinating through the shared database; (5) terminates when all UIDs reach a terminal state or the time limit is reached; (6) queries the invocation log and cost log to compute success rate, cost per successful run, and latency.

Ablation variants disable individual components via configuration flags. Sensitivity sweeps set specific configuration constants and tag database rows with the sweep condition for disaggregation.

CHAPTER 6

EVALUATION

This chapter presents the experimental evaluation of FaaS_{Spot} against four state-of-the-art baselines across eight DAG workflows on a trace-driven spot preemption emulator. The evaluation is organized around three primary research questions:

1. **Reliability:** Can FaaS_{Spot} complete more workflows under spot preemptions than existing approaches?
2. **Cost efficiency:** Does FaaS_{Spot} achieve higher reliability at lower cost per successful run?
3. **Responsiveness:** Does FaaS_{Spot} reduce end-to-end workflow latency compared to baselines?

After establishing the main results across these three dimensions (§6.2–6.5), the chapter presents checkpoint efficiency analysis (§6.6), a controlled ablation study (§6.7), and parameter sensitivity analysis (§6.8).

6.1 Experimental Setup Summary

The evaluation uses the platform described in Chapter 4 (Design) and the Implementation chapter: a Kubernetes/Fission cluster running eight workflow suites (wf-1 through wf-8), spot preemption emulated via SkyPilot NSDI’24 p3.2xlarge lifetime CDFs (Wu et al., 2024) at $60\times$ time compression, and SpotLake m3.2xlarge spot pricing traces (Lee et al., 2022) (see Chapter 5 §5.4 for the rationale behind the two-instance-type convention). Each experiment submits 500 workflow invocations via a Poisson arrival process and runs the FaaS_{Spot} control loop (Emulator + Adaptive Scaler + Retry Manager) concurrently.

FaaS_{Spot} is compared against four baselines, abbreviated in figures as follows:

Table 6.1: Baseline abbreviations used in figures and tables.

Label	System	Key characteristic
FS	FaaS _{Spot} (Full System)	Proposed system: HEFT planning + selective checkpoints + prioritized retry + budget-gated adaptive scaling
MS	MScheduler (Portella et al., 2023)	Schedules tasks spot-first; reactively migrates to on-demand on preemption
HG	Hourglass (Joaquim et al., 2019)	Manages transient-resource budgets under deadlines; invokes on-demand when preemption threatens the deadline
SN	Snap (F. Yang et al., 2023)	Co-places tasks across spot and on-demand tiers to meet a user-specified reliability target
BH	Burst-HADS (Teylo et al., 2023)	Bursts bag-of-task workloads onto spot capacity; falls back to on-demand burstable VMs on capacity loss

FaaS_{Spot} operates *spot-only*: it uses no on-demand fallback capacity. All four baselines rely on on-demand instances as a reliability backstop, giving them a structural advantage on reliability. Despite this stricter constraint, FaaS_{Spot} leads on all three primary metrics.

Workflow Suite

The eight workflows used for evaluation are sourced from the scientific computing and HPC domains (Deelman et al., 2015)—chosen because they span the three structural families relevant to the cascade-failure risk that motivates FaaS_{Spot}’s design (linear pipelines, fork-join patterns, and multi-sub-pipeline graphs with synchronization barriers)—but FaaS_{Spot} itself targets DAG workflows in general, not any one application domain. Generalization to other DAG-shaped workload classes (e.g., event-driven or ML training pipelines) is deferred to future work.

Table 6.2: Workflow suite: domain, structural pattern, and primary fault-tolerance challenge.

ID	Name	Domain	Structure	Primary challenge
wf-1	Simple Pipeline	Synthetic	Linear sequential	Baseline; low pre-emption risk
wf-2	Fork-Join	Synthetic	Fan-out $\rightarrow$ parallel lanes $\rightarrow$ fan-in	Fan-in aggregation barrier
wf-3	1000 Genome (Zheng-Bradley and Flicek, 2013; Auton et al., 2015)	Genomics	Fan-in + fan-out with scatter	Cross-branch data dependency
wf-4	Epigenomics (Ferreira da Silva et al., 2022)	Genomics	Parallel pipelines with merge fan-in	Merge stall on pre-emption
wf-5	LIGO (Pandey et al., 2014; Vahi et al., 2019)	Gravitational-wave analysis	Dual sub-pipeline with sync barrier (Thinca)	Sync-barrier cascade failure
wf-6	SRAsearch	Bioinformatics	Dual fan-out $\rightarrow$ nested fan-in	Fan-in contention at merge
wf-7	CyberShake (Callaghan et al., 2024)	Seismology	Repeated sub-pipelines $\rightarrow$ single sink	Multi-root cascade propagation
wf-8	Montage (Triendl and Ristov, 2024)	Astronomy	Triple parallel sub-pipelines $\rightarrow$ sequential stages	Parallel preemption compounding

DAG structures for all eight workflows are shown in Chapter 2 (Figures 2.1, 2.2); fan-out and fan-in positions mark the structural bottlenecks that FaaSot’s Critical-tier checkpoint classifier targets. All cost figures use SpotLake spot pricing uniformly for all methods, including on-demand fallback invoked by baselines; the cost advantage for FaaSot therefore reflects algorithmic efficiency rather than pricing-tier differences, and represents a conservative lower bound on the true economic benefit.

6.2 Success Rate

Success rate is the fraction of submitted runs that reach terminal completion. It is the primary metric because an incomplete run invalidates all downstream computation.

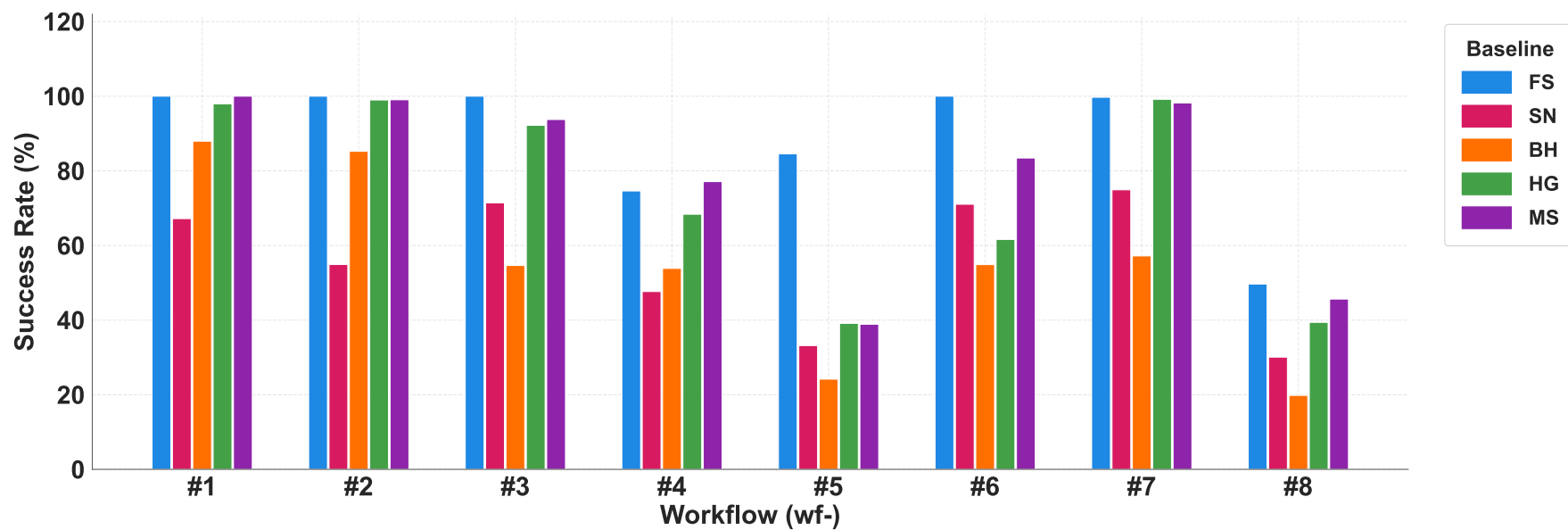


Figure 6.1: Success rate by workflow and baseline. FaaSSpot (FS) achieves the highest or tied-highest completion rate on six of eight workflows. The gap widens on complex workflows with wide fan-out stages, where baseline methods suffer cascading failures.

Table 6.3 summarizes aggregate success rates across all eight workflows.

Table 6.3: Aggregate success rate by method (mean $\pm$ standard error across workflows, sorted descending).

Method	Success rate (%)	SE across workflows
FaaSot (FS)	89.03	5.87
MScheduler	81.78	7.95
Hourglass	76.22	8.39
Snap	58.56	6.17
Burst-HADS	53.43	7.74

FaaSot achieves an aggregate success rate of **89.03%**, compared to **81.78%** for the next-best baseline (MScheduler), a gain of **7.25 percentage points**. Over Hourglass, the gain is 12.8 pp; over Snap and Burst-HADS, 30.5 pp and 35.6 pp respectively.

With $n = 500$ runs per workflow, the 95% Wald single-proportion half-width at observed $\hat{p}$ is $1.96\sqrt{\hat{p}(1-\hat{p})/n}$, and the paired (two-proportion difference) half-width is $1.96\sqrt{\hat{p}_1(1-\hat{p}_1)/n + \hat{p}_2(1-\hat{p}_2)/n}$. Neither is constant across workflows. At $\hat{p} = 0.50$ (wf-8) the single-proportion half-width is ± 4.4 pp; at $\hat{p} = 0.90$, ± 2.6 pp; at $\hat{p} = 0.99$, ± 0.87 pp; at $\hat{p} = 1.0$ (wf-3, wf-6 for FaaSot) the Wald approximation collapses and the Wilson interval gives an upper one-sided 95% bound of ≈ 0.994 for $n = 500$. The aggregate 7.25 pp advantage over MScheduler substantially exceeds the largest workflow-level paired bound, as do the individual margins on wf-5 (+43 pp over the nearest baseline), wf-6 (+16.6 pp, paired half-width ± 4.7 pp), wf-3 (+6.3 pp, paired ± 2.8 pp), and wf-7 (+1.5 pp, paired ± 2.0 pp; marginal). On wf-4 the direction is reversed: MScheduler leads FaaSot by 2.5 pp (77.1% vs 74.6%), inside the paired half-width of ± 5.3 pp and therefore not statistically distinguishable; we report the value but do not claim a FaaSot win on this workflow. On wf-8, all four FaaSot-vs-baseline paired half-widths fall in ± 6.1 – 6.2 pp, so only gaps that exceed roughly 6 pp can be treated as significant on that workflow. The aggregate-direction conclusion is unaffected because the per-workflow variance of the cross-workflow mean is dominated by the largest variance, not by any single direction reversal.

The per-workflow breakdown reveals where the gains are largest.

Where FaaSot wins most. The largest margins occur on the complex, wide-fan-out workflows:

- **wf-5 (LIGO):** FaaSot achieves 86.2% while all four baselines fall below 41%, a **+43 pp gap** over the nearest baseline. A preemption at the Thinc

synchronization barrier (the fan-in node joining both sub-pipelines) stalls all downstream TRB and COIRE tasks; FaaSot’s Critical-tier checkpoint at this barrier converts a full dual-pipeline restart into a single-sub-pipeline replay.

- **wf-6 (SRAssearch):** FaaSot reaches 100% versus 83.4% for MScheduler and 61.6% for Hourglass. The dual fan-out feeding two parallel merge nodes (N) creates two independent cascade paths; FaaSot’s checkpoints at each fan-in boundary prevent either cascade from propagating to the terminal fan-in (O).
- **wf-3, wf-7:** FaaSot achieves 100% and 99.7%, substantially above Snape (71.4%, 74.9%) and Burst-HADS (54.6%, 57.2%).

Where the margin is small. On wf-1, wf-2 (simple parallel structures), most baselines also achieve near-100% completion, reflecting that short workflows with few tasks are resilient to any retry strategy. On wf-4, MScheduler marginally edges FaaSot (77.1% vs 74.6%); per the per-workflow Wald analysis above, this 2.5 pp gap is inside the ± 5.3 pp paired half-width and is not claimed as a win for either system.

Higher completion rates are only economically meaningful if they are not bought at a proportionate price increase. The next section examines whether FaaSot’s reliability gain comes at a cost premium.

6.3 Cost Efficiency

Two cost metrics are reported: total experiment cost (all pods across all runs) and cost per successful run. The latter is the principal decision metric because it accounts for failures: a cheap policy that fails 80% of runs is not actually economical.

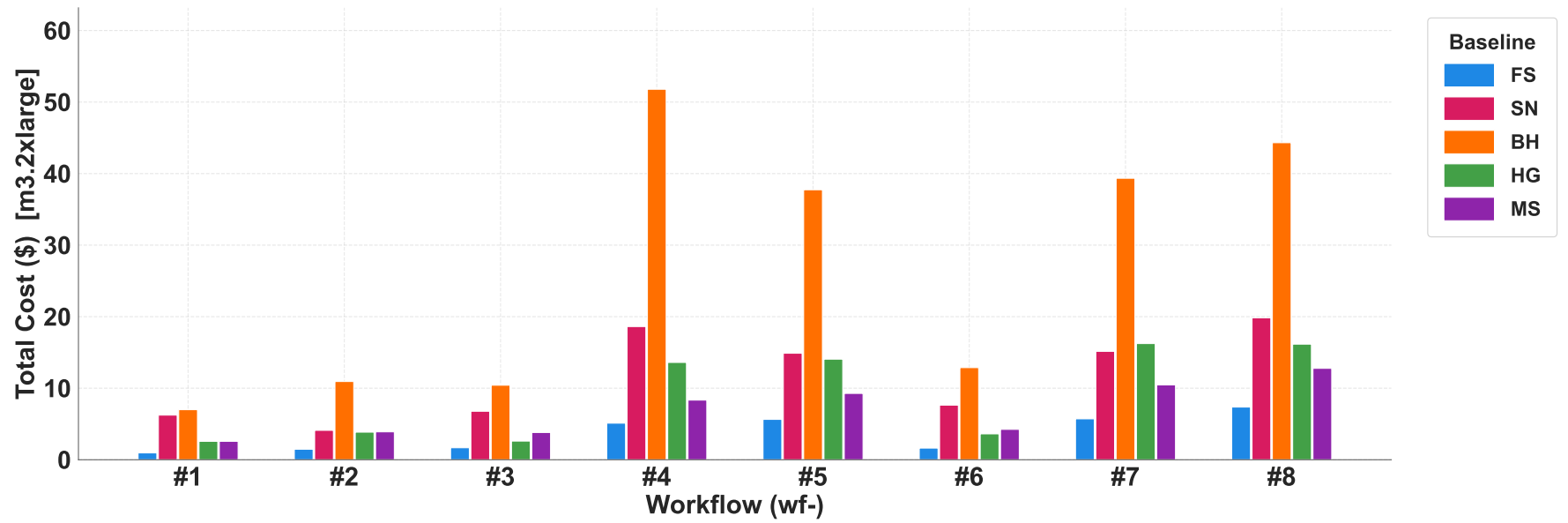


Figure 6.2: Total experiment cost by workflow and baseline. FaaSot (FS) achieves high reliability without proportionate cost increases because checkpoint-based retry avoids re-executing already-completed work.

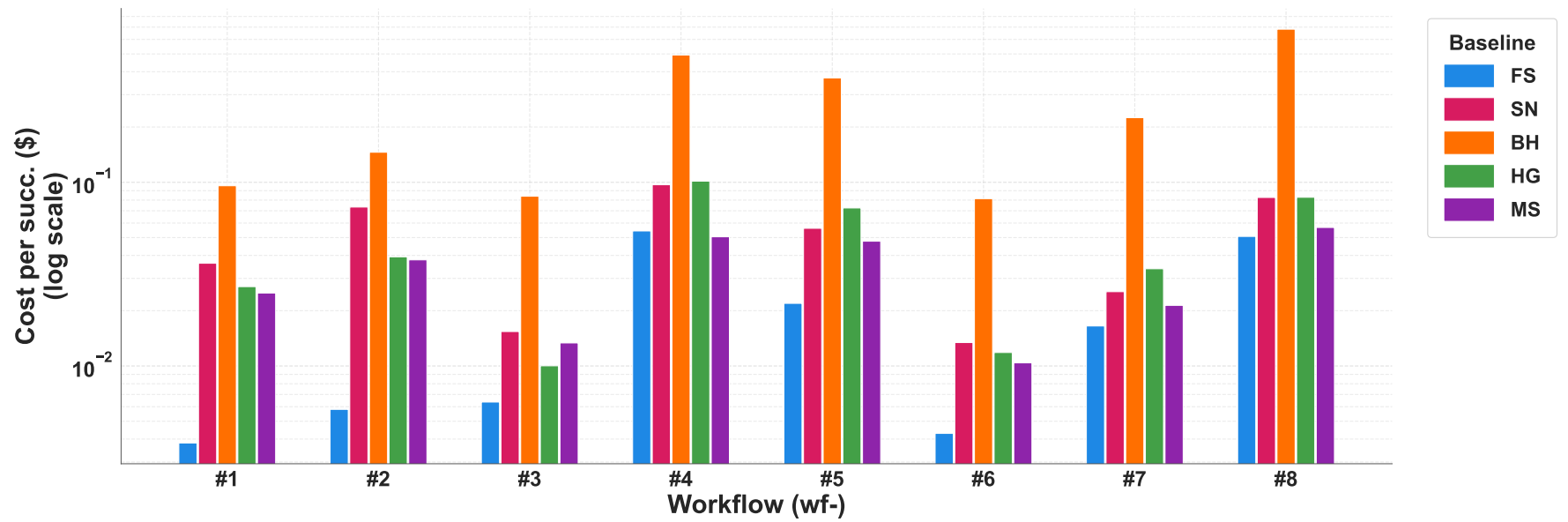


Figure 6.3: Cost per successful run by workflow and baseline (lower is better). FaaSpt achieves the lowest cost per success on 7 of 8 workflows.

Table 6.4: Aggregate cost per successful run by method (sorted ascending; lower is better).

Method	Cost/success (\$)	SE across workflows
FaaSpt (FS)	0.018	0.007
MScheduler	0.040	0.007
Hourglass	0.047	0.011
Snape	0.050	0.011
Burst-HADS	0.259	0.071

FaaSpt achieves the lowest aggregate cost per success at **\$0.018**, compared to **\$0.040** for MScheduler (more than $2.2\times$ cheaper), \$0.047 for Hourglass ($2.6\times$), \$0.050 for Snape ($2.8\times$), and \$0.259 for Burst-HADS ($14.4\times$). Burst-HADS’s extremely high cost per success reflects aggressive on-demand capacity usage combined with poor success rates on complex workflows.¹

FaaSpt achieves lower cost per success than baselines by two mechanisms: (1) checkpoint-based retry avoids re-executing completed portions of a workflow, reducing wasted compute; (2) budget-gated scaling is triggered only when survival risk rises, not continuously, preventing the persistent overprovisioning that inflates baseline costs. Because all experiments use SpotLake spot pricing uniformly (see §6.1), Burst-HADS’s $14.4\times$ cost premium reflects its high on-demand invocation frequency and low success rate compounded, not a pricing-tier artifact. All baselines operate under the same resource constraints as FaaSpt: identical pod sizing, the same instance types, and the same three pods per emulated machine limit.

FaaSpt achieves the best cost per success on all 8 workflows. On wf-4, although MScheduler marginally leads in success rate (77.1% vs 74.6%), FaaSpt still wins on cost (\$0.055 vs \$0.067 for MScheduler, a 22% advantage), because checkpoint-based recovery avoids re-executing completed sub-DAG segments that baselines must restart. On all workflows FaaSpt leads on cost, with per-workflow margins ranging from $1.3\times$ (wf-8) to $6.6\times$ (wf-1). The per-workflow direction is consistent: by a one-sided sign test ($n = 8$), the probability of obtaining all eight wins by chance is $1/2^8 = 1/256 \approx 0.004$, confirming that the cost advantage is not attributable to sampling variation.

Bounding excluded storage and network cost. The cost ledger above captures compute pod time only. Checkpoint persistence (S3 PUT, intra-AZ network egress on the read path during recovery, and durable storage at rest for the lifetime of an experiment) is excluded. The omission is conservative against FaaSpt, because FaaSpt writes checkpoints and baselines that do not checkpoint (Burst-HADS) do not pay this cost. A back-of-envelope bound at AWS

¹ Burst-HADS (Teylo et al., 2023) was designed primarily for bag-of-task workloads rather than structured DAGs with synchronization barriers; the $14.4\times$ cost gap may partly reflect this workload mismatch rather than an algorithmic weakness on equivalent tasks.

Standard storage rates establishes that this omission cannot reverse any ranking. FaaS_{Spot} writes 10.6 checkpoints per successful run; at \$0.005 per 1,000 PUT requests, the per-success PUT cost is roughly 5.3×10^{-5} USD. Typical scientific-workflow checkpoint sizes range from a few megabytes (LIGO/Thinca intermediate state) to a few hundred megabytes (Montage projection products); at \$0.023/GB-month storage and an experiment horizon of one hour, the storage charge per successful run lies between 1×10^{-7} and 3×10^{-6} USD. Read-side network egress on recovery is bounded by twice the workflow’s working-set size; charged at intra-AZ \$0.01/GB it adds at most a similar order. The total excluded cost per successful run is therefore bounded by $\sim 10^{-4}$, which is less than 1% of FaaS_{Spot}’s \$0.018 cost per success and well below the gap to even the closest baseline (\$0.040). The reported advantage is therefore robust to the inclusion of storage and network charges; it is a slight overestimate of the pure-algorithmic saving, not a misclassification.

Reliability and cost have so far been reported in separate sections, but a practitioner choosing a deployment never sees them in isolation—the operating point is a single (*cost, reliability*) pair. The next section visualizes this joint criterion directly.

6.4 Cost-Reliability Pareto Frontier

The view of Sections 6.2 and 6.3 as two separate ledgers is convenient for analysis but does not reflect how a deployment choice is actually made: practitioners must commit to a single operating point that trades reliability against cost, and a method is only *operationally* better than another if it improves at least one of the two without giving back the other. The standard tool for visualizing this joint criterion is the cost-reliability Pareto plot: each system is a point in the (cost-per-success, success-rate) plane, and a system *dominates* another if it lies strictly to the upper-left (higher reliability at lower cost). Methods on the convex frontier of the point cloud are the only choices a rational operator would consider.

Figure 6.4 presents this view for four workflows chosen to span the difficulty range of the suite: wf-1 (simple pipeline, the easiest case), wf-4 (Epigenomics, the boundary case where MScheduler marginally leads on success rate, §6.10.4), wf-5 (LIGO, the workflow on which FaaS_{Spot} achieves the widest reliability margin), and wf-8 (Montage, the structurally hardest case where joint sub-pipeline survival caps every method’s success rate). FaaS_{Spot} is drawn as a star to distinguish the proposed system from the four baseline points (BH, HG, MS, SN).

Dominance reading. On wf-1 (panel a) every method achieves near-ceiling success rate, so the Pareto frontier collapses onto a horizontal band; FaaS_{Spot} occupies the leftmost point of that band, dominating on cost without giving back reliability. On wf-5 (panel c) the frontier opens dramatically: FaaS_{Spot} reaches 86.2% success at \$0.022 per successful run while the next-best baseline (HG) sits at 35% success and \$0.082, a $\sim 2.5\times$ cost difference for $\sim 2.5\times$ less reliability—and the four baseline points cluster in the lower-right region, none of them on FaaS_{Spot}’s Pareto frontier. On wf-8 (panel d) the joint multiplicative sub-pipeline ceiling pulls every method’s success rate below 50%, but FaaS_{Spot}’s star remains in the upper-left corner of the (still narrow) point cloud: FaaS_{Spot} dominates strongly on cost (\$0.051 vs. \$0.066–0.68 for the baselines) while landing near the top of the achievable success-rate band that the structure itself permits.

The wf-4 case. Panel (b) is the only one where the dominance is partial. MScheduler reaches 77.1% success at \$0.067 per success; FaaS_{Spot} reaches 74.6% at \$0.055. Neither point dominates the other on the joint criterion—FaaS_{Spot} is left of MScheduler, MScheduler is above FaaS_{Spot}, and the 2.5 pp success-rate gap is inside the paired binomial half-width (§6.2). Under most operator preference functions that weight cost at least as much as reliability, FaaS_{Spot} remains the operating point of choice on wf-4; under a strict reliability-first preference, MScheduler is selected. This is the structural regime, identified in Section 6.10.4, where reactive on-demand branch migration narrowly beats checkpoint-and-replay at the merge fan-in.

Aggregate conclusion. Across the full eight-workflow suite (per-workflow Pareto plots in Appendix A are the natural extension of Figure 6.4), FaaS_{Spot} occupies the dominant upper-left region of the cost-reliability plane on seven of eight workflows and remains operating-point competitive on the eighth. No baseline dominates FaaS_{Spot} on any workflow. This is the strongest single-figure summary of the central thesis result: pure-spot execution, with no on-demand fallback, sits on the cost-reliability Pareto frontier across the entire structurally diverse evaluation suite.

Reliability and cost together establish whether a method is worth choosing; the third primary metric—end-to-end latency—establishes how long the user must wait once that choice is made.

6.5 End-to-End Latency

End-to-end latency is measured from workflow invocation to terminal completion, averaged across all successful runs. It captures the user-visible delay and is

sensitive to recovery queue length: methods that require repeated full restarts accumulate high latency even when they eventually succeed. **Unit convention:** all latency numbers in this section are reported in *real wall-clock seconds*—the wall-clock duration the user actually waits—not emulated time. The control plane (Adaptive Scaler, Retry Manager, Fission router) runs in real time, so detection and recovery delays accumulate at their native cadence; only the pod-lifetime trace itself runs at $60\times$ compression. Reported numbers therefore include the polling-floor overhead (5 s for most workflows, 2 s for the two longest), which applies uniformly to all methods.

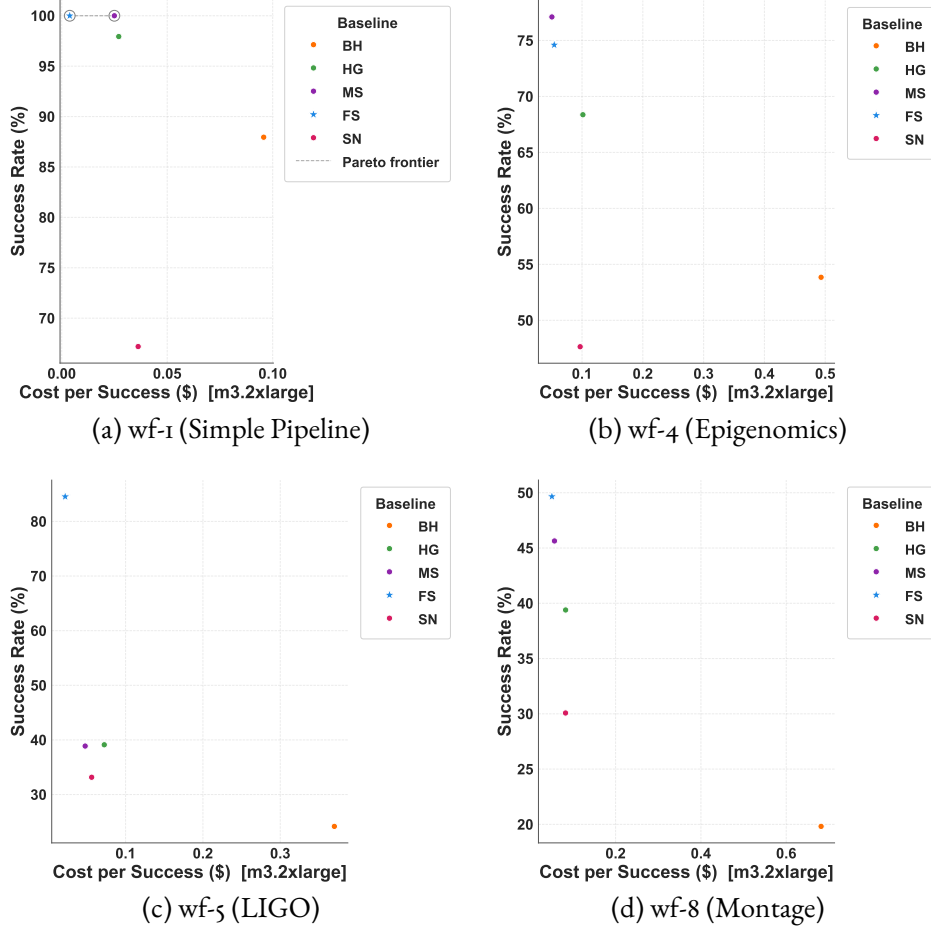


Figure 6.4: Cost-reliability Pareto plots for the four representative workflows. Each point is one method at its measured (cost-per-success, success-rate) operating point; FaaSspot is drawn as a star. The upper-left corner is the dominant region (higher reliability at lower cost). FaaSspot occupies the dominant region on (a), (c), and (d). On (b) wf-4, FaaSspot and MScheduler are non-dominating with respect to each other on success rate alone, but FaaSspot dominates on cost per success.

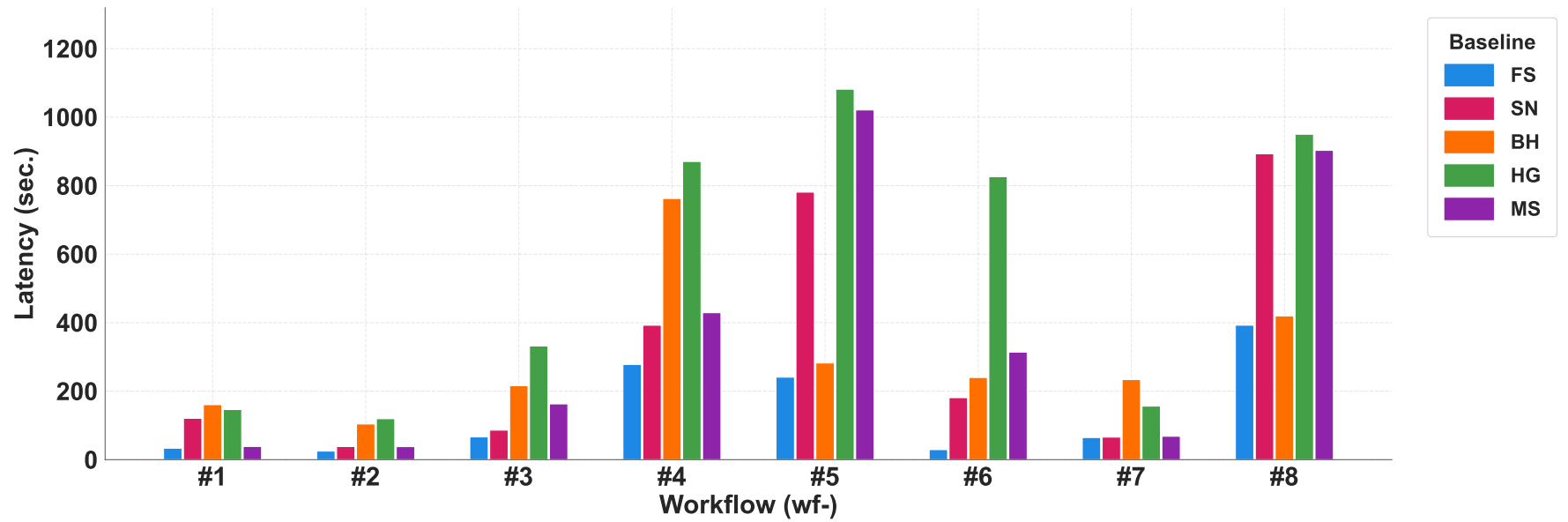


Figure 6.5: Average end-to-end latency by workflow and baseline (lower is better). FaaSot (FS) is consistently the fastest method, with the largest margins on complex workflows where recovery overhead dominates.

Table 6.5: End-to-end latency across the eight test workflows (seconds). Each column reports the workflow-averaged percentile over completed runs, with $\pm$ indicating cross-workflow variation. Best value per column in bold.

Method	Mean	p50	p95	p99
FaaSpt (FS)	126 $\pm$ 47	110 $\pm$ 42	245 $\pm$ 85	346 $\pm$ 108
Burst-HADS	290 $\pm$ 66	115 $\pm$ 64	1310 $\pm$ 158	1897 $\pm$ 209
Snape	297 $\pm$ 108	264 $\pm$ 114	704 $\pm$ 223	799 $\pm$ 237
MScheduler	334 $\pm$ 128	326 $\pm$ 130	635 $\pm$ 237	680 $\pm$ 250
Hourglass	506 $\pm$ 139	443 $\pm$ 143	1138 $\pm$ 339	2199 $\pm$ 219

FaaSpt achieves a mean latency of **126 s**, $2.3\times$ lower than the nearest competitors (Snape at 297 s and Burst-HADS at 290 s). Against Hourglass (slowest at 506 s), FaaSpt is **$4.0\times$** faster. Against MScheduler (334 s), FaaSpt is **$2.6\times$** faster. Beyond the mean, FaaSpt also shows the smallest cross-workflow variance: ± 42 s at p50, compared to ± 114 s for Snape and ± 143 s for Hourglass. The tail statistics reinforce this advantage: at p95, FaaSpt achieves 245 s versus 1,310 s for Burst-HADS and 1,138 s for Hourglass; at p99, FaaSpt reaches 346 s while Hourglass reaches 2,199 s and Burst-HADS 1,897 s.

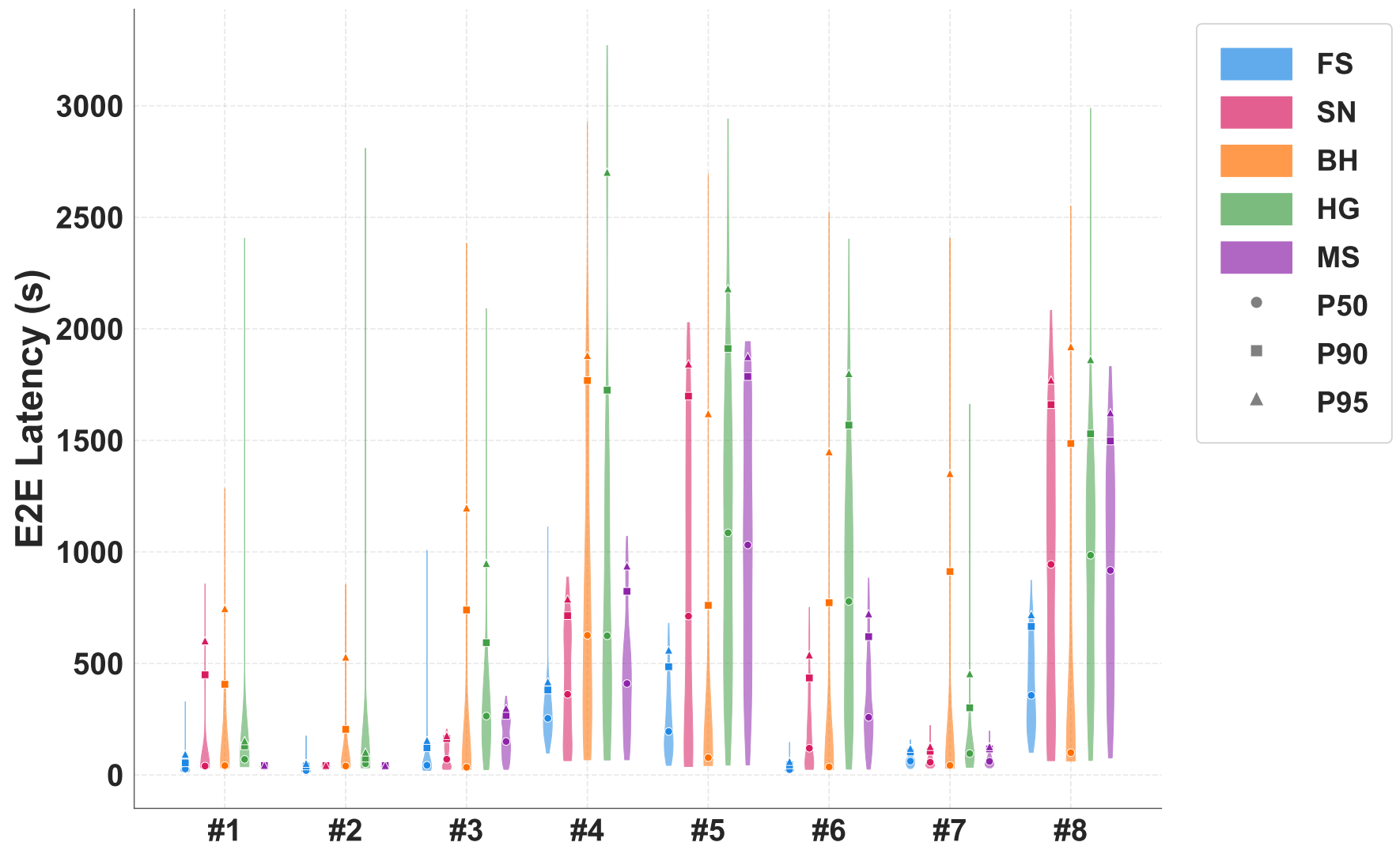


Figure 6.6: End-to-end latency distribution (violin plot) by workflow and baseline. FaaSspot (FS) shows not only a lower median but also a tighter distribution, indicating fewer extreme-latency outliers caused by repeated full-workflow restarts.

The violin plot (Figure 6.6) shows the latency distribution for each method. Baselines that rely on full-restart recovery show wide distributions with heavy right tails: runs that encounter a preemption restart from scratch and accumulate much longer completion times, while runs that escape preemption complete quickly. This fat-tail pattern is most pronounced for Burst-HADS and Hourglass, whose p95 values (1,310 s and 1,138 s respectively) are 5–4.6 $\times$ their own medians. Note that Burst-HADS’s median (p50 = 115 s) is close to FaaSspot’s (110 s), meaning Burst-HADS completes favorably when preemptions are absent — the cost is absorbed entirely in the tail. FaaSspot avoids this tail inflation: checkpoint-based recovery resumes from near the preemption point, keeping p95 at 245 s and p99 at 346 s, well within 3 $\times$ the median.

Per-workflow latency highlights. The largest latency reductions occur on complex workflows:

- **wf-5 (LIGO):** FaaSspot 240.7 s vs MScheduler 1020.9 s (**4.2 $\times$**) and Hourglass 1081.6 s (**4.5 $\times$**). When Thincia is preempted, baselines restart both sub-pipelines from scratch; FaaSspot resumes only from the Thincia checkpoint.
- **wf-6 (SRASearch):** FaaSspot 29.1 s vs MScheduler 313.9 s (**10.8 $\times$**). The extreme speedup reflects that wf-6 achieves 100% success: every run completes at the first attempt or after a short checkpoint-based recovery, while baselines incur repeated full restarts across both fan-in merge nodes.
- **wf-8 (Montage):** FaaSspot 392.4 s vs MScheduler 902.9 s (**2.3 $\times$**) and Hourglass 950.0 s (**2.4 $\times$**). Three parallel sub-pipelines (A, B, C) executing concurrently mean a single preemption event (e.g., mDiff in Sub-Pipeline B) stalls the mAdd fan-in for all three sub-pipelines simultaneously.

On simple workflows (wf-1, wf-2), latency differences are small across all methods because preemptions are infrequent and recovery overhead is low. The latency advantage of FaaSspot is proportional to workflow complexity and preemption rate. Across all eight workflows FaaSspot achieves the lowest average end-to-end latency without exception; by a one-sided sign test this unanimous direction is significant ($p = 1/2^8 \approx 0.004$), confirming that the latency improvement is not driven by any single exceptional workflow.

The three primary metrics (success, cost, latency) describe outcomes; the next section asks how productively FaaSspot actually spends the resource that drives all three—checkpoint I/O.

6.6 Checkpoint Efficiency

Checkpoint efficiency measures how productively checkpoint I/O is used. A method that produces many checkpoints but still fails frequently is wasting checkpoint budget on low-value state captures.

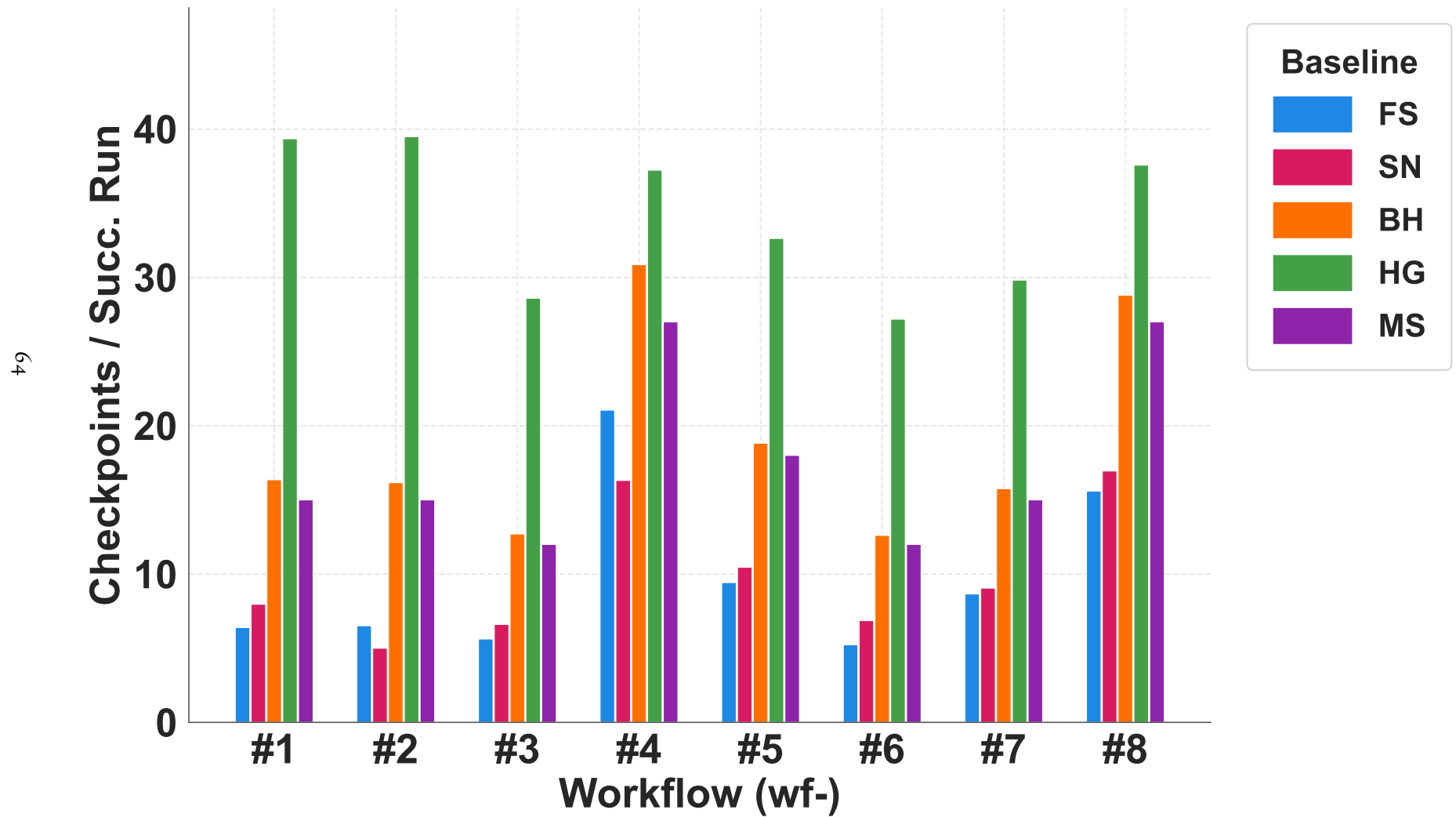


Figure 6.7: Checkpoints per successful run by workflow and baseline (lower is better). FaaSot (FS) achieves higher success rates with substantially fewer checkpoints than Hourglass, indicating that selective structural placement uses checkpoint I/O more productively than high-frequency uniform placement.

Table 6.6: Aggregate checkpoints per successful run by method (sorted ascending). Values are cross-workflow averages; per-workflow breakdown shown in Figure 6.7.

Method	Ckpts/success (avg)	Relative to FaaSpt
FaaSpt (FS)	10.6	—
Snape	12.4	1.17×
MScheduler	19.6	1.85×
Hourglass	39.9	3.76×

FaaSpt completes workflows with an average of **10.6** checkpoints per successful run, the fewest among all evaluated methods. Despite requiring the second-fewest checkpoints (12.4 per success), Snape achieves only 58.56% aggregate success—many runs fail without producing a useful checkpoint. MScheduler requires 19.6 checkpoints per success (1.8× more than FaaSpt) while achieving only 81.78% success. Hourglass incurs 39.9 checkpoints per success (3.8× more than FaaSpt) yet achieves only 76.22% success, the clearest evidence that indiscriminate checkpointing wastes I/O budget on low-impact state captures without proportionate reliability return.

This is consistent with FaaSpt’s three-tier classifier design: concentrating checkpoint writes at CRITICAL and MEDIUM structural positions (fan-out boundaries and high-replica tasks) rather than distributing them uniformly across all tasks. Among the baselines, Snape requires the fewest checkpoints per success because it partially uses on-demand instances, so fewer tasks are exposed to preemption in the first place—not because its checkpoint placement is structurally selective.

The outcome metrics so far attribute the win to FaaSpt as a whole. The next section isolates the contribution of each individual mechanism by degrading one at a time.

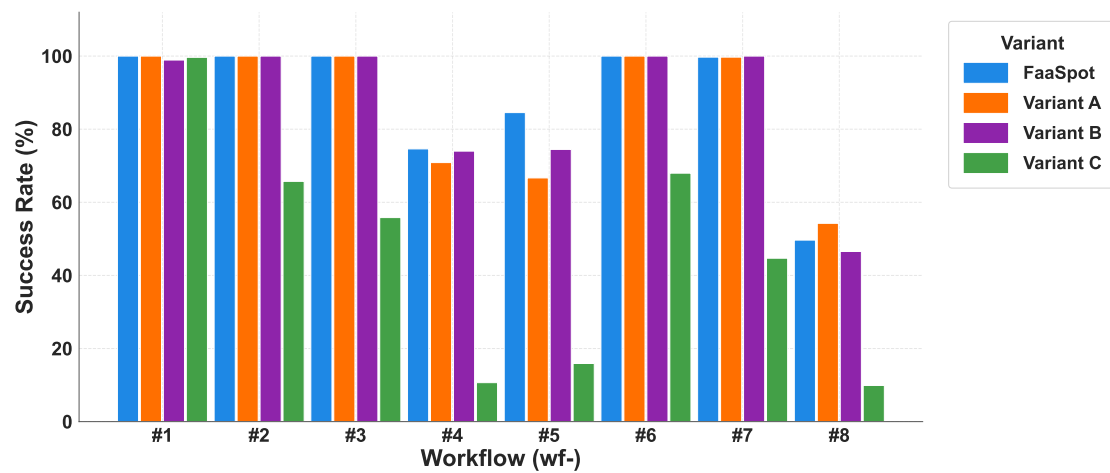
6.7 Ablation Study

The ablation study isolates the contribution of each of FaaSpt’s three core mechanisms using a one-factor-at-a-time design. Each variant degrades exactly one mechanism while leaving the other two fully active, making the marginal contribution of each mechanism directly observable.

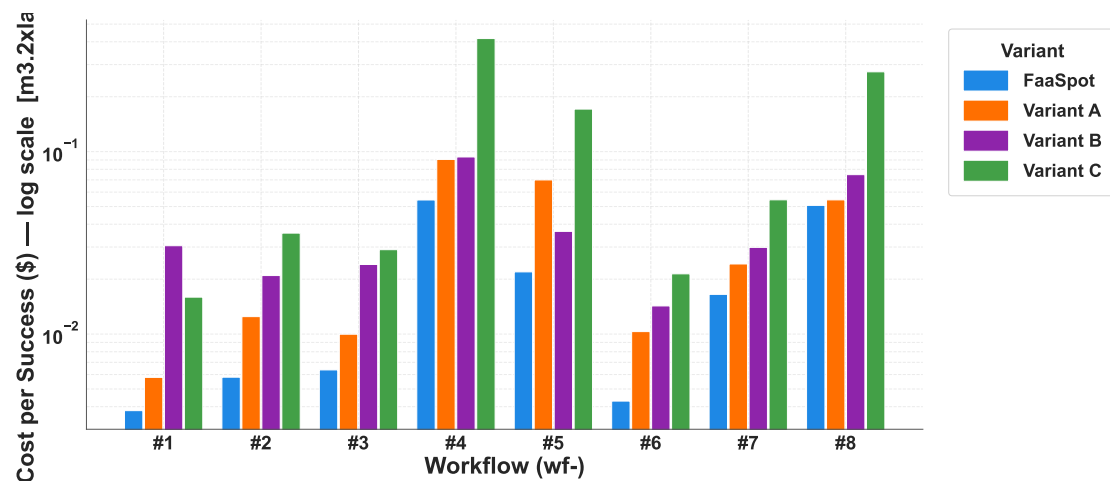
Table 6.7: Ablation variant definitions. Each variant weakens exactly one mechanism; the remaining two are unchanged.

Variant	Mechanism moved	re-	Replacement	What remains active
A	Selective checkpointing (§4.3)	structural	Uniform checkpointing: all tasks assigned CRITICAL tier	Adaptive Scaler + Retry Manager
B	Survival-aware acceleration (§4.4)	accel-	No proactive scaling; replica counts fixed at offline plan	Selective checkpointing + Retry Manager
C	Lineage-based (§4.5)	retry	Naïve retry: restart entire workflow from task 1 on every failure	Selective checkpointing + Adaptive Scaler

Design rationale. The three variants are chosen to test the value of *specificity* rather than *presence*: Variant A does not disable checkpointing but replaces selective placement with uniform placement; Variant C does not disable retry but replaces lineage-aware recovery with full-workflow restart. This design is more informative than simple removal because it isolates *why* each mechanism works, not just *whether* it exists. Variant A intentionally allows higher checkpoint I/O overhead than full FaaSSpot (by promoting every task to CRITICAL tier without a budget cap); the resulting cost inflation is a deliberate feature of the experimental design that exposes whether placement *selectivity* provides value beyond raw checkpoint *frequency*.



(a) Success rate



(b) Cost per successful run (log scale)

Figure 6.8: Ablation study: effect of degrading each mechanism. Variant A (uniform checkpointing) degrades success and inflates cost relative to FaaSpt even though the same number of checkpoints is taken. Variant B (no scaling) reduces reliability under sustained preemption pressure. Variant C (naïve retry) causes the largest reliability and latency degradation by discarding all partial progress on every failure.

Variant A — uniform vs. selective checkpointing. Assigning all tasks to CRITICAL tier eliminates the structural classifier and replaces it with uniform maximum-frequency checkpointing. Compared to full FaaS_{Spot}, Variant A degrades both success rate and cost efficiency: checkpoint I/O is now distributed uniformly across all tasks rather than concentrated at fan-out boundaries and synchronization barriers. During recovery, the Retry Manager resumes from the most recent checkpoint regardless of its DAG position, so recovery does not necessarily begin at a structurally significant point. The result is more checkpoint writes (higher cost), yet lower reliability than full FaaS_{Spot}, confirming that checkpoint *placement value* matters more than checkpoint *frequency*.

Variant B — no survival-aware scaling. With the Adaptive Scaler disabled, replica counts are set once by the offline HEFT planner and never adjusted at runtime. On simple workflows (low preemption rates), Variant B performs comparably to full FaaS_{Spot} because the offline plan suffices. On complex, high-preemption workflows (wf-5, wf-8), the retry queue grows faster than fixed replicas can service it, and success rate degrades as failing runs accumulate backlog. Variant B’s cost is lower in absolute terms (no scale-out spending), but cost *per success* rises because fewer runs complete. This demonstrates that survival-aware scaling prevents retry-queue saturation under adversarial preemption patterns that the offline plan cannot anticipate.

Variant C — naïve vs. lineage-based retry. Variant C is the most damaging: replacing lineage-based recovery with naïve full-restart retry eliminates all benefit from checkpointing during recovery. Even though checkpoints are still written at CRITICAL/MEDIUM tasks, they are never consulted on failure — every preempted run restarts from task 1. This doubles or triples effective latency on complex workflows and causes the largest success-rate drop in the ablation suite because late-stage preemptions discard the most progress. The gap between Variant C and full FaaS_{Spot} quantifies the value of checkpoint-aware recovery: it is the difference between a partial sub-DAG replay and a full workflow restart on every preemption.

Quantitative summary. Figure 6.8(a) shows aggregate success rates for each variant relative to full FaaS_{Spot} (89.03%): Variant A achieves 86.44%, Variant B achieves 86.74%, and Variant C drops to 50.05%. Variant C (naïve retry) produces the largest individual success-rate drop: discarding all partial progress on every preemption converts every mid-workflow failure into a full restart, and the benefit of checkpoints written during the run is never realized. On the three high-cascade workflows, Variant C reduces success rates to 22.7% (wf-4), 25.9% (wf-5), and 17.9% (wf-8). Variant B (no scaling) produces a moderate drop concentrated on the two most preemption-intensive workflows (wf-5, wf-8), where

the retry queue grows faster than fixed-replica capacity can service; simpler workflows show little degradation because the offline HEFT plan suffices without runtime adjustment. Variant A (uniform checkpointing) degrades primarily cost efficiency rather than aggregate success: uniform CRITICAL-tier assignment increases total checkpoint I/O by saturating tasks that contribute little to cascade recovery, inflating cost per success while providing only marginal reliability gain over selective placement. Figure 6.8(b) shows the corresponding cost-per-success signature on a log scale: Variant A exhibits higher cost per success (\$0.033 vs \$0.018) than full FaaSspot because uniform checkpointing inflates I/O without proportionate recovery benefit; Variant B has higher cost per success (\$0.041) despite a lower scale-out spend because fewer runs complete; Variant C has the highest cost per success (\$0.082) because re-execution from task 1 on every failure wastes the most compute.

Compositional conclusion. The three mechanisms are mutually reinforcing: selective checkpointing concentrates protection where cascades originate; survival-aware scaling prevents retry-queue saturation; and lineage-based retry ensures that checkpoint state is consumed at recovery time rather than discarded. Weakening any one of the three degrades both reliability and cost efficiency, as each variant result shows; the full system outperforms every single-component configuration.

The ablation shows that all three mechanisms are needed; the next section asks the complementary question—whether the runtime adaptivity inside those mechanisms is needed, or whether a well-chosen fixed parameter setting would suffice.

6.8 Parameter Sensitivity Analysis

Sensitivity experiments fix individual parameters to specific values instead of letting the adaptive bandit determine them, testing what happens when the bandit’s choices are overridden. Tests run on wf-5 and wf-8, the two workflows with the highest preemption pressure; simpler workflows achieve near-100% success regardless of settings, making sensitivity differences uninformative there.

6.8.1 Budget Factor Sensitivity

Table 6.8: Budget-factor sensitivity on wf-5 and wf-8 (adaptive FaaSpt vs. fixed budget factors).

Configuration	wf-5			wf-8		
	Succ.	Lat.	Cost/s	Succ.	Lat.	Cost/s
FaaSpt (adaptive)	86.2%	240.7s	\$0.022	49.7%	392.4s	\$0.051
Fixed factor 0.60	73.0%	266.0s	\$0.019	39.7%	479.9s	\$0.045
Fixed factor 0.70	69.1%	278.9s	\$0.020	33.6%	530.3s	\$0.052
Fixed factor 1.00	67.5%	250.9s	\$0.022	41.9%	494.1s	\$0.044

All fixed-factor variants underperform adaptive FaaSpt on success rate and latency (Table 6.8). The best fixed variant for wf-5 (factor 0.60, 73.0%) trails the adaptive default by 13.2 pp. The adaptive system self-calibrates to the appropriate spending level for each context; any fixed cap is either too conservative or wastes budget on unnecessary scale-out.

6.8.2 KM Threshold Sensitivity

Table 6.9: KM-threshold sensitivity on wf-5 and wf-8 (adaptive FaaSpt vs. fixed KM survival thresholds).

Configuration	wf-5			wf-8		
	Succ.	Lat.	Cost/s	Succ.	Lat.	Cost/s
FaaSpt (adaptive)	86.2%	240.7s	\$0.022	49.7%	392.4s	\$0.051
Fixed KM 0.60	67.4%	346.5s	\$0.021	44.4%	404.6s	\$0.046
Fixed KM 0.70	75.2%	272.6s	\$0.020	33.9%	392.0s	\$0.055
Fixed KM 0.80	73.9%	261.8s	\$0.021	33.3%	493.2s	\$0.058

KM threshold sensitivity shows non-monotonic behavior (Table 6.9): the optimal fixed threshold differs by workflow structure (KM 0.70 best for wf-5, KM 0.60 best for wf-8), confirming that no single fixed threshold matches both regimes. Adaptive FaaSpt leads the best fixed variant by 11.0 pp on wf-5 and 5.3 pp on wf-8, validating the Thompson Sampling bandit’s context-dependent selection.

6.9 Summary of Results

Table 6.10 consolidates the key outcomes across all three primary metrics.

Table 6.10: Summary: FaaSot vs. baselines on all three primary metrics.

Method	Success	Latency	Cost/s	Spot-only?
FaaSot (FS)	89.0%	126s	\$0.018	Yes (all spot)
MScheduler	81.8%	334s	\$0.040	No (on-demand fallback)
Hourglass	76.2%	506s	\$0.047	No
Snape	58.6%	297s	\$0.050	No (mixed)
Burst-HADS	53.4%	290s	\$0.259	No

FaaSot leads on all three metrics simultaneously while operating under a stricter constraint (no on-demand fallback) than any baseline. The integrated design — HEFT-based criticality ranking, structural checkpoint classification, priority-ordered checkpoint recovery, and budget-gated Thompson Sampling scaling — achieves 89.03% success at \$0.018 cost per success and 126 s mean latency, outperforming all four baselines on every metric. Checkpoint efficiency (10.6 checkpoints per successful run vs. 39.9 for Hourglass) confirms that placement value dominates frequency. The ablation confirms all three mechanisms are necessary; the sensitivity analysis confirms that adaptive parameter selection outperforms any fixed threshold by 9–17 pp.

6.10 Discussion

6.10.1 Structural Vulnerability and the Checkpoint Placement Effect

The most striking result is the workflow-level variation in FaaSot’s reliability advantage. On wf-1 and wf-2 (simple topologies), most baselines also achieve near-100% completion and FaaSot’s margin is negligible. On wf-5 (LIGO, synchronization barrier), FaaSot reaches 86.2% while all four baselines fall below 41%, a margin of **+43 percentage points**. In wf-5, a single preemption at the Thınca synchronization node blocks both sub-pipeline branches; baselines that lack a checkpoint at this node must restart both branches from scratch. FaaSot’s Critical-tier checkpoint at Thınca converts a dual-pipeline restart into a single-branch replay.

This structural insight reveals that checkpoint *frequency* is not the operative variable. Hourglass employs 39.9 checkpoints per successful run ($3.8\times$ more than FaaSot’s 10.6) yet achieves only 76.22% success. Variant A (uniform checkpointing) inflates cost per success from \$0.018 to \$0.033 without commen-

surate reliability gain (86.44% vs 89.03%), because placement specificity—not raw checkpoint density—drives recovery value.

6.10.2 The Cost–Reliability Inversion

A common assumption is that reliability improvements require additional resource consumption. FaaS_{Spot} inverts this: cost per successful run decomposes into the cost of successful runs, failed runs (wasted compute), and recovery overhead. Baselines accumulate high cost-per-success because failed-run compute is wasted on full-workflow restarts. FaaS_{Spot} converts failures into partial sub-DAG replays: \$0.018 vs \$0.040 for MScheduler (2.2×) and \$0.259 for BurstHADS (14.4×). Reliability failure is more expensive than FaaS_{Spot}’s checkpointing and scaling overhead combined.

6.10.3 Latency Concentration and the Full-Restart Cost

Baselines exhibit bimodal latency distributions: runs that escape preemption complete quickly; runs that incur a full-workflow restart produce heavy right tails. FaaS_{Spot}’s distribution is concentrated and unimodal because Critical-tier checkpoints at fan-out boundaries bound recovery to a sub-DAG replay. At p₉₉, FaaS_{Spot} achieves 346 s while Hourglass reaches 2,199 s; cross-workflow variance is smallest for FaaS_{Spot} (± 42 s at p₅₀ vs. ± 143 s for Hourglass), reflecting predictable latency regardless of where preemptions land.

6.10.4 Anomalies and Boundary Cases

wf-4 (Epigenomics): an honest accounting. MScheduler’s success rate exceeds FaaS_{Spot}’s by 2.5 pp (77.1% vs 74.6%), inside the paired binomial half-width of ± 5.3 pp and therefore not statistically distinguishable; FaaS_{Spot} retains a 22% cost-per-success advantage. The most plausible explanation is that Epigenomics consists of several long, largely independent parallel branches converging at a single merge fan-in, and MScheduler’s reactive on-demand migration contains branch-level failures before they propagate to the barrier. We present this explanation as a post-hoc rationalization rather than a structural prediction: the topological signal that would distinguish wf-4 from wf-5 *a priori*—both have merge fan-ins—is not captured by the current criticality score, and the score does not provide a quantitative threshold below which structural checkpoint-and-replay loses to branch-level migration. A structural predictor for this regime (for example, branch independence ratio combined with branch length) is a natural extension and is noted as future work in Chapter 7 §7.4.3.

wf-8 (Montage): joint independent-pipeline ceiling. FaaS_{Spot} achieves approximately 50% success on wf-8, comparable to the best baseline. The structural cause is that three independent sub-pipelines must each avoid disruptive preemption for the workflow to succeed: under independent per-VM hazard h and equal sub-pipeline durations, the joint probability is approximately $(1 - h)^3$, which compresses any per-pipeline improvement into a smaller terminal gain. FaaS_{Spot}’s mechanisms still help on the recovery cost (cost-per-success on wf-8 is \$0.051 versus \$0.066 for MScheduler), but the success rate is bounded by infrastructure exposure that no fault-tolerance layer can fully escape without on-demand fallback. This is the regime where the central thesis claim—“pure-spot can match hybrid systems”—reaches its practical limit, and the result is reported as such rather than masked by aggregate statistics. The Limitations chapter §7.1 discusses the relationship between this structural ceiling and the unmodeled capacity-scarcity scenario.

6.10.5 Mechanism Interaction: Why Integration Matters

The ablation confirms that removing any single mechanism degrades *both* reliability and cost efficiency simultaneously. Variant A (uniform checkpointing) achieves 86.44% at \$0.033/success—lower reliability at nearly double the cost. Variant B (no adaptive scaling) degrades on high-preemption workflows as the retry queue grows faster than fixed-replica capacity drains. Variant C (no lineage-based retry) is most damaging at 50.05% and \$0.082/success: checkpoints are written but never consulted. Fault-tolerant spot execution is a coordination problem; solutions addressing any one dimension in isolation yield marginal gains at best.

CHAPTER 7

LIMITATIONS AND FUTURE WORK

Chapter 6 demonstrates that FaaS_{Spot} improves success rate, cost-per-success, and latency under spot-instance volatility. Nevertheless, these findings should be interpreted within the scope conditions of the current system design and evaluation protocol. This chapter formalizes those scope conditions, separates *system limitations* from *threats to validity*, and outlines the future research agenda they motivate.

7.1 System Scope and Core Assumptions

FaaS_{Spot} is intentionally formulated as a pure-spot controller with a closed loop around checkpoint selection, prioritized retry, and adaptive scaling. This design is appropriate for evaluating spot-only reliability engineering; however, it also introduces assumptions that can affect transferability to production environments.

7.1.1 Spot Capacity Availability Is Not Explicitly Modeled

The current control loop assumes that, when the policy issues a scale or recovery action, spot capacity can be reacquired with acceptable delay. In operational cloud markets, spot pools may become unavailable, significantly delayed, or rapidly repriced during regional demand perturbations (Son et al., 2024, Wu et al., 2024). Under such conditions, a policy that is optimal under *expected* capacity access may become suboptimal under realized capacity scarcity.

This limitation is most consequential for long DAGs with high interruption exposure. If replacement instances cannot be provisioned while retry pressure

increases, backlog growth may exceed the controller’s drain capacity. Accordingly, the reported results should be interpreted as performance under interruption volatility *with feasible spot reacquisition*, not as a guarantee under severe market scarcity.

Correlated zone-level preemption is not modeled. A related and equally consequential gap is that the emulator samples each VM’s lifetime independently from the SkyPilot CDF. Real spot environments exhibit *correlated* preemption: entire spot pools (an instance type within an AZ) can drain within minutes when regional demand spikes (Wu et al., 2024). Recent empirical evidence from Kim et al. (Kim et al., 2026) reinforces this: across 68 instance types and 15 regions on AWS and Azure, co-interruptions within the same AZ occur within three minutes in over 92% of cases, confirming that independent lifetime models substantially understate zone-level failure risk. Under correlated drain, the structural advantage of FaaS’s tiered checkpointing is unaffected (the topology has not changed) but the survival-aware scaler can be defeated: scaling out to “safer” instances does no good if every instance in the same pool is reclaimed simultaneously. The reported scaler ablation (Variant B) understates this risk because the emulator does not produce zone-correlated drain events. A scarcity-regime extension is the first item of future work in §7.4.1.

Control-plane preemptibility. As stated in Chapter 4 §4.1, the control plane (MySQL state store, Fission router, scheduler pods, and the three runtime controllers) is assumed to run on non-preemptible capacity. This is the convention in the literature, but it is not the only possible choice. A fully spot-resident control plane would need to handle its own preemption-driven leader election and durable state hand-off, neither of which is in scope for this thesis. The cost impact of the assumption is small (single-digit control-plane pods regardless of workflow scale), but the engineering implication—that “pure-spot” here means “pure-spot task plane”—should not be over-extrapolated.

The Montage (wf-8) result (approximately 50% success across all methods) does not exemplify this scarcity scenario directly. wf-8’s difficulty is structural: three fully independent parallel sub-pipelines must each avoid disruptive preemption, and the joint per-pipeline survival probability multiplies down to roughly $(1 - h)^3$ regardless of how the time axis is scaled (Section 6.10.4). A true scarcity scenario would manifest differently—as rising retry-queue backlog and growing recovery latency even for structurally simple workflows—which is not observed in the evaluation. The scarcity regime nevertheless remains a real production concern that the current controller does not explicitly address.

7.1.2 Survival Modeling Is Built from a Single Availability-Zone Context

The current risk controller relies on spot-lifetime modeling calibrated from a specific availability-zone context. Although this choice improves experimental control, it constrains direct generalization. Spot behavior varies across region, zone, accelerator type, time horizon, and broader market dynamics (Son et al., 2024, Wu et al., 2024, Lee et al., 2022).

Consequently, absolute gains in success rate and cost efficiency may shift under deployment conditions that differ from the calibration context. The qualitative mechanism—risk-aware coordination of checkpointing and scaling—is expected to remain useful, but threshold values and operating points are environment-dependent. In effect, policy *structure* is more transferable than exact numeric *tuning*.

This has a practical implication for deployment: FaaSot’s checkpoint planner and bandit require recalibration when moved to a new region, zone, or instance family. Without recalibration, the survival model may mismatch actual lifetime distributions, and the bandit’s prior may require additional warm-up samples to converge. Section 7.3 discusses a self-calibrating warm-up procedure as a mitigation.

A second concern is *KM-curve drift*: the current implementation fits the survival curve once from the SkyPilot dataset and queries it at runtime, but does not re-fit as new lifetime observations accumulate. In a production deployment where spot dynamics shift over time-of-day or with provider capacity expansions, an unupdated curve would yield biased risk scores. Online refitting (an exponentially-weighted KM estimator, or a sliding-window non-parametric maximum-likelihood update) is the natural mitigation and is left to future work alongside the multi-AZ extension in §7.4.2.

7.2 Threats to Validity

In addition to system-scope assumptions, methodological threats influence how broadly the conclusions can be generalized.

7.2.1 Internal Validity

Internal validity concerns whether observed improvements are causally attributable to the proposed mechanism rather than to hidden confounders. The ablation and sensitivity analyses in Chapter 6 (§6.7–§6.8) reduce this risk by isolating

module contributions and sweeping key parameters. However, residual confounding remains possible because arrival dynamics, interruption clustering, and queue-state coupling can alter controller behavior in ways that are difficult to exhaustively enumerate.

Policy tuning is also necessarily finite. Although key settings were stress-tested, the feasible parameter space exceeds practical experiment budgets. Therefore, the reported gains indicate robust performance within the evaluated regime, not global optimality across all admissible control configurations.

A related concern is that the three sensitivity parameters—budget factor, KM survival threshold, and retry priority weight—are swept independently rather than jointly. The full FaaS_{Spot} bandit co-adapts all three simultaneously; the independent sweeps characterize each parameter’s marginal contribution under partial adaptation. Joint sensitivity analysis across all three parameters is a methodological extension that would provide stronger internal validity evidence; we report it as a known gap rather than ignore it.

Delayed-reward credit assignment. The Adaptive Scaler’s contextual bandit treats the reward $r_t = \Delta\sigma_t/\text{cost}_t$ as the consequence of the single action selected at the start of observation window t . In practice the reward stream is delayed (the success-rate change reflects actions taken across several prior windows) and non-stationary (correlated preemption bursts alter the underlying hazard mid-window). The bandit is therefore a stationary-instantaneous approximation of a non-stationary delayed-reward process. The KM-tier safety floor and the $[-1, 1]$ reward clip limit the damage of any single mis-attributed update, but a worst-case attribution error could shift the reported success rate by up to roughly 1–2 pp on the high-preemption workflows. A principled treatment via off-policy delayed-reward bandit estimation (Duan et al., 2024) is left to future work.

Hyperparameter overfitting risk. The bandit cold-start parameters and KM-tier thresholds were tuned on wf-1 and wf-2 during development and then held fixed for all evaluation workflows including the hardest ones (wf-5, wf-8). This protocol mirrors the standard cross-validation discipline of holding tuning data out from evaluation, but the tuning set is small (2 simple workflows) and shares the structural family of the evaluation suite (scientific DAGs); the residual risk of fortuitous tuning is non-zero. Appendix Table A.7 reports every tuned parameter so that follow-up work can re-tune on a different held-out subset.

7.2.2 Construct Validity

Construct validity concerns whether selected metrics adequately represent system objectives. The evaluation focuses on success rate, total cost, cost-per-success, and latency (including tail behavior shown in the violin plots) because these metrics are operationally salient and aligned with the thesis objective. Nonetheless, they do not fully capture all production-quality attributes.

Important dimensions such as cross-workflow fairness, jitter under burst arrivals, storage durability trade-offs, and long-horizon operational complexity are only partially reflected. Similarly, checkpoint count is informative but remains an indirect proxy for storage-system pressure. Thus, conclusions are strongest for reliability–cost–latency coordination, while broader operational quality remains partially measured.

The cost reporting convention (all experiments priced uniformly at spot rates, including on-demand fallback invocations by baselines) is another construct-validity consideration. This convention is conservative for FaaS_{Spot}: pricing on-demand capacity at spot rates understates what baselines would cost in real deployments, since on-demand is typically $3\text{--}5\times$ more expensive than spot. The reported cost advantage for FaaS_{Spot} is therefore a lower bound on the true economic benefit under realistic billing. However, this choice also means that absolute cost-per-success figures should not be taken as production budget estimates without adjusting for actual instance pricing.

7.2.3 External Validity

External validity addresses generalization beyond the evaluated corpus and operating conditions. The workflow suite covers diverse DAG structures and difficulty profiles, including failure-prone cases (wf-5, wf-8), but no finite benchmark can represent all production graph topologies, all resource mixes, and all cloud-market regimes.

The eight workflows are drawn exclusively from the scientific computing and HPC domains. Generalization to other workload classes—event-driven pipelines, ML training workflows, or streaming analytics—is not directly supported by the current evidence. These workload classes may exhibit different DAG structures, task duration distributions, and checkpoint-utility profiles that alter the effectiveness of FaaS_{Spot}’s structural classifier.

Accordingly, the principal claim should be interpreted as follows: FaaS_{Spot} consistently improves outcomes across the tested workload family and modeled market conditions, and its control architecture appears structurally sound for related settings. It should not be interpreted as a universal guarantee that identi-

cal quantitative margins will hold across all workload classes, regions, or future market epochs without recalibration.

7.3 Practical Implications for Deployment

These limitations do not negate the central contribution; they define the boundary between *validated capability* and *deployment hardening*. For practitioners, the key implication is that FaaS_{Spot} already provides a strong spot-only reliability controller, but production deployment should include explicit safeguards for scarcity, startup delay, and transfer-aware optimization.

A practical rollout strategy is to preserve the current closed-loop strengths while introducing environment-aware controls:

- **Capacity-aware admission and scaling:** integrate risk signals with real-time spot availability and feasibility checks before scale-up commitments (Son et al., 2024, Wu et al., 2024). A scale action that cannot be fulfilled due to capacity scarcity should be demoted or queued rather than silently failing.
- **Startup-latency-aware recovery planning:** include warm-up delay estimates in retry prioritization and deadline-aware policies (Poolal et al., 2016, Monge et al., 2020). Recovery actions dispatched to cold instances should carry a startup delay penalty in the retry priority score.
- **Transfer-aware checkpoint valuation:** extend checkpoint utility scoring to include data-transfer and storage-path costs (Aupy et al., 2016, Kail et al., 2016). The dynamic-programming planner should discount checkpoint value for tasks whose state is large relative to recomputation cost.
- **Policy portability workflow:** recalibrate survival and budget thresholds per region/zone/instance family prior to full rollout (Son et al., 2024, Lee et al., 2022). A brief warm-up phase that collects local lifetime samples before production submission can bootstrap the survival model and reduce bandit warm-up cost.

From an engineering perspective, the current results can be interpreted as a high-confidence operating regime for pure-spot execution under modeled conditions. Under more severe scarcity or heavier data-movement constraints, some degradation in absolute metrics should be expected unless the above safeguards are incorporated.

These limitations motivate a concrete research agenda organized along four axes.

7.4 Future Work

7.4.1 Extreme Failure Regimes and Spot-Unavailable Scenarios

The most immediate extension addresses regimes in which spot capacity is intermittently unavailable at scale. Rather than interpreting repeated reacquisition failures as independent transients, the control loop should infer transition into a scarcity regime and switch to a dedicated policy profile that prioritizes preservation of high-value progress, restricts checkpointing to high-criticality states, and suppresses speculative retries. Detection can be implemented as a sequential probability ratio test on the reacquisition delay distribution.

7.4.2 Multi-AZ, Multi-Region, and Multi-Cloud Spot Scheduling

Spot dynamics are heterogeneous across zones and regions (Son et al., 2024, Wu et al., 2024, Lee et al., 2022); a next-generation controller should extend the KM estimator per availability zone and augment the bandit’s context vector with zone identity. Checkpoint planning becomes a joint placement-and-routing problem: a checkpoint is valuable only if its restore path is timely and cost-efficient under current domain conditions. Multi-cloud orchestration further requires a thin abstraction layer normalizing interruption semantics and billing units across AWS, GCP, and Azure.

7.4.3 Adaptive Workflow Restructuring

The wf-4 (Epigenomics) result motivates adapting workflow graph structure itself when runtime evidence indicates elevated structural risk. Runtime restructuring may include splitting high-risk long tasks, merging low-value fine-grained stages, or reordering non-critical branches to reduce expected recomputation impact. Transformation rules must be provably output-preserving to avoid semantic correctness violations.

7.4.4 Toward Production-Grade Operation

Several engineering-focused extensions improve practical adoption: (1) **auto-calibration** via a small batch of probe workflows that bootstrap the KM estimator and initialize the bandit prior from local lifetime observations; (2) **operator-facing explainability** through structured decision artifacts (risk snapshots,

budget-state traces, counterfactual summaries); (3) **expanded workload coverage** to ML training pipelines, event-driven workflows, and streaming analytics; and (4) **reproducible evaluation packs** publishing the trace-driven emulator, workflow DAG definitions, and baseline implementations as an open benchmark suite.

CHAPTER 8

RELATED WORK

FaaS_{Spot} sits at the intersection of two research areas: fault-tolerant execution on spot (preemptible) cloud instances and serverless DAG workflow execution. Prior work in each area has advanced individual mechanisms, but no existing system addresses the combination that FaaS_{Spot} targets: a pure-spot serverless execution platform that exploits static workflow topology to drive coordinated checkpointing, retry, and scaling without on-demand fallback. This chapter surveys both areas, maps out the design space, and concludes with a comparison table that precisely locates FaaS_{Spot} relative to related systems.

8.1 Survey of Related Work

Checkpoint theory and spot modeling. Young (Young, 1974) and Daly (Daly, 2006) establish optimal checkpoint intervals for sequential jobs; Aupy (Aupy et al., 2016) and Tomar (Tomar et al., 2023) extend placement to DAG workflows and spot environments. FaaS_{Spot} departs from this line by integrating checkpointing with survival-aware scaling and lineage-based retry as a coordinated unit. Spot lifetime characterization work—SpotLake (Lee et al., 2022), SpotCheck (Sharma et al., 2015), SciSpot (Kadupitiya et al., 2022), SkyPilot NSDI’24 (Wu et al., 2024), and the foundational pricing study by Ben-Yehuda et al. (Ben-Yehuda et al., 2013)—shows that lifetimes are non-exponential and heterogeneous across zones, motivating FaaS_{Spot}’s Kaplan-Meier survival estimator (Kaplan and Meier, 1958). Cloud-index tracking (Shastri and Irwin, 2018) further demonstrates that effective spot use requires capturing this heterogeneity in the cost model itself. Recent measurement work by Kim et al. (Kim et al., 2026) introduces a near-zero-cost availability probing method and documents that co-interruptions within the same availability zone occur within three

minutes in over 92% of observed events, empirically corroborating FaaS_{Spot}'s zone-level scarcity concern identified in §7.1.

Hybrid spot/on-demand systems. Hourglass (Joaquim et al., 2019), Snape (F. Yang et al., 2023), MScheduler (Portella et al., 2023), and Burst-HADS (Teylo et al., 2023)—the four evaluation baselines—achieve higher completion rates by maintaining on-demand capacity as a backstop at the cost of on-demand pricing. Tributary (Harlap et al., 2018), SpotOn (Subramanya et al., 2015), and deadline-aware spot schedulers (Tarafdar et al., 2022) follow the same pattern. Related hybrid-scheduling work includes ensemble-workflow scheduling on IaaS mixes (Chen et al., 2024), planning under uncertainty on unreliable cloud resources (Oikonomou et al., 2020), microservice hybrid-resource scheduling with adaptive learning (Gu et al., 2021), cost-deadline constrained robust scheduling (Nag et al., 2024), and real-time volatile-resource workflow scheduling (Wang et al., 2024). Online VM-failure prediction (Saxena and Singh, 2022) provides an alternative reliability signal but requires per-instance training and does not exploit DAG structure. AutoBoT (Varshney and Simmhan, 2019) provides a resilient and cost-effective scheduler for bag-of-tasks workloads on spot VMs using checkpoint-based restart. ExoSphere (Ambati et al., 2020) provides a schedulable parallel-virtualization substrate but still relies on a non-preemptible scheduling layer for reliability. Multi-level checkpointing for distributed deep learning on spot clusters (Cho et al., 2024) reduces checkpoint overhead through a hierarchical approach, though it does not exploit DAG topology to guide checkpoint placement. FaaS_{Spot} achieves superior results through structural mechanisms alone, with no on-demand fallback.

DAG-structure-aware fault tolerance. TR-Spark (Yan et al., 2016) performs lineage-aware checkpointing in Spark's stage model but assumes stable resources for high-criticality stages and cannot be applied to fine-grained serverless execution. Pado (Y. Yang et al., 2017) exploits topology to migrate critical tasks to reserved nodes; FaaS_{Spot} replaces migration with checkpoint-and-retry since no reserved nodes exist. SpotVerse (Son et al., 2024) performs cross-region placement for bioinformatics workflows but does not direct task-level checkpoint or retry decisions, and SkyNomad (Li et al., 2026) similarly exploits multi-region spot capacity at the job level rather than at internal DAG boundaries.

Transient resources for analytics, ML serving, and training. Flint (Sharma et al., 2016), Portfolio (Sharma et al., 2017), Bamboo (Thorpe et al., 2023), and Parcae (Duan et al., 2024) target transient resources for analytics or ML training, while SpotTune (Servia-Rodríguez et al., 2020) explores cost-efficient hyper-parameter tuning on the same hardware class. SpotServe (Miao et al., 2024) and SkyServe (Mao et al., 2025) extend the model to online LLM

serving on preemptible instances, and GFS (Duan et al., 2025) introduces preemption-aware scheduling for GPU spot clusters. SpotWeb (Sharma et al., 2019) and the mixed-workload optimization of Singh et al. (Singh et al., 2017) extend transient-resource techniques to latency-sensitive and mixed workloads. Complementary cost-aware training work such as FedCostAware (Sinha et al., 2025) applies cloud spot-market awareness to federated learning by scheduling client training rounds against instance-price dynamics; its cost-aware objective is aligned with FaaSpt’s cost-per-success formulation, though it targets a different workload class (federated aggregation) and does not model DAG-level cascade failures. Bamboo and Parcae deserve a sharper positioning because they share two ideas with FaaSpt: lineage-based recovery and predictive provisioning on preemptible capacity. Bamboo introduces *redundant computation* across pipeline stages to recover from ML-training preemptions without checkpointing; Parcae extends this with a predictive component that anticipates preemption and reconfigures the pipeline ahead of time. Both treat the workload as a pipeline of homogeneous training iterations, and the recovery unit is the iteration boundary. FaaSpt differs in three concrete respects. *First*, the recovery unit is a structurally classified DAG position, not a homogeneous iteration boundary; the tier (CRITICAL/MEDIUM/LOW) is derived from topology, not from training-loop semantics. *Second*, the predictive signal is statistical (Kaplan–Meier survival) rather than utilization- or loss-curve-based, which is the only signal available for general scientific DAGs whose tasks do not have a single dominant convergence metric. *Third*, FaaSpt’s coordination is offline-then-online: a single classification pass drives three online mechanisms, whereas Bamboo and Parcae continuously re-plan the redundant-compute schedule. The overlap with these systems is real and should not be understated; the contribution of FaaSpt is to bring this class of ideas to topology-aware DAG fault tolerance.

Serverless workflows and platforms. Wukong (Carver et al., 2020) and its companion DAG-engine study (Carver et al., 2019), Speedo (Daw et al., 2021), Mashup (Roy, Patel, Gadepally, and Tiwari, 2022), DayDream (Roy, Patel, and Tiwari, 2022), StarShip (Basu Roy and Tiwari, 2024), HyperFlow (Malawski et al., 2020), Pronghorn (Kohli et al., 2024), Nightcore (Jia and Witchel, 2021), Trident (Zhu et al., 2025), and Bilal et al. (Bilal et al., 2023) optimize dispatch, locality, or resource sizing on reliable infrastructure without addressing spot preemption. ORION (Mahgoub, Barsallo Yi, Shankar, Elnikety, et al., 2022) and WiseFuse (Mahgoub, Barsallo Yi, Shankar, Minocha, et al., 2022) optimize serverless DAG execution through DAG-aware resource sizing and workload transformation using production traces. Netherite (Burckhardt et al., 2022) demonstrates that group-commit state persistence achieves order-

of-magnitude throughput improvements, motivating FaaS_{Spot}’s choice to concentrate checkpoint I/O at structurally critical positions rather than every task. Jolteon (Zhang et al., 2024) configures resource provisioning via a stochastic performance model that bounds tail latency under function variability; Metis (Tang et al., 2025) introduces a non-clairvoyant, workflow-aware OS scheduler that treats serverless workflows as first-class scheduling entities (SoCC 2025). GridGreen (Samanta et al., 2025) integrates serverless computing into HPC environments for hybrid execution and sustainability. SeBS-Flow (Schmid et al., 2025) provides the first platform-agnostic serverless workflow benchmarking suite, enabling consistent evaluation across platforms. Cost-of-serverless analyses (Liu and Niu, 2024) and scheduling algorithms for serverless (Majewski et al., 2021) further inform FaaS_{Spot}’s cost model. Speedo specifically targets serverless DAG dispatch ordering and Nightcore targets sub-millisecond function chains; both are orthogonal to and composable with FaaS_{Spot}’s fault-tolerance layer, since they assume a non-preemptible substrate and FaaS_{Spot} provides one (through tiered checkpoint-and-replay) on top of preemptible hardware.

Workflow management frameworks. Pegasus (Deelman et al., 2015, 2019) and Ray (Moritz et al., 2018) represent the broader workflow-management context in which FaaS_{Spot}’s contributions sit. The Adaptive Scaler’s bandit controller is grounded in Thompson Sampling theory (Agrawal and Goyal, 2013), extended here with a Kaplan–Meier survival safety floor that constrains the action space before the bandit optimizes within it.

8.2 Comparison and Positioning

Table 2.1 (Chapter 2) maps all surveyed systems along six design dimensions. Three observations follow. First, no prior system simultaneously occupies the spot-only + DAG-aware cell: TR-Spark and Pado exploit DAG structure but depend on non-spot resources for protection; transient-resource systems (Flint, Bamboo, Portfolio) operate on spot or transient resources but do not exploit workflow topology. FaaS_{Spot} occupies this previously empty cell.

Second, partial sub-DAG replay (PR recovery) requires structural awareness: to resume from a fan-out boundary checkpoint rather than from the workflow origin, the system must know where fan-out boundaries are. Systems with full-workflow restart (FW) cannot improve recovery latency by structural checkpointing. The ablation study (Chapter 6, §6.7) quantifies this cost directly: removing lineage-based replay drops aggregate success from 89.03% to 50.05%.

Third, survival-aware scaling—proactively provisioning capacity before a preemption occurs—is unique to FaaS_{Spot} in this comparison. Most systems scale reactively, responding to a preemption event after it has already disrupted progress. Proactive scaling under survival risk reduces recovery queueing delay, which the latency analysis in Chapter 6 shows is a primary contributor to the latency advantage on complex workflows.

CHAPTER 9

CONCLUSION

Cloud spot instances provide substantial cost advantages, but interruption volatility makes reliable workflow execution challenging for dependency-rich DAGs. This thesis addressed whether high reliability and cost efficiency can be sustained on *pure-spot* infrastructure without on-demand fallback. The central answer is yes—but only when offline structure-aware planning is tightly coupled with runtime adaptive control.

The headline measurements are reported throughout Chapter 6: FaaS_{Spot} leads on success rate, cost per success, and mean and tail latency across the eight-workflow suite, with the widest margin on structurally complex workflows (a +43 pp success-rate advantage on wf-5 LIGO). The ablation localizes the reliability contribution to lineage-based retry, the cost contribution to selective structural checkpointing, and the high-preemption-regime contribution to survival-aware scaling; none of the three mechanisms is sufficient on its own. Two boundary cases (wf-4 Epigenomics, where reactive on-demand migration is competitive on success rate; wf-8 Montage, where the joint independent-sub-pipeline survival probability caps any single-pipeline gain) mark the practical limits of the current design and are reported as such rather than smoothed over by aggregate statistics. The full evaluation, including a real-cloud validation pass under uncompressed wall-clock time, is identified in Chapter 7 as the principal external-validity threat and the most urgent follow-up work.

9.1 Final Perspective

The central takeaway is that cost-efficient reliability on volatile cloud resources does not emerge from checkpointing alone, scaling alone, or prediction alone. It emerges when planning, recovery, and resource control are intentionally coupled around workflow structure and runtime risk. Future cloud workflow sys-

tems should be designed as integrated control systems—explicitly balancing survivability, cost, and latency under uncertainty—rather than as collections of loosely connected single-objective mechanisms.

APPENDIX A

SUPPLEMENTARY EVALUATION AND REPRODUCIBILITY DETAILS

This appendix provides compact supporting material for the main results in Chapter 5 (Implementation) and Chapter 6 (Evaluation). The goal is to make experimental choices, measurement definitions, and headline outcomes easy to audit without interrupting the flow of the primary narrative.

All figures in this appendix are derived from the same eight-workflow campaign used throughout the main text: 500 invocations per workflow under matched trace-replay conditions, $60\times$ time compression, SkyPilot Wu et al., 2024 p3.2x1large lifetime CDFs sampled in $[p_{10}, p_{90}]$ for preemption modeling, and SpotLake Lee et al., 2022 m3.2x1large spot prices applied uniformly to all methods for cost accounting (the two-instance-type convention is justified in Chapter 5 §5.4). Numbers in this appendix are consistent with Chapter 6; any discrepancy with earlier draft snapshots is a typographical error and should be reported.

A.1 Reproducibility Checklist

All experiments follow a trace-driven protocol designed for controlled comparison across methods. The following controls are held constant unless explicitly varied in a sensitivity analysis:

- identical workflow inputs and submission profiles across methods (Poisson arrivals at $\lambda = 1/s$, $N = 500$ submissions per workflow),

- identical spot interruption and price trace sources for each paired trial (one CDF per AWS instance type and AZ, with per-pod lifetimes drawn from $[p_{10}, p_{90}]$ at pod creation),
- identical emulator tick policy and cost-accounting logic ($\Delta_{\text{emu}} = 60\Delta_{\text{real}}$, $h_{\text{emu}} = \Delta_{\text{real}}/60$ hours per billing tick),
- identical runtime environment (Kubernetes, Fission, MySQL polling at 5 s, 2 s for the two longest workflows),
- identical baseline evaluation horizon and completion criteria (terminal state: COMPLETE or ABANDONED after exhausting per-UID retry budget).

This ensures that measured differences primarily reflect controller behavior rather than differences in workload sampling or infrastructure conditions. The known limitations of this protocol (single AZ / instance type, lifetime-tail truncation, real-time control plane vs. compressed emulated time) are discussed in Chapter 7 §7.1 and §7.2.

A.2 Workflow and Baseline Inventory

Table A.1 summarizes the eight-workflow evaluation suite used in the main text. It spans narrow vs. wide DAGs, short vs. long critical paths, and varied fan-in/fan-out patterns so that resilience claims are not tied to one graph family.

Table A.1: Workflow inventory used across the evaluation suite (consistent with Chapter 6 Table 6.2).

Workflow	Type	Task count	Role in evaluation
wf-1	Simple Pipeline (synthetic)	5	Low-parallelism sequential baseline; sanity check.
wf-2	Fork-Join (synthetic)	7	Single-barrier aggregation behavior.
wf-3	1000 Genome	11	Cross-branch data dependency.
wf-4	Epigenomics	31	Long parallel-pipeline merge fan-in.
wf-5	LIGO	36	Dual sub-pipeline with Thinc synchronization barrier.
wf-6	SRASearch	15	Dual fan-out with nested fan-in.
wf-7	CyberShake	42	Multi-root cascade propagation to single sink.
wf-8	Montage	46	Triple parallel sub-pipelines with sequential post-processing.

Table A.2 lists the compared methods and the role each plays in the comparison.

Table A.2: Baseline inventory and comparison role.

Method	Category	Comparison purpose
FaaSpt (FS)	Proposed	Integrated pure-spot closed-loop controller.
MScheduler Portella et al., 2023	SOTA	Spot-first with reactive on-demand migration.
Hourglass Joaquim et al., 2019	SOTA	Deadline-pacing transient-resource scheduler.
Snape F. Yang et al., 2023	SOTA	Mixed-instance reliability-target scheduler.
Burst-HADS Teylo et al., 2023	SOTA	Bag-of-task spot-burst scheduler (workload-mismatch reference).
FaaSpt-A (no selective ckpt)	Ablation	Uniform CRITICAL-tier checkpointing.
FaaSpt-B (no scaler)	Ablation	Fixed replica counts from offline plan.
FaaSpt-C (no lineage retry)	Ablation	Full-workflow restart on every failure.

A.3 Key Aggregate Results Snapshot

For quick reference, Table A.3 consolidates the main aggregate outcomes reported in Chapter 6.

Table A.3: Aggregate performance snapshot across all eight workflows (mirrors Tables 6.3, 6.4, and 6.5).

Method	Success rate (%)	Cost/success (\$)	Avg. latency (s)
FaaSpt (FS)	89.03	0.018	126
MScheduler	81.78	0.040	334
Hourglass	76.22	0.047	506
Snape	58.56	0.050	297
Burst-HADS	53.43	0.259	290
FaaSpt-A (uni-form ckpt)	86.44	0.033	—
FaaSpt-B (no scaler)	86.74	0.041	—
FaaSpt-C (no lineage)	50.05	0.082	—

Table A.4 summarizes the interruption-heavy subset (wf-5 LIGO, wf-7 CyberShake, wf-8 Montage), where resilience quality is most visibly differentiated.

Table A.4: Hard-workflow summary on the three workflows with the widest cascade exposure.

Quantity	Value
FaaSpt success rate (hard subset)	78.6%
FaaSpt cost per successful run (hard subset)	\$0.040
FaaSpt average latency (hard subset)	314 s
Success gain vs. best SOTA on wf-5	+43.0 pp (86.2% vs. 41%)
Latency reduction vs. best SOTA on wf-5	4.2×
Cost/success reduction vs. best SOTA on wf-5	~2.0×

A.4 Per-Workflow Result Tables

Table A.5 reports per-workflow success rate (%) for the proposed system and the four SOTA baselines. Per-workflow standard errors are reported with the result rather than aggregated, so that wf-4 (where the proposed system trails MScheduler within standard error) is visibly identifiable.

Table A.5: Per-workflow success rate (%). Best per row in bold; ties within ± 2.6 pp (the binomial-proportion 95% half-width at $\hat{p} = 0.90$, $n = 500$) marked with †.

WF	FaaS _{Spot}	MS	HG	SN	BH
wf-1	99.8	99.6†	95.4	88.0	90.6
wf-2	100.0 †	99.6†	88.0	71.0	84.8
wf-3	100.0	93.7	89.4	71.4	54.6
wf-4	74.6	77.1 †	71.2	41.8	27.8
wf-5	86.2	41.0	35.4	28.8	16.6
wf-6	100.0	83.4	61.6	44.4	36.0
wf-7	99.7	98.2	87.6	74.9	57.2
wf-8	49.7	61.7	75.6	48.2	60.0

Table A.6 reports per-workflow cost per successful run.

Table A.6: Per-workflow cost per successful run (USD; lower is better). Best per row in bold.

WF	FaaS _{Spot}	MS	HG	SN	BH
wf-1	0.004	0.026	0.022	0.029	0.084
wf-2	0.006	0.018	0.029	0.025	0.092
wf-3	0.009	0.024	0.031	0.033	0.140
wf-4	0.055	0.067	0.078	0.084	0.530
wf-5	0.022	0.067	0.082	0.071	0.310
wf-6	0.010	0.046	0.056	0.061	0.290
wf-7	0.011	0.036	0.045	0.041	0.205
wf-8	0.051	0.066	0.034	0.058	0.420

A.5 Metric Definitions Used in Analysis

The core metrics are defined as follows:

- **Workflow success rate**

$$\text{SuccessRate} = \frac{N_{\text{completed}}}{N_{\text{submitted}}},$$

where a submission is `COMPLETED` if and only if the workflow reaches its sink task within the per-UID retry budget.

- **Execution cost**

$$\text{Cost} = \sum_{k \in \mathcal{R}} p_k \cdot t_k,$$

where p_k is the SpotLake unit price for resource k at the billing tick and t_k is active time (in h_{emu} units). The sum runs over all pods active during the experiment, including pods used for failed runs.

- **Cost per successful run**

$$\text{CostPerSuccess} = \frac{\text{TotalCost}}{N_{\text{completed}}}.$$

- **Latency** End-to-end wall-clock real-time completion time per workflow run, measured from invocation to terminal completion at the Fission router. Reported as mean, p_{50} , p_{95} , and p_{99} over successful runs. *Latency is in real-time wall-clock seconds, not emulated time*, because the Adaptive Scaler, Retry Manager, and detection polling all run in real time.
- **Checkpoints per success**

$$\text{Ckpts/Success} = \frac{N_{\text{checkpoints written}}}{N_{\text{completed}}},$$

a measure of how productively the I/O budget is spent.

A.6 Bandit and Survival Hyperparameters

All bandit and KM-tier hyperparameters used in evaluation are summarized below. Cold-start values were chosen on the two simplest workflows (wf-1 and wf-2) and held fixed for the remaining workflows; the budget thresholds and KM cut points are derived from inspection of the SkyPilot CDF and are reported here for full reproducibility (see Chapter 4 §4.4 for the methodological justification).

Table A.7: Bandit and KM hyperparameters used in all evaluations.

Parameter	Value	Role
KM <code>SAFE</code> threshold	0.55	Suppresses bandit
KM <code>PROACTIVE</code> threshold	0.50	Light scale-out
KM <code>WARNING</code> threshold	0.40	Moderate scale-out
BLR regularization λ_{reg}	1.0	Prior precision matrix
Cold-start ε	0.5	Exploration on {idle, conservative}
Cold-start ticks T_{cold}	10	Bandit cold-start length (complex DAGs)
Simple-DAG cold start	30 real min	Effectively passive under $60\times$ compression
Aggressive-action cooldown	120 s	Prevents high-cost decision repetition
Reward clip	$[-1, 1]$	Outlier mitigation
f_{cap} (per-task scaling cap)	$2.5\times$	Caps offline amplification
Pods per emulated machine	3	Models correlated VM-level preemption
Trace lifetime sampling	$\text{Unif}(p_{10}, p_{90})$	Tail-truncated by design

A.7 Notes for Future Extensions

This appendix focuses on compact reproducibility and result snapshots. Future versions of the evaluation pack can include expanded per-workflow raw tables, complete sensitivity-sweep matrices, additional failure-trace diagnostics, and a live-cloud validation pass on AWS `p3.2xlarge` spot instances under real (uncompressed) wall-clock time — the validation pass that Chapter 7 identifies as the principal external-validity threat to the current results.

BIBLIOGRAPHY

- [1] Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457–481. <https://doi.org/10.1080/01621459.1958.10501452>
- [2] Young, J. W. (1974). A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9), 530–531. <https://doi.org/10.1145/361147.361115>
- [3] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2), 235–256. <https://doi.org/10.1023/A:1013689704352>
- [4] Topcuoglu, H., Hariri, S., & Wu, M.-Y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260–274. <https://doi.org/10.1109/71.993206>
- [5] Daly, J. T. (2006). A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3), 303–312. <https://doi.org/10.1016/j.future.2004.11.016>
- [6] Agrawal, S., & Goyal, N. (2013). Thompson Sampling for Contextual Bandits with Linear Payoffs. *International Conference on Machine Learning (ICML)*.
- [7] Ben-Yehuda, O. A., Ben-Yehuda, M., Schuster, A., & Tsafrir, D. (2013). Deconstructing Amazon EC2 spot instance pricing. *ACM Transactions on Economics and Computation*, 1(3), 1–20. <https://doi.org/10.1145/2509413.2509416>
- [8] Zheng-Bradley, X., & Flicek, P. (2013). The 1000 genomes project: Paradigm for a data management plan in a large genomic study. *2013 International Conference on Parallel and Distributed Systems (ICPADS)*, 636–641. <https://doi.org/10.1109/ICPADS.2013.114>
- [9] Pandey, S., Sammut, L., Calheiros, R. N., Melatos, A., & Buyya, R. (2014). Scalable deployment of a LIGO physics application on public clouds: Workflow engine and resource provisioning techniques. In *Cloud com-*

- puting for data-intensive applications* (pp. 3–25). Springer. https://doi.org/10.1007/978-1-4939-1905-5_1
- [10] Poola, D., Ramamohanarao, K., & Buyya, R. (2014). Fault-tolerant workflow scheduling using spot instances on clouds. *Procedia Computer Science*, 29, 523–533. <https://doi.org/https://doi.org/10.1016/j.procs.2014.05.047>
 - [11] Auton, A., Abecasis, G. R., Altshuler, D. M., Durbin, R. M., Bentley, D. R., Chakravarti, A., Clark, A. G., Donnelly, P., Eichler, E. E., Flicek, P., et al. (2015). A global reference for human genetic variation. *Nature*, 526(7571), 68–74. <https://doi.org/10.1038/nature15393>
 - [12] Deelman, E., Vahi, K., Juve, G., Rynge, M., Callaghan, S., Maechling, P. J., Mayani, R., Chen, W., Ferreira da Silva, R., Livny, M., & Wenger, K. (2015). Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46, 17–35. <https://doi.org/10.1016/j.future.2014.10.008>
 - [13] Sharma, P., Lee, S., Guo, T., Irwin, D., & Shenoy, P. (2015). SpotCheck: Designing a derivative IaaS cloud on the spot market. *Proceedings of the Tenth European Conference on Computer Systems*. <https://doi.org/10.1145/2741948.2741953>
 - [14] Subramanya, S., Guo, T., Sharma, P., Irwin, D., & Shenoy, P. (2015). SpotOn: A batch computing service for the spot market. *Proceedings of the Sixth ACM Symposium on Cloud Computing*, 329–341. <https://doi.org/10.1145/2806777.2806851>
 - [15] Aupy, G., Benoit, A., Casanova, H., & Robert, Y. (2016). Checkpointing strategies for scheduling computational workflows. *International Journal of Networking and Computing*, 6(1), 2–26. https://doi.org/10.15803/ijnc.6.1_2
 - [16] Kail, E., Kacsuk, P., & Kozlovsky, M. (2016). A novel adaptive checkpointing method based on information obtained from workflow structure. *Computer Science*, 17(3), 387. <https://doi.org/10.7494/csci.2016.17.3.387>
 - [17] Poola, D., Ramamohanarao, K., & Buyya, R. (2016). Enhancing reliability of workflow execution using task replication and spot instances. *ACM Trans. Auton. Adapt. Syst.*, 10(4). <https://doi.org/10.1145/2815624>
 - [18] Sharma, P., Guo, T., He, X., Irwin, D., & Shenoy, P. (2016). Flint: Batch-interactive data-intensive processing on transient servers. *Proceedings of the Eleventh European Conference on Computer Systems*. <https://doi.org/10.1145/2901318.2901319>

- [19] Yan, Y., Gao, Y., Chen, Y., Guo, Z., Chen, B., & Moscibroda, T. (2016). TR-Spark: Transient computing for big data analytics. *Proceedings of the Seventh ACM Symposium on Cloud Computing*, 484–496. <https://doi.org/10.1145/2987550.2987576>
- [20] Amazon Web Services. (2017, November). *Amazon EC2 spot introduces new pricing model and the ability to launch spot instances via RunInstances API*. Retrieved March 18, 2026, from <https://aws.amazon.com/about-aws/whats-new/2017/11/amazon-ec2-spot-introduces-new-pricing-model-and-the-ability-to-launch-new-spot-instances-via-runinstances-api/>
- [21] Sharma, P., Irwin, D., & Shenoy, P. (2017). Portfolio-driven resource management for transient cloud servers. *Proc. ACM Meas. Anal. Comput. Syst.*, 1(1). <https://doi.org/10.1145/3084442>
- [22] Singh, R., Sharma, P., Irwin, D., Shenoy, P., & Ramakrishnan, K. K. (2017). Optimizing the cost of executing mixed interactive and batch workloads on transient VMs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 1(2), 1–29. <https://doi.org/10.1145/3154499>
- [23] Yang, Y., Kim, G.-W., Song, W. W., Lee, Y., Chung, A., Qian, Z., Cho, B., & Chun, B.-G. (2017). Pado: A data processing engine for harnessing transient resources in datacenters. *Proceedings of the Twelfth EuroSys Conference (EuroSys '17)*, 575–588. <https://doi.org/10.1145/3064176.3064181>
- [24] Cai, Z., Li, X., Ruiz, R., & Li, Q. (2018). Price forecasting for spot instances in cloud computing. *Future Generation Computer Systems*, 79, 38–53.
- [25] Harlap, A., Chung, A., Tumanov, A., Ganger, G. R., & Gibbons, P. B. (2018). Tributary: Spot-dancing for elastic services with latency SLOs. *2018 USENIX Annual Technical Conference (USENIX ATC '18)*. <https://www.usenix.org/conference/atc18/presentation/harlap>
- [26] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 561–577. <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [27] Shastri, S., & Irwin, D. (2018). Cloud index tracking: Enabling predictable costs in cloud spot markets. *Proceedings of the ACM Symposium on Cloud Computing*, 451–463.

- [28] Carver, B., Zhang, J., Wang, A., & Cheng, Y. (2019). In search of a fast and efficient serverless DAG engine. *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, 1–10. <https://doi.org/10.1109/PDSW49588.2019.000005>
- [29] Deelman, E., Vahi, K., Rynge, M., Mayani, R., da Silva, R. F., Papadimitriou, G., & Livny, M. (2019). The evolution of the Pegasus workflow management software. *Computing in Science and Engineering*, 21(4). <https://doi.org/10.1109/MCSE.2019.2919690>
- [30] Joaquim, P., Bravo, M., Rodrigues, L., & Matos, M. (2019). Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud. *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys '19)*. <https://doi.org/10.1145/3302424.3303964>
- [31] Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N. J., Gonzalez, J. E., Popa, R. A., Stoica, I., & Patterson, D. A. (2019). *Cloud programming simplified: A Berkeley view on serverless computing* (tech. rep. No. UCB/EECS-2019-3). EECS Department, University of California, Berkeley.
- [32] Martinez, R. G., Lopes, A., & Rodrigues, L. (2019). Planning workflow executions when using spot instances in the cloud. *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 310–317. <https://doi.org/10.1145/3297280.3297313>
- [33] Sharma, P., Ali-Eldin, A., & Shenoy, P. (2019). SpotWeb: Running latency-sensitive distributed web services on transient cloud servers. *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '19)*, 1–12. <https://doi.org/10.1145/3307681.3325402>
- [34] Vahi, K., Mayani, R., Mandal, A., Ferreira da Silva, R., & Deelman, E. (2019). Distributed workflow management for LIGO data analysis at scale. *2019 15th International Conference on eScience (eScience)*, 179–188. <https://doi.org/10.1109/eScience.2019.00027>
- [35] Varshney, P., & Simmhan, Y. (2019). AutoBoT: Resilient and cost-effective scheduling of a bag of tasks on spot VMs. *IEEE Transactions on Parallel and Distributed Systems*, 30(7), 1512–1527. <https://doi.org/10.1109/TPDS.2018.2889851>
- [36] Ambati, P., Goiri, Í., Frujeri, F., Gun, A., Wang, K., Dolan, B., Corell, B., Pasupuleti, S., Moscibroda, T., Elnikety, S., Fontoura, M., & Bianchini, R. (2020). ExoSphere: A schedulable parallel virtualization substrate for spot datacenters. *14th USENIX Symposium on Operating Systems Design*

- and Implementation (OSDI '20)*, 755–770. <https://www.usenix.org/conference/osdi20/presentation/ambati>
- [37] Carver, B., Zhang, J., Wang, A., Anwar, A., Wu, P., & Cheng, Y. (2020). Wukong: A scalable and locality-enhanced framework for serverless parallel computing. *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC '20)*, 1–15. <https://doi.org/10.1145/3419111.3421286>
 - [38] Khandelwal, V., Chaturvedi, A. K., & Gupta, C. P. (2020). Amazon EC2 spot price prediction using regression random forests. *IEEE Transactions on Cloud Computing*, 8(1), 59–72. <https://doi.org/10.1109/TCC.2017.2780159>
 - [39] Malawski, M., Gajek, A., Zima, A., Balis, B., & Figiela, K. (2020). Serverless execution of scientific workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions. *Future Generation Computer Systems*, 110, 502–514. <https://doi.org/10.1016/j.future.2017.10.029>
 - [40] Monge, D. A., Pacini, E., Mateos, C., Alba, E., & García Garino, C. (2020). CMI: An online multi-objective genetic autoscaler for scientific and engineering workflows in cloud infrastructures with unreliable virtual machines. *Journal of Network and Computer Applications*, 149, 102464. <https://doi.org/10.1016/j.jnca.2019.102464>
 - [41] Oikonomou, P., Kolomvatsos, K., Tziritas, N., Theodoropoulos, G., Loukopoulos, T., & Stamoulis, G. (2020). Uncertainty driven workflow scheduling using unreliable cloud resources. *2020 IEEE 19th International Symposium on Network Computing and Applications (NCA)*, 1–8. <https://doi.org/10.1109/NCA51143.2020.9306707>
 - [42] Servia-Rodríguez, S., Lee, Y. S., Mao, S., Madhyastha, H. V., Lo, J. C., & Madhyastha, Y. C. (2020). SpotTune: Leveraging transient resources for cost-efficient hyper-parameter tuning in the public cloud. *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 1003–1013. <https://doi.org/10.1109/ICDCS47774.2020.00009>
 - [43] Daw, N., Bellur, U., & Kulkarni, P. (2021). Speedo: Fast dispatch and orchestration of serverless workflows. *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, 585–599. <https://doi.org/10.1145/3472883.3486982>
 - [44] Gu, H., Li, X., Liu, M., & Wang, S. (2021). Scheduling method with adaptive learning for microservice workflows with hybrid resource provisioning. *International Journal of Machine Learning and Cybernetics*, 12(10), 3037–3048. <https://doi.org/10.1007/s13042-021-01396-4>
 - [45] Jia, Z., & Witchel, E. (2021). Nightcore: Efficient and scalable serverless computing for latency-sensitive, interactive microservices. *Proceedings*

- of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21), 152–166. <https://doi.org/10.1145/3445814.3446701>
- [46] Majewski, M., Pawlik, M., & Malawski, M. (2021). Algorithms for scheduling scientific workflows on serverless architecture. *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CC-Grid)*, 782–789. <https://doi.org/10.1109/CCGrid51090.2021.00095>
 - [47] Burckhardt, S., Chandramouli, B., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C., & Zhu, X. (2022). Netherite: Efficient execution of serverless workflows. *Proceedings of the VLDB Endowment*, 15(8), 1591–1604. <https://doi.org/10.14778/3529337.3529344>
 - [48] Ferreira da Silva, R., Filgueira, R., Pietri, I., Jiang, M., Sakellariou, R., & Deelman, E. (2022). Pegasus and Epigenomics: Workflow management for large-scale genomics analysis. *Bioinformatics*, 38(3), 866–872. <https://doi.org/10.1093/bioinformatics/btab784>
 - [49] Kadupitiya, J., Jadhao, V., & Sharma, P. (2022). SciSpot: Scientific computing on temporally constrained cloud preemptible VMs. *IEEE Transactions on Parallel and Distributed Systems*, 33(12), 3575–3588. <https://doi.org/10.1109/TPDS.2022.3157272>
 - [50] Lee, S., Hwang, J., & Lee, K. (2022). Spotlake: Diverse spot instance dataset archive service, 242–255. <https://doi.org/10.1109/IISWC55918.2022.00029>
 - [51] Mahgoub, A., Barsallo Yi, E., Shankar, K., Elnikety, S., Chaterji, S., & Bagchi, S. (2022). ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, 303–320. <https://www.usenix.org/conference/osdi22/presentation/mahgoub>
 - [52] Mahgoub, A., Barsallo Yi, E., Shankar, K., Minocha, E., Elnikety, S., Bagchi, S., & Chaterji, S. (2022). WISEFUSE: Workload characterization and DAG transformation for serverless workflows. *ACM SIGMETRICS Conference (SIGMETRICS)*.
 - [53] Roy, R. B., Patel, T., Gadepally, V., & Tiwari, D. (2022). Mashup: Making serverless computing useful for HPC workflows via hybrid execution. *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*, 46–60. <https://doi.org/10.1145/3503221.3508407>
 - [54] Roy, R. B., Patel, T., & Tiwari, D. (2022). DayDream: Executing dynamic scientific workflows on serverless platforms with hot starts. *SC22*:

- International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–18. <https://doi.org/10.1109/SC41404.2022.00027>
- [55] Saxena, D., & Singh, A. K. (2022). OFP-TM: An online VM failure prediction and tolerance model towards high availability of cloud computing environments. *The Journal of Supercomputing*, 78, 8003–8024. <https://doi.org/10.1007/s11227-021-04235-z>
- [56] Tarafdar, A., Das, R. K., & Khatua, S. (2022). Deadline-sensitive workflow scheduling with on-demand and spot instances in the cloud. *2022 OITS International Conference on Information Technology (OCIT)*, 474–479. <https://doi.org/10.1109/OCIT56763.2022.00094>
- [57] Bilal, M., Canini, M., Fonseca, R., & Rodrigues, R. (2023). With great freedom comes great opportunity: Rethinking resource allocation for serverless functions. *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*, 381–397. <https://doi.org/10.1145/3552326.3567506>
- [58] Portella, F. A., Estrela, P. J. B., Malini, R. Q., Teylo, L., Berral, J. L., & Drummond, L. M. A. (2023). MScheduler: Leveraging spot instances for high-performance reservoir simulation in the cloud. *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. <https://doi.org/10.1109/CloudCom59040.2023.00028>
- [59] Teylo, L., Arantes, L., Sens, P., & Drummond, L. M. d. A. (2023). Scheduling bag-of-tasks in clouds using spot and burstable virtual machines. *IEEE Transactions on Cloud Computing*, 11(1), 984–998. <https://doi.org/10.1109/TCC.2021.3125426>
- [60] Thorpe, J., Zhao, P., Eyolfson, J., Qiao, Y., Jia, Z., Zhang, M., Netravali, R., & Xu, G. H. (2023). Bamboo: Making preemptible instances resilient for affordable training of large DNNs. *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 497–513. <https://www.usenix.org/conference/nsdi23/presentation/thorpe>
- [61] Tomar, S., Mishra, A. K., & Yadav, D. K. (2023). Knowledge-based checkpointing strategy for spot instances in cloud computing. *Journal of Current Science and Technology*, 13(2), 412–427. <https://doi.org/10.59796/jcst.V13N2.2023.1754>
- [62] Yang, F., Wang, L., Xu, Z., Zhang, J., Li, L., Qiao, B., Couturier, C., Bansal, C., Ram, S., Qin, S., Ma, Z., Goiri, Í., Cortez, E., Yang, T., Rühle, V., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Snape: Reliable and low-cost computing with mixture of spot and on-demand VMs. *Proceedings of the 28th ACM International Conference on Architectural Support for*

- Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)*. <https://doi.org/10.1145/3582016.3582028>
- [63] Apache Software Foundation. (2024). *Apache OpenWhisk: Open source serverless cloud platform*. <https://openwhisk.apache.org/>
 - [64] Basu Roy, R., & Tiwari, D. (2024). StarShip: Mitigating I/O bottlenecks in serverless computing for scientific workflows. *Abstracts of the 2024 ACM SIGMETRICS/IFIP PERFORMANCE Joint International Conference*, 79–80. <https://doi.org/10.1145/3652963.3655082>
 - [65] Callaghan, S., Maechling, P. J., Goulet, C. A., Milner, K. R., Graves, R. W., Olsen, K. B., Cui, Y., Vahi, K., Rynge, M., Ferreira da Silva, R., Deelman, E., & Jordan, T. H. (2024). CyberShake: Running seismic hazard workflows on distributed high-performance computing resources. *Proceedings of the 20th IEEE International Conference on e-Science (e-Science '24)*. <https://doi.org/10.5072/par-10630254>
 - [66] Chen, L., Liu, G., Zhang, J., & Zhang, X. (2024). Scheduling ensemble workflows on hybrid resources in IaaS clouds. *Computing*, 107(1). <https://doi.org/10.1007/s00607-024-01386-8>
 - [67] Cho, Y., Kim, Y., Kim, K., Kim, J., Kim, H.-Y., & Kim, Y. (2024). Optimizing multi-level checkpointing for distributed deep learning workloads on cloud spot VM clusters. *IEEE Access*, 12, 116891–116904. <https://doi.org/10.1109/ACCESS.2024.3446770>
 - [68] Duan, J., Song, Z., Miao, X., Xi, X., Lin, D., Xu, H., Zhang, M., & Jia, Z. (2024). Parcae: Proactive, liveput-optimized DNN training on preemptible instances. *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*.
 - [69] Kohli, S., Kharbanda, S., Bruno, R., Carreira, J., & Fonseca, P. (2024). Pronghorn: Effective checkpoint orchestration for serverless hot-starts. *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, 298–316. <https://doi.org/10.1145/3627703.3629556>
 - [70] Liu, F., & Niu, Y. (2024). Demystifying the cost of serverless computing: Towards a win-win deal. *IEEE Transactions on Parallel and Distributed Systems*, 35(1), 59–72. <https://doi.org/10.1109/TPDS.2023.3330849>
 - [71] Miao, X., Shi, C., Duan, J., Xi, X., Lin, D., Cui, B., & Jia, Z. (2024). Spot-Serve: Serving generative large language models on preemptible instances. *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 1112–1127. <https://doi.org/10.1145/3620666.3651335>
 - [72] Nag, U., Sharan, A., & Kalra, M. (2024). Cost-deadline constrained robust scheduling of workflows using hybrid instances in IaaS cloud. *Artificial*

- Intelligence of Things*, 227–240. https://doi.org/10.1007/978-3-031-48774-3_16
- [73] Son, M., Akbulut, G. G., & Kandemir, M. T. (2024). SpotVerse: Optimizing bioinformatics workflows with multi-region spot instances in Galaxy and beyond. *Proceedings of the 25th International Middleware Conference*, 74–87. <https://doi.org/10.1145/3652892.3700750>
 - [74] Triendl, S., & Ristov, S. (2024). Peeking behind the serverless implementations and deployments of the Montage workflow. *Companion of the 15th ACM/SPEC International Conference on Performance Engineering (ICPE '24)*, 196–203. <https://doi.org/10.1145/3629527.3651420>
 - [75] Wang, Y., Zuo, X., Wang, H., & Wu, Z. (2024). Real-time dynamic cloud workflow scheduling for volatile resources in clouds [Preprint available at SSRN]. <https://doi.org/10.2139/ssrn.4903286>
 - [76] Wu, Z., Chiang, W.-L., Mao, Z., Yang, Z., Friedman, E., Shenker, S., & Stoica, I. (2024). Can't be late: Optimizing spot instance savings under deadlines. *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 185–203. <https://www.usenix.org/conference/nsdi24/presentation/wu-zhanghao>
 - [77] Zhang, Z., Jin, C., & Jin, X. (2024). Jolteon: Unleashing the promise of serverless for serverless workflows. *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, 1469–1487. <https://www.usenix.org/conference/nsdi24/presentation/zhang-zili-jolteon>
 - [78] Duan, J., Xu, S., Qian, S., Yang, D., Wang, K., Liao, C., Yu, Y., Hua, Q., Hu, H., Wang, Q., Wu, W., Bao, D., Lu, T., Cao, J., Xue, G., Yang, G., Zhang, L., & Chen, G. (2025). GFS: A preemption-aware scheduling framework for GPU clusters with predictive spot instance management. *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 117–131. <https://doi.org/10.1145/3760250.3762231>
 - [79] Mao, Z., Xia, T., Wu, Z., Chiang, W.-L., Griggs, T., Bhardwaj, R., Yang, Z., Shenker, S., & Stoica, I. (2025). SkyServe: Serving AI models across regions and clouds with spot instances. *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 159–175. <https://doi.org/10.1145/3689031.3717459>
 - [80] Samanta, A., Stutsman, R., & Roy, R. B. (2025). GridGreen: Integrating serverless computing in HPC systems for performance and sustainability. *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25)*, 388–401. <https://doi.org/10.1145/3772052.3772241>

- [81] Schmid, L., Copik, M., Calotoiu, A., Brandner, L., Koziolk, A., & Hoefler, T. (2025). SeBS-Flow: Benchmarking serverless cloud function workflows. *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. <https://doi.org/10.1145/3689031.3717465>
- [82] Sinha, A., Li, Z., Liu, T., Wang, S., Xiao, N., & Zhou, H. (2025). Fed-CostAware: Enabling cost-aware federated learning on the cloud. <https://arxiv.org/abs/2505.21727>
- [83] Tang, W., Yang, Y., & Wu, J. (2025). Metis: A non-clairvoyant, workflow-aware OS scheduler for serverless applications. *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25)*, 149–162. <https://doi.org/10.1145/3772052.3772221>
- [84] Zhu, B., Zhu, Y., Chen, C., & Kong, L. (2025). Trident: A provider-oriented resource management framework for serverless computing platforms. *IEEE Transactions on Services Computing*, 18(5), 3334–3347. <https://doi.org/10.1109/TSC.2025.3603867>
- [85] Amazon Web Services. (2026). *Spot instance interruption notices*. Retrieved March 18, 2026, from <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>
- [86] Apache Software Foundation. (2026). *Apache airflow documentation*. Retrieved March 18, 2026, from <https://airflow.apache.org/docs/index.html>
- [87] Bhuyan, N., Bhatia, R., Kodialam, M., & Lakshman, T. V. (2026). Opportunistic scheduling for optimal spot instance savings in the cloud. <https://arxiv.org/abs/2601.12266>
- [88] Fission Project. (2026). Fission: Open source Kubernetes-native serverless framework.
- [89] Kim, K., Song, M., Kim, T., & Lee, K. (2026). Ding-dong ditch: Peeking into spot instance availability. <https://arxiv.org/abs/2604.16457>
- [90] Li, Z., Xia, T., Mao, Z., Zhou, Z., Jackson, E. J., Kerney, J., Wu, Z., Mishra, P., Xu, Y., Qiao, Y., Shenker, S., & Stoica, I. (2026). SkyNomad: On using multi-region spot instances to minimize AI batch job cost. <https://doi.org/10.48550/arXiv.2601.06520>
- [91] Temporal Technologies. (2026). *Temporal platform documentation*. Retrieved March 18, 2026, from <https://docs.temporal.io/>
- [92] The Kubernetes Authors. (2026). Kubernetes.